Three-tier architecture and API contract

docs/platform/THREE_TIER_ARCHITECTURE.md

Three-tier architecture and strategic borrowing

Semester platform maturity program · 2026-10-05 · Source: uploads/Apple vs Google vs Microsoft in EdTech build a st.pdf._

Sell the unified Education OS, deploy a focused pilot, expand as a trusted institutional platform, and mature into an open ecosystem where students, institutions, developers, employers and partners all gain from the same connected graph.

Tier 1 · Experience platform

Apple-like: calm, clear, fast, accessible, coherent.

- Student OS
- Faculty Course Studio
- Advisor workspace
- Registrar tools
- Campus services
- Family portal
- Career/alumni
- Company site
- Operations console

Tier 2 · Education intelligence platform

Google-like: data, search, AI, source authority and recommendations working together.

- Identity graph
- Education graph
- Search
- AI gateway
- Policy engine
- Workflow engine
- Evidence/provenance
- Analytics
- Recommendations

- Outcome models
- AI evaluation
- Human escalation

Tier 3 · Institutional infrastructure platform

Microsoft-like: safe to buy, configure, deploy, govern, integrate and operate for years.

- Tenant control plane
- RLS and authorization
- SSO/SCIM
- Roles/capabilities
- Approval workflows
- Audit and evidence
- Integration gateway
- LTI/OneRoster/Edu-API
- Migration and reconciliation
- Feature flags and rollout
- Billing
- Trust/compliance
- SLOs/incidents
- Backup/restore/offboarding
- Developer ecosystem

What to borrow, and what not to copy

Model	Borrow	Semester equivalent	Do not copy
Apple	Integrated experience, design discipline, privacy posture, premium support	Calm student OS, cohesive navigation, accessibility, offline-safe, student-controlled sharing, quality mobile	Closed ecosystem, forced lock-in, opaque platform rules
Google	Search, cloud, identity, AI, data systems, developer distribution	Education search, education graph, membership layer, AI gateway, APIs, partners	Ad-tech incentives on student data, uncontrolled aggregation
Microsoft	Enterprise identity, productivity, administration, procurement readiness, long contracts	Tenant admin, SSO/SCIM, workspace, compliance controls, implementation, annual licensing, marketplace	Complex enterprise UX, configuration overload, fragmented surfaces
Canvas/Brightspace	Course workflow and institutional deployment	Course Studio connected to planning, actions, support, AI, career	Course experience isolated from the wider journey
SIS/ERP vendors	Record authority, policy, transaction integrity	Canonical academic domains, deterministic rules, audit, migration, reconciliation, rollback	Legacy UX, closed integrations
ChatGPT/Claude/Perplexity	Conversational intelligence and research	Governed copilot, policy-aware routing, citations, academic context, human escalation	Treating model output as official truth

Deployment ladder

Development → Tenant-safe test environment → Sandbox integration → Preview deployment → Internal beta → Private beta tenant → Pilot tenant → Controlled production cohort → Annual customer deployment → Multi-campus expansion

Each step requires: Configuration, Policy approval, Security review, Accessibility review, Data validation, Integration test, Rollback plan, Support readiness, Monitoring, Release evidence, Owner approval.

Expansion stages

1. Daily student OS + registration readiness
2. Course Studio + advising/support workflows
3. Institution control plane + integrations + rollout factory
4. Native registrar/academic domains where authorized
5. Campus services + student account + career + family
6. Developer platform + marketplace + global ecosystem
7. Education infrastructure company: identity, cloud, AI, operating layer, standards, partners, global, lifelong learners

---

`docs/platform/API_CONTRACT_STANDARD.md`

API domains and contract standard

Semester platform maturity program · 2026-10-05 · Source: uploads/Apple vs Google vs Microsoft in EdTech build a st.pdf.

APIs are organized by authority and risk, not by frontend page. High-impact writes never mutate raw tables from the browser; they go through capability-checked server commands.

Domains

API	Primary resources	Write-control model
Identity	Accounts, sessions, memberships, SSO, SCIM, roles	Server-authorized; auditable
Tenant	Institutions, campuses, policies, configurations, feature state	Tenant admin + approval path
Academic	Terms, catalog, sections, enrollment, holds, records, grades	Deterministic policy + registrar authority
Learning	Courses, objectives, resources, assignments, assessments, study packs	Faculty/course-scoped capabilities
Workspace	Tasks, notes, files, calendar, plans, projects	Student ownership + sharing grants
AI	Requests, policy, providers, sources, usage, evaluations	Gateway-enforced, tenant/course policy
Student success	Caseload, check-ins, referrals, success plans, support	Purpose-limited, consent/role-scoped
Campus	Dining, housing, events, organizations, spaces	Service-owner policy + tenant scope
Career	Skills, evidence, credentials, opportunities, mentors	Student consent + verified claim rules
Family	Grants, invitations, scoped shares, revocation	Student-controlled, time-limited
Integrations	Connections, scopes, mappings, syncs, webhooks	Approval-gated, server-only credentials

API	Primary resources	Write-control model
Commercial	Quotes, contracts, subscriptions, invoices, entitlements	Finance capability + approvals
Trust	Controls, evidence, claims, artifacts, trust-room grants	Security/legal scoped
Operations	Approvals, incidents, SLOs, audits, rollouts, risks	Internal capabilities, two-person controls
Platform	Apps, API keys, OAuth clients, webhooks, marketplace	Tenant install approval + developer governance

Every API answers

Check	Question
Identity	Who is calling?
Scope	Which tenant, customer, course or person may be accessed?
Purpose	Why is this data or action needed?
Authority	Official, derived, AI-generated, user-owned or external?
Policy	Which rules constrain it?
Validation	Are inputs structurally and semantically valid?
Idempotency	Can a retry create duplicate harm?
Audit	What event must be recorded?
Freshness	Live, delayed, stale or unavailable?
Error	What safe recovery or human hand-off is available?

Example resources

```
 GET /v1/identity/me
GET /v1/tenants/{tenantId}
GET /v1/tenants/{tenantId}/policy
GET /v1/academic/terms
GET /v1/academic/sections
POST /v1/academic/registration-requests
POST /v1/academic/registration-requests/{id}/decision
GET /v1/learning/courses/{courseId}
GET /v1/learning/gradebook
GET /v1/workspace/tasks
POST /v1/ai/requests
GET /v1/ai/sources
POST /v1/support/requests
GET /v1/advising/caseload
GET /v1/career/opportunities
POST /v1/family/grants
GET /v1/integrations/sync-runs
POST /v1/integrations/{id}/scope-approval
POST /v1/commercial/quotes/{id}/send
POST /v1/ops/approval-requests/{id}/decision
POST /v1/privacy/data-subject-requests/{id}/complete
GET /v1/platform/apps
```

Architecture cards

Front end

Student app
Five destinations, 64 screens

Staff + console
Same shell, role-scoped routes

Design system
Tokens → semantic CSS → components

Offline
Queue mutations, replay on reconnect

Middleware

Auth proxy
Refresh session; reject expired

Tenant resolver
Host/SSO → tenant_id on every request

Capability check
role × screen × grant first

Audit writer
Event written before the action runs

Server / API

Query RPCs
api.* reads of read models

Commands
private.* fns; idempotency key

Integrations
LTI 1.3, OneRoster, SIS read, SAML

AI gateway
Scope, policy, citations, spend log

Database

Supabase Postgres
RLS on every table

tenant_id
On every row

private.*
Authoritative, never exposed

read_model.*
Projections + version + freshness

arch layers

WRITE → READ

1 write		2 outbox		3 project		4 stamp		5 invalidate		6 signal		7 render
Authoritative write	→	Event, same transaction	→	Worker applies, idempotent	→	Version + freshness	→	Cache namespaces	→	Targeted refetch	→	Server read model

FRESHNESS → UI POLICY (FRESHNESS.TS)

State	Lag	Display	Consequential actions
Live	≤ 5 s	Normal	Allowed if approvals satisfied
Recent	≤ 60 s	Normal	Normal guardrails
Delayed	> 60 s	Delay badge + age	Re-check source first
Failed / Rebuilding	—	Last good + notice	Held

arch read model

Request		Identity + scope		Policy		Routing		Answer		Evidence
Student, staff or institution asks	→	Who; which data they may read	→	Course AI policy, institution switches, FERPA boundary	→	Approved model per task; search, calendar, materials	→	Labelled AI-assisted, sources, confidence	→	Citations, cost, outcome logged

arch ai gateway

Environment	Hosting	Data	Access
Local	Supabase CLI + Next.js dev	Synthetic seed	Engineers
Staging	Vercel preview + Supabase staging	Synthetic + institution test IdP accounts	Engineers, institution IT testers
Production	Vercel production + Supabase production (separate keys)	Real student records under the DPA	Named operators, MFA, access logged

Rule: no copying data between environments; secrets set per environment. Cutover steps: docs/launch/Cutover Plan.html.

arch environments

Master control

ui_kits/master-control/ — product, console & back end, business OS, company site, design system & engineering

Rule
Every new file is linked here

Master catalog

51 roles · 673 screens · 319 workflow steps (17 workflows) · 88 systems · 82 docs

Rule
Status from evidence: code, tests, release

Role lens

64 screens × 14 roles: rendering, sources, tables, RLS, API

Access levels
full · scoped · shared · read · audited · none

Rule: company roles (investor, CS, RevOps) never see student records. Role model in roles.js is design-level; verify against migrations.

maps of the whole

Architecture explorer

Interactive architecture of stack layers, read models, AI gateway and environments. **11 screens.**



Opening view

SEMESTER

Education OS map | Full stack & data | Read-model pipeline | AI gateway | Lag simulator | Migration phases

Architecture
Design-level · verify in repo
Full stack & dataSemester platform

Search or ask… ⌘K

○On this device only
Architecture · Full stack & data

Full stack & data

Front end | Middleware | Server / API | Database

RuleRLS on every table; tenant_id on every row; browser calls api.* only. Design-level schema — reconcile with supabase/migrations.

Table	Key columns	RLS: who can read	Domain
tenants	id, slug, name, plan, sso_entity_id, created_at	admin of tenant	Tenant
memberships	user_id, tenant_id, role, status, granted_by, expires_at	self + tenant admin	Identity
role_grants	id, user_id, tenant_id, capability, scope_kind, scope_id, expires_at	tenant admin; audited	Authorization
people	id, tenant_id, display_name, pronouns, source	self; scoped staff	Profile
courses	id, tenant_id, term_id, code, title, source, synced_at	enrolled; instructors	Learning
sections	id, course_id, instructor_id, capacity, waitlist	enrolled; registrar	Registration
enrollments	person_id, section_id, status, source, version	owner; registrar	Registration
assignments	id, course_id, title, due_at, weight, source	enrolled; instructors	Learning
grades	person_id, assignment_id, score, released_at, version	owner after release; instructors	Gradebook
holds	person_id, kind, blocks, source, synced_at	owner; registrar; finance	Records
items	id, owner_id, kind, title, due_at, origin, done, snoozed_until	owner	Today
shares	id, owner_id, grantee_id, screen, expires_at, revoked_at	owner; grantee read	Consent
share_events	share_id, actor_id, action, at	owner read	Consent
support_requests	id, requester_id, office, severity, status	requester; office	Support
approvals	id, duty, requester_id, approvers[], state, expires_at	approvers; audited	Governance
audit_log	seq, tenant_id, actor_id, event, target, authority, at	append-only; auditors	Audit
domain_outbox_events	id, tenant_id, type, payload, created_at, claimed_at, processed_at, attempts	server only	Read models
read_model.tenant_overview_v1	tenant_id, version, applied_at, metrics jsonb	api RPC only	Read models
ai_requests	id, user_id, task, scope_ids[], dropped_ids[], cost, model, at	owner; security	AI
integrations	id, tenant_id, provider, scopes, status, last_sync_at	tenant IT	Integration

Request flows, end to end

Flows

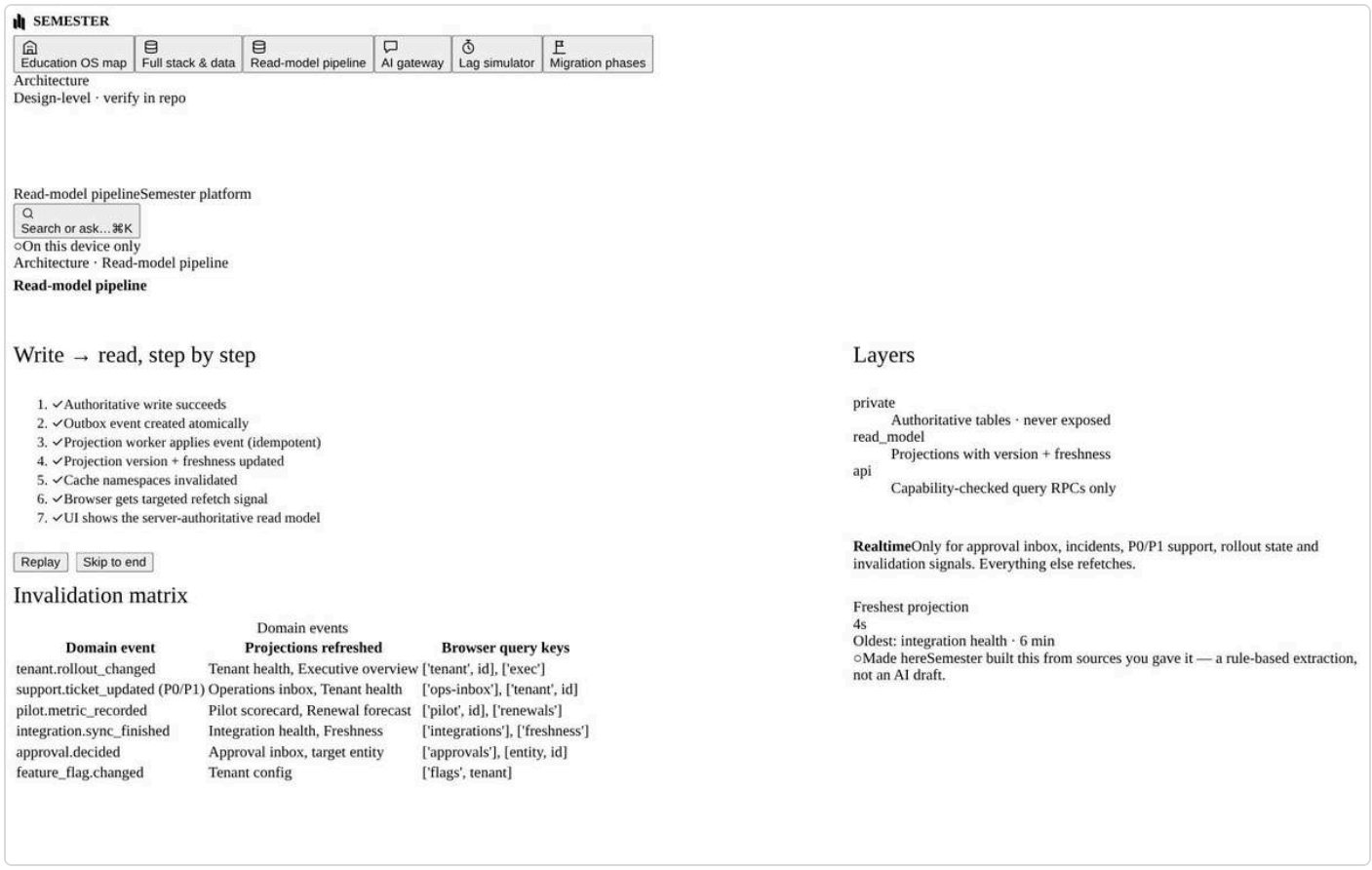
Flow	Path
Read	Browser → auth proxy → tenant + capability → api.rpc → read_model → response with version + freshness
Write	Browser (optimistic) → command fn → validate + RLS → domain table + outbox (one transaction) → audit → worker → projection → invalidate keys → UI confirms
Sync	Scheduler → edge worker → provider API → staging → map + validate → reconcile → upsert with source + synced_at → outbox
AI	Browser → gateway → scope + policy + classification → drop student records → model → citations → log spend → AIResponse
Approval	Request → approver 1 (MFA ≤15 min) → approver 2 → requester executes → audit before change

Operations

Operations

Practice	Execution	Owner
Deploy	PR → CI gate → preview → staged tenant rollout → full	Engineering
Migrations	Reviewed SQL, forward-only, backfill job, parity check	Engineering
Monitoring	Errors, uptime, projection lag, sync failures, AI spend	Engineering
On-call	P0 1 h · P1 4 h; runbook per alert	Founder → contractor
Backups	Daily + PITR; quarterly restore test with RTO/RPO	Engineering
Access review	Quarterly; contractor offboarding same day	Security
Incident	Commander, tech lead, comms; postmortem in 5 days	Founder

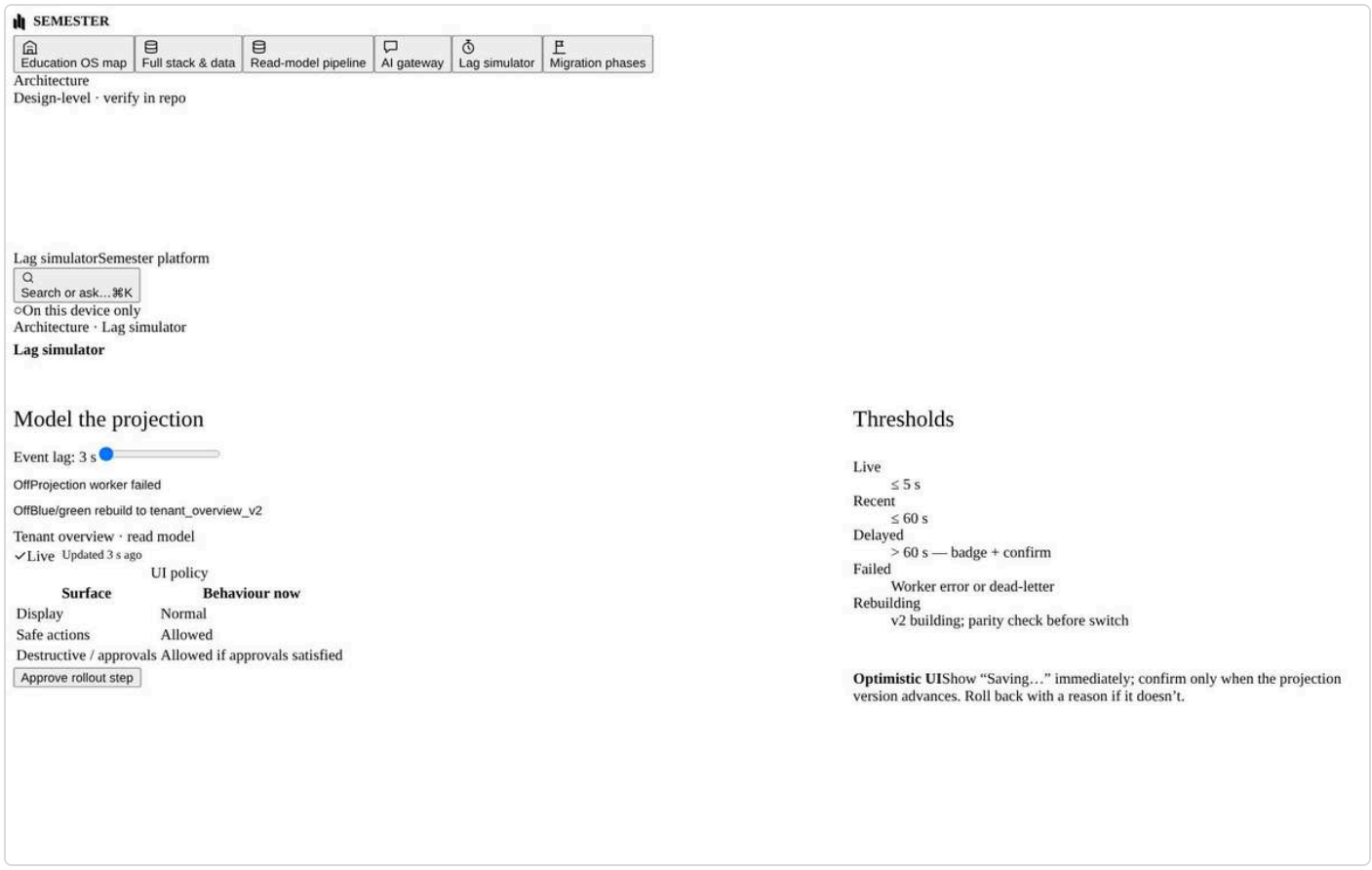
Full stack & data



Read-model pipeline


SEMESTER
Education OS map
Full stack & data
Read-model pipeline
AI gateway
Lag simulator
Migration phases
Architecture
Design-level · verify in repo
AI gatewaySemester platform
Search or ask…⌘K
○On this device only
Architecture · AI gateway
AI gateway
1
Request
Student / staff / institution asks
○Student enteredYou typed this in. Semester has not checked it with your institution.
2
Identity + scope
Who is asking; which data they may read
◆Institution verifiedConfirmed by your institution’s own system.
3
Policy check
Course AI policy, institution switches, FERPA boundary — student records never leave
◆Institution verifiedConfirmed by your institution’s own system.
4
Model + tool routing
Approved model per task; tools: search, calendar, course materials
○Made hereSemester built this from sources you gave it — a rule-based extraction, not an AI draft.
5
Answer
Labelled AI-assisted, numbered sources, confidence
✦AI-assistedWritten with Semester’s assistant. It is not an official answer — check anything you act on.
6
Evidence + logs
Citations, cost, outcome logged for audit
◆Institution verifiedConfirmed by your institution’s own system.
Model-flexible, Semester-governedAny approved model can answer; Semester decides what it may read, labels what it wrote, and keeps the evidence.
✦AI-assistedWritten with Semester’s assistant. It is not an official answer — check anything you act on.Semester assistant§Outlining allowed
Example answer leaving the gateway.
1. 1BIO 210 Syllabus◆Institution verifiedConfirmed by your institution’s own system.
Allowed to read BIO 210 materialsConfidence HighRequest cost 1 request

AI gateway



Lag simulator



Migration phases



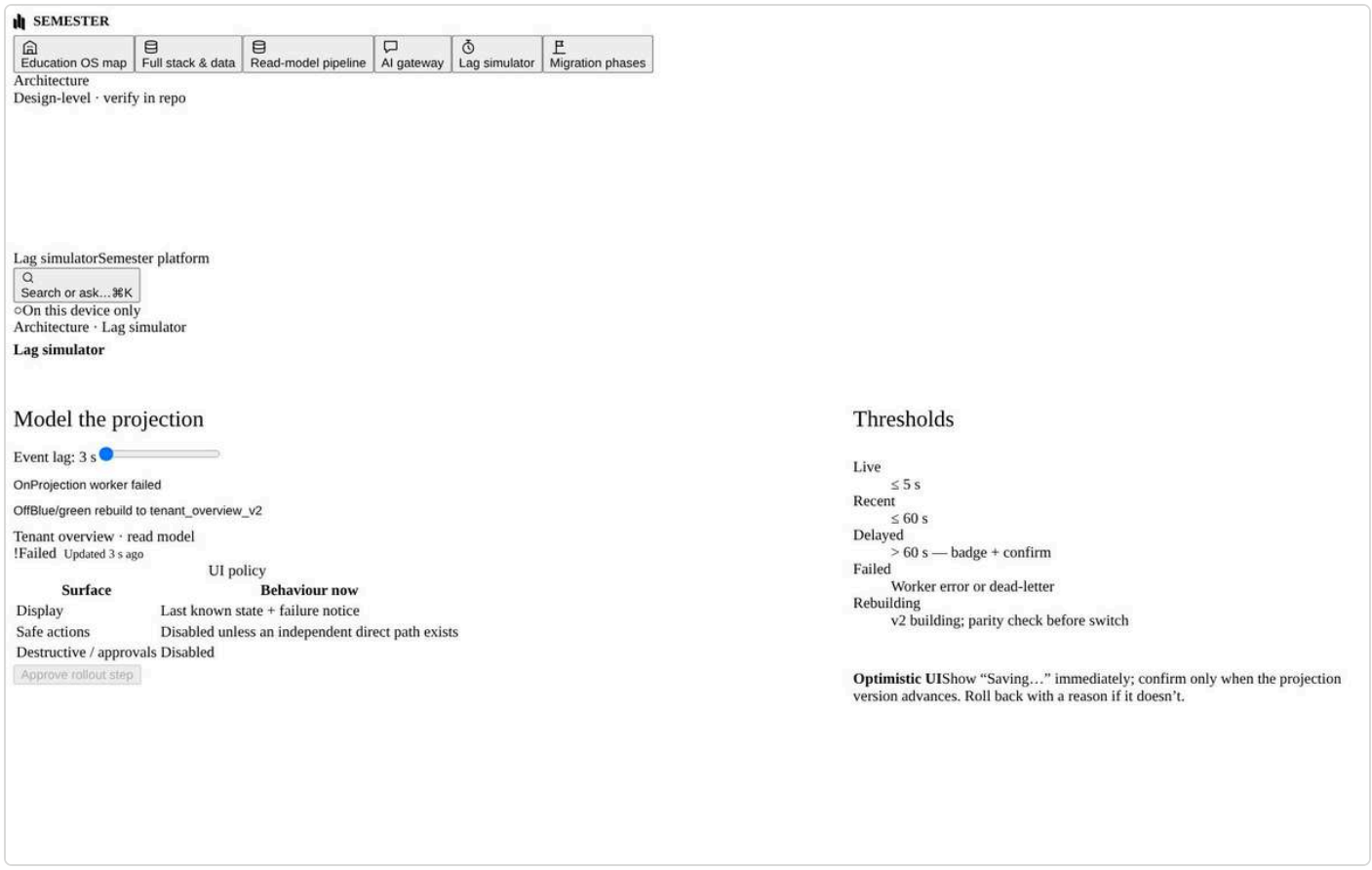
Full stack & data › Front end



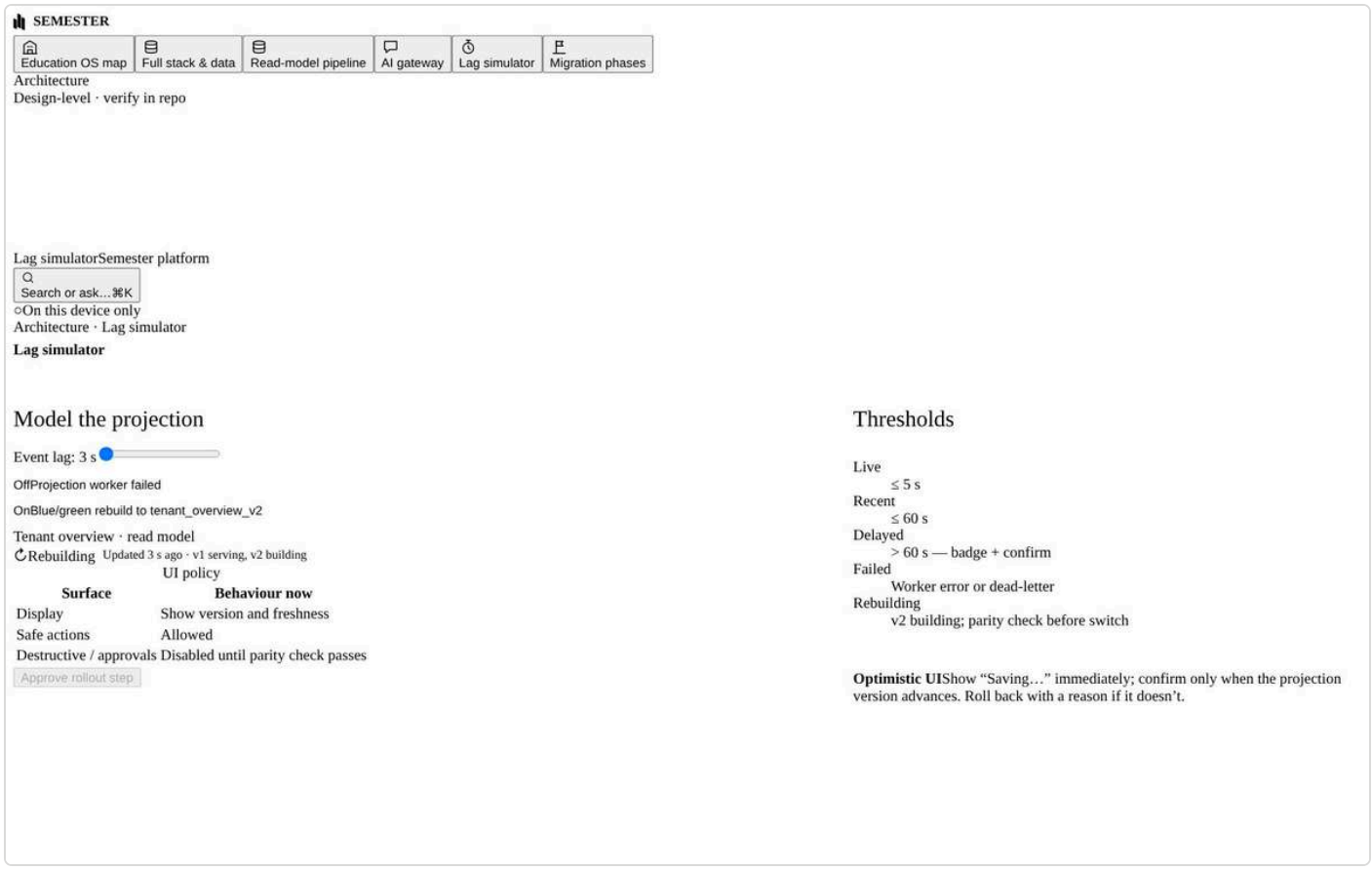
Full stack & data › Middleware



Full stack & data › Server / API



Lag simulator › OffProjection worker failed



Lag simulator › OffBlue/green rebuild to tenant_overview_v2

SEMESTER

Developer guide

Set up, build, test and ship. v0.1, October 2026.

1. Repos and code in this project

- `harrisonjrubin7-cmyk/semester`: the product app (ground truth).
- `handoff/semester-platform/`: database migrations with RLS, typed API routes, and tests for community, marketplace and ops (new).
- `handoff/semester-core/`: pure logic (access, AI gateway, approvals, freshness, provenance), with tests.
- `handoff/claude-code-toolkit/`: design-system sync skills and the audit script.
- `handoff/workflow-router-app/`: a full Next.js + Supabase reference app.

Code files in this project end in `.txt` so they don't execute here. Strip the suffix when you copy them into a repo.

2. Local setup

```
git clone git@github.com:harrisonjrubin7-cmyk/semester.git && cd semester
npm install
npx supabase start            # local Postgres + auth
npx supabase db reset         # runs migrations + seed
cp .env.example .env.local    # fill SUPABASE_URL / ANON_KEY
npm run dev
```

3. Conventions

- TypeScript strict. Validate every input with Zod at the boundary.
- UI uses design-system tokens only (`styles.css`). No raw hex values in components.
- Every query runs as the user (anon key + JWT). The service role is only for workers.
- Every mutation writes an audit row: actor, role, institution, action, target, before/after.
- Copy follows the readme's voice rules: sentence case, the status sentence, no overclaims.

4. Branching and release

1. Branch from `main`, named `feat/…` or `fix/….`
2. Open a PR. Vercel builds a preview, and CI runs lint, typecheck, unit tests, RLS tests and the design-system audit.
3. One review is required, and the founder approves anything touching migrations.
4. Merge to `main`, which deploys to staging. Promote to production after the smoke check.

5. Testing

Layer	Tool	Must cover
Unit	Vitest	semester-core logic, Zod schemas
RLS	SQL tests (pgTAP or Vitest against local Supabase)	Each role can read only what it should; company roles get zero rows from edu
AI gateway	Vitest red-team suite	edu fields never leave; prompt-injection cases
E2E	Playwright	Sign-in per role, key workflows, marketplace offer flow
Accessibility	axe + manual screen reader pass	WCAG 2.2 AA on each release

6. Adding a feature (checklist)

1. Migration with RLS policies and RLS tests.
2. Zod schema and a typed route handler or server action.
3. UI built from design-system components, with every state: empty, loading, error, offline.
4. Audit write and telemetry event.
5. Feature flag per institution.
6. Docs: update the user guide section and the master catalog status.

SEMESTER

Engineering playbook

Owner: founder / founding engineer. Status: draft, October 2026. Applies to harrisonjrubin7-cmyk/semester. Prompts that execute it: `handoff/execute/`.

Three rules that never bend: RLS on every table with a test that proves it; the browser calls `api.*` only; student records never reach an AI provider.

1. Repository layout

Path	Holds
app/	Student and staff app (Vite + React + TypeScript): src/screens, src/components, src/lib, src/styles
app/src/lib/core/	semester-core: pure decision logic with unit tests
supabase/migrations/	Numbered SQL migrations; never edit one that has been applied
supabase/tests/	RLS and policy tests (pgTAP or SQL test harness)
workflow-lab/	Next.js AI workflow router and benchmark lab
docs/	Architecture, design system, master catalogs, execute reports
design/	Exported design-system project (reference only)
.github/workflows/	CI

2. Branching and pull requests

- **main** is always deployable. No direct pushes.
- Branch per change: `semester/<stream>` for execute streams, `fix/…`, `feat/…` otherwise.
- Small PRs (under ~400 changed lines where possible). Each PR states: what, why, risk, how tested, screenshots for UI, migration notes.
- One approving review. Founder approves anything touching auth, RLS, AI gateway or billing.
- Squash-merge; the PR title becomes the changelog line.

3. CI on every PR

Job	Command	Blocks merge
Typecheck	npx tsc -b	Yes
Lint	npm run lint	Yes
Unit tests	npm test (vitest)	Yes

Job	Command	Blocks merge
Design system	npm run design-system:check	Yes (baseline ratchet)
RLS tests	supabase db reset && supabase test db	Yes
Accessibility	axe on every route in the preview build	Yes for serious/critical
Build	npm run build	Yes
Preview deploy	Vercel preview URL on the PR	—

4. Environments

Environment	App	Database	Data	Deploys
Local	Vite dev	supabase start (CLI)	Synthetic seed	—
Preview	Vercel preview per PR	Supabase branch or shared staging	Synthetic	Every push
Staging	Vercel staging	Supabase staging project	Synthetic + institution test IdP	Merge to main
Production	Vercel production	Supabase production (separate keys)	Real, under DPA	Tagged release, manual promote

Secrets live in Vercel and Supabase environment settings, never in the repo. Production keys are readable only by named operators with MFA.

5. Database changes

1. Write the migration and its RLS policies in the same PR.
2. Add tests: own tenant allowed, other tenant denied, company roles denied on student tables, expired grant denied.
3. Prove each test fails without its policy (red), then passes (green); note it in the PR.
4. Backwards-compatible first (add columns, dual-write), destructive later in a separate release.
5. Run against staging; check the read-model projection still builds.

6. Release process

Step	Who	Evidence
Cut release from main; tag vYYYY.MM.DD-n	Engineer	Tag
Release notes from PR titles	Engineer	CHANGELOG entry
Staging smoke test: sign-in per role, Today, registration readiness, Ask Semester block test	Engineer	Checklist in the release issue
Migration applied to production (if any) with backup taken	Founder	Backup ID

Step	Who	Evidence
Promote to production	Founder	Vercel deployment link
Watch errors and SSO failures for 30 minutes	Engineer	Note in release issue
Roll back if a rollback trigger fires	Founder	Incident log

No releases during a pilot's launch week, finals week, or registration opening unless fixing an S1/S2.

---

7. Feature flags

Every institution-visible feature ships behind a per-tenant flag with a kill switch. Flags default off in production. Remove a flag within two releases of 100% rollout.

---

8. Coding standards

- TypeScript strict; no `any` without a comment saying why.
- Tokens and components from the design system; no raw hex, z-index or durations.
- Every screen handles loading, empty, error, forbidden and offline.
- Consequential actions use preview → audit → apply → undo/request.
- Server-side validation with Zod on every command.
- No student data in logs, error trackers or analytics events; use internal IDs.

---

9. Dependencies and patching

Cadence	Do
Weekly	Review Dependabot alerts; patch critical/high within 7 days
Monthly	Minor version bumps in one PR, full CI
Quarterly	Major upgrades planned; review subprocessors and SDKs

---

10. Definition of done

A change is done when CI is green, tests cover the new behavior, RLS is proven for new tables, the screen passes axe and a keyboard pass, docs and the catalog row are updated, monitoring exists for new failure modes, and it's released with notes.

Product and engineering operations

docs/business/PRODUCT_AND_ENGINEERING_OPERATIONS.md

Product and engineering operations

Semester Company OS · 2026-10-05 · Source: uploads/what else on the bussnius and opprations site also.pdf.

> Semester must operate with the same coherence, evidence, reliability and trust it promises to institutions.

Field	Value
Mission	Turn evidence into shipped, reliable, accessible product.
Owner	Founder → founding engineer
Inputs	Customer signals, research, catalog gaps, incidents
System of record	GitHub (code, PRs), /ops product operations (backlog, flags, releases)
KPIs	Release cadence; Change failure rate; Error budget remaining; Accessibility issues open
Reports	Release notes; monthly product review
Meeting cadence	Weekly planning; release per engineering playbook
Risk controls	CI gates, RLS proof, two-person review for auth/RLS/AI/billing
Approval thresholds	No release in pilot launch week, finals or registration opening except S1/S2 fixes
Required documents	Engineering playbook, QA test plan, Accessibility test plan, ADR log, PRD template
Required integrations	GitHub, Vercel, Supabase (deploy status in /ops)
Current status	docs/build/, handoff/execute/, docs/master/
Gaps	PRD template, ADR log, experiment registry
Priority	P1
Next action	Run execute stream 00 and approve the design-system baseline

Workflows

- Signal → research → problem → opportunity → strategy fit → priority score → PRD → design → threat/privacy/accessibility review → technical design → build → test → controlled release → measure → iterate or retire

SEMESTER

QA test plan by role

Owner: engineering. Status: draft, October 2026. Covers the pilot scope first; every row maps to a workflow in `docs/master/SEMESTER_WORKFLOW_CATALOG.md`.

1. Test levels

Level	Tool	Runs	Covers
Unit	vitest	Every PR	semester-core logic, validators, formatters
Database / RLS	SQL test harness	Every PR	Policies per table and role
Component	vitest + Testing Library	Every PR	States, accessible names, keyboard
End to end	Playwright	Every PR (smoke), nightly (full)	Role journeys below
Accessibility	axe + manual	Every PR (axe), each release (manual)	See accessibility test plan
Exploratory	Human	Before each pilot launch	Pilot paths with real institution config in staging

2. Test accounts

One synthetic account per role in staging, mapped through the test IdP: student, applicant, guardian, faculty, TA, advisor, registrar, finance, student affairs, IT/security, institution admin, Semester support operator, Semester executive. No real student data in any non-production environment.

3. Role journeys (end to end)

Role	Journey	Must pass
Student	Sign in with SSO → onboarding → Today → registration readiness → resolve hold hand-off → enroll request	Each item shows its source; hold hand-off goes to registrar; nothing writes to SIS
Student	Share with advisor → revoke	Advisor sees only shared items; access ends immediately on revoke
Student	Ask Semester with a question containing a student ID	Blocked; event logged; friendly message
Student	Export personal data → request deletion	Export file complete; deletion request routed to the institution

Role	Journey	Must pass
Guardian	Accept invitation → view shared items → expiry	Only chosen categories visible; access ends at expiry
Faculty	Publish course AI policy → create assignment → release grade	Policy appears on student AI answers; release is two-step with preview
TA	Grade within assigned sections only	Other sections forbidden
Advisor	Caseload → check-in → referral	Only consented shares visible; referral status updates
Registrar	Grant override (two-person) → reconcile with SIS	Second approver required; audit event before apply
Finance	View balance → payment plan request	No payment handled by Semester; processor hand-off only
IT / security	Configure SSO → role mapping → audit explorer	Changes audited; break-glass needs reason and review
Institution admin	Feature cohort on → kill switch off	Feature disappears for the cohort within a minute
Semester support	Open ticket → view tenant health	No student table readable from /ops
Semester executive	Executive overview	Business data only; no student records

4. Negative and security tests

- Cross-tenant URL tampering for every role → forbidden and logged.
- Expired session and revoked membership → signed out.
- Rate limits on AI and auth endpoints.
- Input validation: oversized fields, script injection in every text input.
- Offline: queue a task offline, reconnect, verify single write (idempotency key).

5. Release entry and exit

Gate	Criterion
Entry	All PRs merged; staging deployed; seed data reset
Exit	Smoke suite green; no open S1/S2; full e2e green within the last 24 h; accessibility checklist signed

6. Defect severity

Severity	Example	Fix by
S1	Data exposure across tenants; AI receives a record	Stop release; fix now
S2	SSO broken; wrong SIS data shown	Before release
S3	One feature broken with workaround	Next release
S4	Cosmetic	Backlog

SEMESTER

Accessibility test plan · VPAT / ACR preparation

Owner: engineering + accessibility tester. Status: draft, October 2026. Target: WCAG 2.2 Level AA. Output: an Accessibility Conformance Report (ACR) using the VPAT® 2.x WCAG edition template.

Status, not claims: until the ACR is complete, say "working toward WCAG 2.2 AA; ACR in preparation." Never say "fully compliant."

1. Scope

Surface	Priority
Pilot student paths: sign-in, onboarding, Today, registration readiness, Ask Semester, sharing, settings	P0
Staff pilot paths: advisor caseload, registrar desk, institution console	P0
Public site	P1
Remaining student and staff screens	P2

2. Automated checks

- axe-core in CI on every route of the preview build; serious and critical issues block merge.
- Lint rules for accessible names on icon-only controls and labels on inputs.
- Color contrast checked from tokens: body text 4.5:1, large text and UI parts 3:1.

3. Manual checks (each release on P0 paths)

Check	How	WCAG
Keyboard only	Complete every P0 journey without a mouse; visible focus throughout; no traps	2.1.1, 2.1.2, 2.4.7, 2.4.11
Screen readers	NVDA + Firefox, JAWS + Chrome, VoiceOver + Safari (macOS and iOS)	1.3.1, 4.1.2, 4.1.3
Zoom and reflow	200% zoom and 320 px width without loss of content	1.4.4, 1.4.10
Text spacing	Apply the text-spacing bookmarklet	1.4.12
Target size	Interactive targets at least 24×24 px (product standard 44 px)	2.5.8
Motion	Reduced motion respected; nothing flashes	2.3.1, 2.3.3

Check	How	WCAG
Forms	Labels, instructions, error identification and suggestions, focus moves to first error	3.3.1–3.3.3, 3.3.7
Time limits	Session timeout warns and allows extending	2.2.1
Authentication	No cognitive test to sign in (SSO)	3.3.8

4. Assistive-technology matrix

AT	Browser	Platform
NVDA	Firefox, Chrome	Windows
JAWS	Chrome	Windows
VoiceOver	Safari	macOS, iOS
TalkBack	Chrome	Android
Dragon / voice control	Chrome	Windows

5. Recording results

Each finding records: screen, criterion, severity, steps, AT and browser, screenshot, fix PR. Findings roll into the ACR as Supports / Partially supports / Does not support / Not applicable, with remarks.

6. ACR production

1. Complete manual checks on P0 scope.
2. Fix all "does not support" items on P0 paths, or document a workaround and date.
3. Fill the VPAT template per criterion with remarks.
4. Founder reviews; publish to institutions on request, with the test date and scope.
5. Re-test on every major release; update the ACR at least yearly.

7. Barrier reports

Reports go to access@[domain] and the in-app "Report a barrier" link. Reply within two business days; S2 accessibility barriers on pilot paths get fixed before the next release.

BUILD HANDOFF

handoff/README.md

Semester — Claude Code handoff package

> **Start at `BUILD.md`.** It is the single entry point: preflight → install → streams in order → verify → human-only steps → go.

Everything a developer needs to turn the design system into the product. Open Claude Code at the repo root (harrisonjrubin7-cmyk/semester) and work through these in order.

#	Folder	What it is	Start with
1	`claude-code-toolkit/`	Skills (build-semester-ui, audit-semester-design-sync, create-semester-component), .mcp.json for Figma, design-system audit + report scripts, tests, CI steps, baseline ratchet	`EXECUTE.md`
2	`semester-core/`	Workflow algorithms as pure TypeScript with tests: provenance, next action, access, freshness, consequence, AI gateway, approvals, scoring, pilot readiness	`README.md`
3	`workflow-router-app/`	Next.js + Supabase AI Workflow Router & Benchmark Lab (already landed as workflow-lab/ upstream)	`README.md`

Code files carry a `.txt` suffix so the design project doesn't bundle them: `find . -name "*.txt" -exec sh -c 'mv "$1" "${1%.txt}"' _ {} \;`

| 0 | `execute/` | **Start here.** 12 stream prompts (setup to launch to grow) plus a master index that runs them in order on branches semester/ | `execute/00-INDEX.md` |

Reference (stay in the design system)

- `readme.md`, `SKILL.md` — design rules, tokens, components, patterns
- `ui_kits/master-catalog/` — 51 roles · 673 screens · 319 workflow steps (17 workflows) · 88 systems · 82 docs, with status and a Claude Code prompt that writes `docs/master/`
- `ui_kits/architecture/` — full stack & data: 20 tables, request flows, operations
- `ui_kits/role-lens/`, `ui_kits/atlas/`, `ui_kits/wireframes/` — every role × screen
- `ui_kits/business/Operating Manual.html` — how to run the company

Order of work (PO first)

1. Install the toolkit and pass `npm run design-system:check`.
2. Drop `semester-core` into `app/src/lib/core/`; reconcile names with `lib/source.ts`, `lib/status.ts`; run its tests.
3. Ship the pilot blockers: SAML SSO, AI-gateway test (no student records), DPA mapping, audit-before-action.
4. Run the master-catalog prompt to produce `docs/master/` from real evidence.

Never commit, push or deploy from these prompts without review.

---

handoff/BUILD.md

BUILD.md — the single entry point for Claude Code or Codex

Read this whole file before doing anything. It is written so an agent can build Semester without guessing. When anything here conflicts with another file, **this file wins**, then `execute/00-INDEX.md`, then stream files, then design docs.

---

0. Ground rules (non-negotiable)

1. Work in `harrisonjrubin7-cmyk/semester`. One branch per stream: `semester/<stream>`. One PR per stream. Never push to `main`, deploy, change production settings or secrets.
2. **Repo beats design.** If the repo already has something, extend it. Record exists / extended / new for every item in the PR.
3. Never mark anything done without the verify output pasted into `docs/execute/<nn>-<slug>-report.md`.
4. Stop and report (do not improvise) when: a gate fails twice; a migration would edit an applied migration; a test needs weakening to pass; anything touches real student data; a human-only step is reached (§7).
5. Product rules enforced in code: RLS + tenant_id everywhere; browser calls server/RPC only; company roles never read student tables; AI never receives student records (scope gate + content scan); every consequential action = preview → audit → apply → undo/request; official systems stay authoritative.

---

1. Preflight

```
 git checkout main && git pull
mkdir -p design && cp -R <design-project-export>/* design/      # handoff/, templates/, docs/, ui_kits/,
guidelines/, readme.md, SKILL.md
bash design/handoff/preflight.sh                                 # must print PREFLIGHT OK
```

---

2. App target

- The API routes, middleware and tests in `semester-platform` are **Next.js App Router + Supabase**. If `app/` is already Next.js, install there (`APP=app`).
- If `app/` is Vite + React: keep it for the student UI and create `web/` with `npx create-next-app@latest web --ts --app --eslint --no-tailwind --src-dir`, then `APP=web`. The Vite app calls `web`'s API. Do not rewrite the Vite app in this stream.

---

3. Install

```
 APP=<app|web> bash design/handoff/install.sh                    # never overwrites; FORCE=1 only with a
reason in the PR
```

```
(cd $APP && npm i @supabase/supabase-js @supabase/ssr zod && npm i -D vitest dotenv dotenv-cli)
cp $APP/.env.example $APP/.env.local                                  # fill from: npx supabase status
```

4. Order of work

Stream	File	Gate to pass before the next
00	execute/00-setup.md	CI green; REPO_AUDIT.md; design-system baseline approved by founder
01	execute/01-platform-core.md	`verify.sh` steps 1–5 green; every policy red-then-green
02	execute/02-identity-sso.md	SSO test login per pilot role (staging IdP)
03	execute/03-student-os.md	Student workflow e2e; 5 states per screen
04	execute/04-staff-academic.md	Two-person approvals tested
05	execute/05-institution-integrations.md	Reconciliation 0 unexplained mismatches
06	execute/06-ai-gateway.md	ai-redteam suite green; no prompt content in logs
07	execute/07-community-career-family.md	Feature switches; revoke works immediately
08	execute/08-ops-command-center.md	/ops cannot read student tables (test)
09	execute/09-trust-security.md	Restore drill timed; axe clean on pilot routes
10	execute/10-launch.md	Every go/no-go gate has evidence
11	execute/11-company-site-grow.md	Claims sourced; a11y ≥ 95
12	execute/12-one-place.md	Every one-place row has an in-app path
13	execute/13-platform-maturity.md	Scorecard re-scored from repo evidence

Pilot-critical path: **00 → 01 → 02 → 03 → 06 → 05 → 09 → 10**. Streams 04, 07, 08, 11–13 may follow after the pilot launches.

5. Migrations (exact order)

010 core → 020 community → 030 marketplace → 040 ops → **050 security hardening** → **060 command functions** → **070 AI gateway** → **080 sources, storage, delegations**. Renumber after the repo's latest migration if needed, keeping this order. 050 must be applied before any data.

6. Verify (run after every stream)

```
APP=<app|web> bash design/handoff/verify.sh                           # must end with ALL GATES PASSED
```

Then add the stream's own exit gates. Paste the full output into the report.

7. Human-only steps (the agent stops and asks)

- Bind cyber + E&O insurance · sign the DPA/FERPA addendum · submit HECVAT Full · attorney review of docs/legal/*

- Provide the Vanderbilt staging IdP metadata and test accounts · name IT, registrar and sponsor contacts
- Approve the design-system baseline · approve every PR · approve production cutover (docs/launch/Cutover Plan.html)
- Choose the AI provider and sign its data terms (no training, no retention)

---

8. Definition of go

All of: streams on the pilot-critical path merged; `verify.sh` green on main; launch runbook go/no-go gates each with evidence; §7 complete. Then follow docs/launch/Cutover Plan.html C1–C11.

---

10. Coverage map — every part of the design project and the stream that builds it

Design project area	What it is	Built by stream	Reference
Tokens, type, colour, spacing, motion, patterns	Design foundations	00 (toolkit sync), then every UI stream	tokens/, guidelines/, readme.md
Components (core, forms, data, feedback, surfaces, overlays, navigation, trust, governance)	64 components	00 maps to app; 03–08 use them	components/, docs/system/design/SEMESTER_COMPONENT_LIBRARY.md
Student app, mobile, privacy center, platform shell	Student OS	03	ui_kits/student, ui_kits/mobile, templates/privacy-center, templates/platform-shell
Role onboarding (13 roles)	First-run per role	02	templates/role-onboarding
Staff workspaces, Course Studio	Faculty, TA, advisor, registrar, finance, student affairs	04	ui_kits/staff
Institution console, Institution OS	Control plane, integrations, migration	05	ui_kits/console, templates/institution-os, templates/console-screen
AI gateway, Ask Semester	Governed AI	06	semester-platform/app/api/ai, lib/ai, 070
Community & marketplace, external audiences, career, family, developer platform	Social, market, career, guardian, partners	07	ui_kits/community, ui_kits/external
Operations consoles, Command center, One place	Company runs inside Semester	08, 12	ui_kits/ops, templates/command-center, ui_kits/one-place
Security, privacy, accessibility, compliance	Controls, evidence, ACR	09	docs/secure, docs/build/Accessibility Test Plan.html, ui_kits/compliance
Launch runbook, cutover, support, launch kit	Go-live	10	docs/launch, ui_kits/launch-kit
Company site, GTM, legal pages, brief	Public site and sales	11	ui_kits/company-site, docs/grow, docs/legal, docs/brief
Platform maturity, strategy, business OS, system docs	Programs and standards	13 (plus monthly review)	docs/platform, docs/strategy, docs/business, docs/system

Design project area	What it is	Built by stream	Reference
Master catalog, every screen, gap screens, role pages, workflow runner, atlas, role lens, wireframes	The full inventory (673 screens, 17 workflows, 319 steps, 49 roles)	Acceptance list for 03–08: each catalogued screen and workflow step must exist in the app or be listed as deferred in the PR	ui_kits/master-catalog, every-screen, gap-screens, role-pages, workflow-runner, docs/master
Architecture, read models, environments	System design	01	ui_kits/architecture, guidelines/arch-*.html
Decks, reports, teaser, poster, research brief, vision deck	Sales and launch materials	Not code: export from the design project (PDF/PPTX)	templates/*
Business workspace, operating manual, finance model, hiring	Running the company	Not code: used by the founder; /ops modules in 08	ui_kits/business, docs/run

Completeness rule: at the end of streams 03–08 the agent runs a coverage check: every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md is either implemented (route + test) or listed as deferred with a reason and priority. Nothing is silently skipped.

---

9. Where things are

`execute/` stream prompts · `semester-platform/` DB, API, tests · `semester-core/` decision logic · `claude-code-toolkit/` design-system tooling · `manifest.json` every file and its destination · design references in `../ui_kits`, `../templates`, `../docs`.

Execution plan, step by step

handoff/execute/00-INDEX.md

Semester — execute everything

> Read `handoff/BUILD.md` first. Run `preflight.sh`, `install.sh`, then these streams; run `verify.sh` after each.

One prompt per stream, run in order. Each stream works on its own branch and ends with a PR and a report. Do not start a stream until the previous one's exit gates are ticked.

Master prompt (paste into Claude Code or Codex at the repo root)

```
 You are building Semester in harrisonjrubin7-cmyk/semester.
Read handoff/BUILD.md and follow it exactly. Run handoff/preflight.sh and handoff/install.sh. Then run
each stream file in order:
handoff/execute/00-setup.md
handoff/execute/01-platform-core.md
handoff/execute/02-identity-sso.md
handoff/execute/03-student-os.md
handoff/execute/04-staff-academic.md
handoff/execute/05-institution-integrations.md
handoff/execute/06-ai-gateway.md
handoff/execute/07-community-career-family.md
handoff/execute/08-ops-command-center.md
handoff/execute/09-trust-security.md
handoff/execute/10-launch.md
handoff/execute/11-company-site-grow.md
handoff/execute/12-one-place.md
handoff/execute/13-platform-maturity.md
For each stream: create branch semester/<slug> from origin/main, do the tasks, run every check, write the
report to docs/execute/<nn>-<slug>-report.md, open a PR, and STOP for human review before the next
stream.
If an input file is missing from the repo, copy it from the Semester design project export under design/.
Never deploy, change secrets, or mark anything complete without evidence.
```

Streams

#	Stream	Branch	Depends on
00	Repo audit and design-system sync	semester/setup	—
01	Database, tenancy, RLS and core logic	semester/platform-core	00
02	Identity, SAML SSO, onboarding	semester/identity-sso	01
03	Student OS	semester/student-os	01, 02

#	Stream	Branch	Depends on
04	Faculty, TA, advisor, registrar, finance, student affairs	semester/staff-academic	01, 02
05	Institution console, integrations, migration	semester/institution-integrations	01, 02
06	Governed AI gateway	semester/ai-gateway	01, 02
07	Community, career, family, developer platform	semester/community-career-family	01, 02
08	Operations command center and site operations	semester/ops-command-center	01, 02
09	Security, privacy, accessibility, compliance	semester/trust-security	01, 02
10	Pilot launch	semester/launch	01, 02
11	Public site, GTM, run and grow	semester/company-site-grow	01, 02
12	One place: nobody leaves Semester	semester/one-place	00–08
13	Platform maturity	semester/platform	00–10

Getting the design files into the repo

Download this design project, then copy into the repo under `design/`: `handoff/`, `templates/`, `docs/`, `ui_kits/`, `guidelines/`, `readme.md`, `SKILL.md`. Code files in handoff/ end in .txt; strip the suffix when installing.

Founder-only steps (no agent can do these)

- Sign the DPA/FERPA addendum; bind cyber and E&O insurance; submit HECVAT Full.
- Attorney review of docs/legal/*.
- Approve the design-system baseline (stream 00) and every PR.
- Name Vanderbilt contacts for IT, registrar and sponsor.

`handoff/execute/00-setup.md`

Stream 00 · Repo audit and design-system sync

Make the repo ready for every later stream.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `handoff/claude-code-toolkit/`
- `docs/master/*.md`
- `github.md` (screen map)

Tasks

1. Run handoff/claude-code-toolkit/EXECUTE.md steps 1, 2, 4 and 5 (skip Figma unless connected).
2. Write docs/master/REPO_AUDIT.md: every route, screen, component, migration, API route, test and doc in the repo, mapped to the rows of docs/master/SEMESTER_SCREEN_CATALOG.md and SEMESTER_WORKFLOW_CATALOG.md. Mark each row exists / partial / missing.
3. Update docs/master/SEMESTER_GAP_REGISTER.md from real evidence (repo beats design).
4. Add CI jobs if missing: typecheck, lint, unit tests, design-system check, RLS test job (can be empty until stream 01).

Exit gates

- [ ] CI green
- [ ] REPO_AUDIT.md covers every catalog row
- [ ] Baseline JSON committed and approved

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/setup` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

handoff/execute/01-platform-core.md

Stream 01 · Database, tenancy, RLS and core logic

Land the data foundation every screen reads from.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `handoff/semester-platform/` (migrations 010-040, lib, app routes, tests)
- `handoff/semester-core/`
- `ui_kits/architecture/` (20-table design schema, flows)

Tasks

1. Reconcile migrations 010-040 with existing supabase/migrations; renumber after the latest existing one. Never edit an applied migration; add new ones.
2. Add memberships, roles, capabilities, grants (scoped, expiring), sessions, delegations, audit.events (append-only), outbox and read_model schemas from the architecture kit.
3. Install semester-core into app/src/lib/core/; reconcile names with lib/source.ts, lib/status.ts, lib/navareas.ts; run its tests.
4. Write RLS tests per table: own-tenant read/write, cross-tenant denied, company roles denied on student tables, expired grant denied, audit insert-only.
5. Apply 050_security_hardening.sql and prove tests/security.test.ts red-then-green (docs/secure/Platform Security Review.html), then 060_command_functions.sql (tests/commands.test.ts), 070_ai_gateway.sql and 080_sources_storage_delegations.sql; add middleware.security.ts and the CI security job.
6. Outbox worker and projections with version and freshness; freshness.ts drives UI policy.

Exit gates

- [ ] npx supabase db reset runs clean
- [ ] RLS suite passes with a red-then-green proof per policy
- [ ] semester-core tests pass

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/platform-core` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.

- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/02-identity-sso.md`

Stream 02 · Identity, SAML SSO, onboarding

Sign-in for the Vanderbilt pilot and first-run onboarding for every role.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `templates/role-onboarding/` (13 role onboardings + RoleOnboarding)
- `docs/launch/Cutover Plan.html` (steps C2-C3)
- `ui_kits/student` (Connect, Settings, Privacy)

Tasks

1. SAML SSO via Supabase Auth (or the current auth in the repo) with role mapping from IdP attributes; staging test IdP first.
2. Account switcher, sessions list with remote sign-out, delegated access with expiry.
3. Build onboarding for each of the 13 roles from templates/role-onboarding/: sign in, connect, privacy choice, AI choice, first home.

4. Capabilities and Permissions screen: camera, microphone, location, notifications. Each asks, has a fallback and a revoke path.

Exit gates

- [ ] Test login per pilot role in staging
- [ ] Role mapping tests
- [ ] Onboarding a11y check per role

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/identity-sso` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/03-student-os.md`

Stream 03 · Student OS

Every student screen in catalog group B and the student side of F and G.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/student/` (64 screens)
- `ui_kits/mobile/`
- `templates/platform-shell/`
- `templates/student-screen/`
- `ui_kits/workflow-runner/` (Student, Student finance, Campus services)

Tasks

1. For each screen in docs/master/SEMESTER_SCREEN_CATALOG.md group B: route, data from read models, source labels, all states.
2. Today, Action Center, Path, Plan, Registration readiness, Courses, Notes, Files, Search, Ask Semester, Support, Campus, Career, Account, Privacy and data, Data export/delete.
3. Run every Student workflow step from the runner end to end against staging data.
4. Mobile: five-destination tab bar, offline queue, PWA install.

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Each Student workflow step passes an e2e test
- [ ] Every screen has all 5 states
- [ ] Lighthouse a11y 95+ on the top 10 screens

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/student-os` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.

- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/04-staff-academic.md`

Stream 04 · Faculty, TA, advisor, registrar, finance, student affairs

Staff workspaces (catalog groups C, D, E and the staff side of F).

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/staff/`
- `ui_kits/workflow-runner/` (Faculty, Advisor, Registrar, Student finance)
- `templates/console-screen/`

Tasks

1. Course Studio (faculty authoring): syllabus, objectives, assignments, rubrics, course AI rules, gradebook, grade release, regrade queue.
2. Advisor caseload, check-ins, referrals, consented shares.
3. Registrar desk: terms, sections, holds sync, overrides (two-person), add/drop, reconciliation with SIS.
4. Student accounts: balance, payment plans, hold release, reconciliation. Payments go through a processor only.

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Two-person approvals tested
- [ ] Official-record changes go to the SIS, never edited in Semester
- [ ] Workflow steps pass e2e

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/staff-academic` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/05-institution-integrations.md`

Stream 05 · Institution console, integrations, migration

Control plane (K) and integration fabric (L).

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/console/`
- `templates/institution-os/`
- `ui_kits/workflow-runner/` (Institutional implementation, Controlled integration, Domain migration)

Tasks

1. Tenant overview, roles, role grants, feature cohorts, kill switches, policies, audit explorer.
2. SIS read sync and LMS (LTI 1.3 / OneRoster) connectors, read-only first; mapping studio with versions; reconciliation dashboard; dead-letter and replay.
3. Migration center following the 17-step domain migration workflow.

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Reconciliation shows 0 unexplained mismatches on staging data
- [ ] Kill switch disables a feature in under 1 minute
- [ ] Connector credentials server-side only

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/institution-integrations` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.

- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/06-ai-gateway.md`

Stream 06 · Governed AI gateway

One gateway for every AI request.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `handoff/semester-core/aigateway.ts.txt`
- `ui_kits/architecture/` (AI gateway tab)
- `guidelines/02-ai-governance.md`
- `components/trust/AIResponse.jsx`

Starting code

- `handoff/semester-platform/app/api/ai/route.ts.txt`, `lib/ai/redact.ts.txt`, `supabase/migrations/070_ai_gateway.sql.txt`, `tests/ai-redteam.test.ts.txt`

Tasks

1. Implement the 17-step AI workflow: scope, classification, consent, policy, provider routing, approved sources, tool permissions, response with sources, suggested action, human escalation, spend and audit log.
2. Red-team suite: prompts containing student IDs, grades or records must be blocked, and every block logged.
3. Course AI rules editor for faculty; per-tenant switches.

Exit gates

- [ ] Red-team suite passes (launch gate)
- [ ] Every AI answer is labelled with sources
- [ ] Spend logged per tenant

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/ai-gateway` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/07-community-career-family.md`

Stream 07 · Community, career, family, developer platform

Groups H, I, J and the developer platform.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/community/`
- `ui_kits/external/`
- `handoff/semester-platform/` (migrations 020, 030)
- `ui_kits/workflow-runner/` (Community, Career, Family consent, Developer platform)

Tasks

1. Community: verified profiles with per-field visibility, groups, Q&A with endorsed answers, reporting, moderation queue, appeals. AI pre-screen on public posts only.
2. Marketplace: listings with the academic-integrity filter, offers in chat, safe meet-up spots; payments via a processor.
3. Family: consent invitation, scoped categories, expiry, access history, revoke.
4. Career: skills, evidence portfolio, verified skills, opportunities; employers see approved evidence only.
5. Developer platform: OAuth clients, scopes, webhooks, sandbox tenants (can ship after the pilot).

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Feature switch per institution
- [ ] Revocation removes access immediately (tested)
- [ ] Moderation audit trail

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/community-career-family` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/08-ops-command-center.md`

Stream 08 · Operations command center and site operations

/ops for Semester operators (group N).

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/ops/` (12 consoles)
- `templates/command-center/`
- `docs/launch/Support Plan.html`

Tasks

1. Executive overview, ops inbox, approvals, tenants, pilots, support desk, incidents, SLOs and error budgets, feature flags, releases, audit explorer, finance dashboard.
2. Business data only: a test proves every /ops query fails against student tables.
3. Monitoring: error tracking, uptime, SSO failures, SIS lag, gateway blocks, with alert routes from the support plan.

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Company-role isolation test passes
- [ ] Alerts fire in staging
- [ ] On-call runbook linked from each alert

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/ops-command-center` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

---

handoff/execute/09-trust-security.md

Stream 09 · Security, privacy, accessibility, compliance

Evidence for HECVAT, SOC 2 readiness and VPAT/ACR.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `docs/secure/Security Policies.html`
- `docs/secure/Security Readiness.html`
- `ui_kits/compliance/`

Tasks

1. Map each policy control to code or a procedure; store evidence links in the evidence register.
2. Backups and a timed restore drill; break-glass with review; access reviews; secret inventory.
3. Accessibility: axe in CI on every route, manual screen-reader pass on pilot paths, and the ACR test log.
4. Data inventory, retention jobs, privacy request flows (export, correction, deletion).

Exit gates

- [ ] Restore drill timed and documented
- [ ] axe clean on pilot routes
- [ ] Evidence register complete for HECVAT answers

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/trust-security` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.

- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/10-launch.md`

Stream 10 · Pilot launch

Take the Vanderbilt pilot live.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `docs/launch/` (runbook, cutover plan, support plan)

Tasks

1. Implement the cohort flag, holding message and status page.
2. Script cutover steps C1-C11 where possible; produce the cutover log template.
3. Support desk macros from the support plan; known-issues page.

Exit gates

- [ ] Every go/no-go gate has evidence attached
- [ ] Rollback rehearsed in staging

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/launch` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/11-company-site-grow.md`

Stream 11 · Public site, GTM, run and grow

The public site and company operating documents.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/company-site/`
- `docs/brief/`
- `docs/legal/` (drafts)
- `ui_kits/business/`

Tasks

1. Full public site: home, product, institutions, students, security/trust, pricing, about, contact, legal. Claims come only from the public claims register: status, not certifications.
2. Contact and pilot-request forms with spam protection; no student data collected.
3. Publish legal drafts as pages marked "draft pending attorney review" until reviewed.

Exit gates

- [ ] Every claim cites its source
- [ ] a11y 95+
- [ ] Forms tested

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/company-site-grow` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/12-one-place.md`

Stream 12 · One place: nobody leaves Semester

Make every job in `ui_kits/one-place/one-place-data.js` doable inside Semester. Paste into Claude Code (or Codex) at the repo root after streams 00–08.

Inputs from the design project

- `ui_kits/one-place/` (job map: designed / to build / embedded)
- `ui_kits/ops/` (company consoles)
- `ui_kits/student/` (Write, Sheets, Decks, Mail, Calendar)

Tasks

1. For each "to build" row, build the module (Meet, team spaces, contracts with in-app signing, career portfolio) using existing shells and components.
2. For each "embedded" row, build the in-app hand-off: provider screen inside Semester (processor checkout, payroll status, e-sign, IdP sign-in), status written back to Semester, audit event on each step. No regulated step is re-implemented.
3. Run the company inside /ops: CRM, support, incidents, social, finance, people, compliance, product ops, board reports. Semester staff use Semester for docs, sheets, decks, mail and calendar.
4. Add an "open in Semester" check to the release checklist: any new feature that sends users to another app needs an embedded path or a written exception.

Exit gates

- [ ] Every row in one-place-data.js is designed, built, or embedded with a working in-app path
- [ ] Company operating cadence (weekly metrics, monthly close, QBRs) runs from /ops
- [ ] No student data in company modules (isolation test passes)

Follow the shared rules in any other stream file (branch `semester/one-place`, PR + report, stop for review).

`handoff/execute/13-platform-maturity.md`

Stream 13 · Platform maturity

Turn Semester from a broad product architecture into a compounding education platform. Run after the pilot is live (streams 00–10).

Inputs from the design project

- `docs/platform/*.md` (12 pillar standards, scorecard, risk register, 36-month roadmap, next 50 actions)

- `ui_kits/platform-maturity/`
- `docs/strategy/` (competitive audit, coherence test, implementation factory, AI differentiation, trust advantage)
- `docs/system/` (brand, UI grammar, component gaps, algorithm governance, workflow type, data classification, provenance)
- `docs/business/` (company OS, five business systems, systems of record)

Tasks

1. Audit the repo against each pillar's capability record; write current evidence into docs/platform/*.md (repo beats design).
2. Re-score PLATFORM_READINESS_SCORECARD.md with evidence links; never raise a score without evidence.
3. Work the next 50 actions in order; each completed action links its PR or document.
4. Build the platform pieces in pillar order: daily experience standard checks in CI, graph canonical IDs + source authority, AI evaluation harness, connector certification suite, status page, public API beta.
5. Add the coherence test to the PR template and release checklist.
6. Implement the SemesterWorkflow type, universal state machine, data classification tags and provenance columns from docs/system/.
7. Build the missing components: PolicyBadge, ConsentBadge, HealthBadge, SupportHandoff, ApprovalBanner, ErrorSummary, PageHeader.

Exit gates

- [ ] Scorecard re-scored from repo evidence
- [ ] Every pillar has a named owner and next action with a date
- [ ] Monthly scorecard review in the operating cadence

Follow the shared rules in any other stream file (branch `semester/platform`, PR + report, stop for review).

Code packages

handoff/semester-core/README.md

semester-core — the workflows and algorithms, as code

Pure TypeScript, no dependencies, no I/O. Each module is the decision logic behind a Semester workflow the design system renders; the app supplies data and persistence. Every module has a vitest file proving its rules.

Module	Workflow	Used by
`provenance.ts`	Rank sources; resolve conflicting facts; freshness label	SourceBadge, ProvenanceChips, Today
`nextaction.ts`	Score and order a student's next actions	Today, NextSteps, Runway
`access.ts`	Role × screen access, share expiry, break-glass	Role lens, every route guard
`freshness.ts`	Projection lag → UI policy (display / safe / destructive)	Read models, Lag simulator
`consequence.ts`	Classify an action → undo / preview / two-person	Dialog, ActionPreview, Approvals
`aigateway.ts`	Scope + policy check before any model call	Ask Semester, AIResponse
`approvals.ts`	Two-person duty state machine	Console approvals, Finance holds
`scoring.ts`	Weighted 0–5 grading with overrides	Workflow lab benchmark
`pilot.ts`	Launch readiness from the tracker	Business gap analysis

Files carry a `.txt` suffix so this design project does not bundle them; drop it when copying (`for f in *.txt; do mv "$f" "${f%.txt}"; done`). Install into `app/src/lib/core/` (or a workspace package). Run `npx vitest run src/lib/core`. These are reference implementations reconciled with the design system — check names against `lib/source.ts`, `lib/status.ts` and `lib/navareas.ts` before replacing anything that already exists.

handoff/semester-platform/README.md

semester-platform

Database and API code for community, marketplace, ops and audit, built on the core Semester app (Next.js + Supabase). Files end in .txt so they don't run in the design project. Strip the suffix when you copy them into the repo.

Install: cp supabase/migrations/*.sql.txt /supabase/migrations/ (rename to .sql) cp -r app lib tests / (rename .ts.txt to .ts) npx supabase db reset && npm test

Status: written to the architecture guide. Not yet run, typechecked or tested; do that before you rely on it.

Security hardening (Oct 2026)

migrations/050_security_hardening.sql.txt closes 11 findings from the review of 010–040 (2 critical/high: ops.staff self-insert as 'ceo', and moving a profile to another institution). tests/security.test.ts.txt proves each one. Apply 050 before any real data. Review: docs/secure/Platform Security Review.html in the design project.

migrations/060_command_functions.sql.txt: all community/market writes go through audited api.* functions (validation, rate limits, integrity filter, campus checks); storage policies for listing photos. Proven by tests/commands.test.ts.txt.

AI gateway: app/api/ai/route.ts.txt (auth → policy → gate → content scan → provider → audit), lib/ai/redact.ts.txt (blocks IDs, SSN, GPA, grades, transcripts, DOB, cards typed into prompts), migrations/070_ai_gateway.sql.txt (tenant policy, course rules, usage log with no prompt content), tests/ai-redteam.test.ts.txt (launch gate).

080_sources_storage_delegations: approved-source AI context, private buckets with path rules, audited delegations. middleware.security.ts.txt: CSP/HSTS/headers. .github/ci-security-job.yml.txt: secret scan, audit, guards, security tests.

---

handoff/workflow-router-app/README.md

Semester AI Workflow Router & Benchmark Lab

Next.js App Router + Supabase app that routes a Semester workflow to **Claude Artifacts**, **ChatGPT Canvas** or **v0**, compares them in sortable matrices, and records a graded 12-workflow benchmark with evidence.

> Built in Claude Design as a source hand-off. Every code file carries a `.txt` suffix (and `[suiteId]` is saved as `-suiteId-`) so the design-system bundler ignores them. Restore with `sh install.sh`. **Typecheck, lint, tests and build have not been run here** — run them first (see below) and fix anything they report.

Install

```
cp -R workflow-router-app ~/code/semester-router && cd ~/code/semester-router
sh install.sh            # strips .txt, restores app/benchmark/[suiteId]
npm install
cp .env.example .env.local   # fill NEXT_PUBLIC_SUPABASE_URL + NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
```

Supabase

```
npx supabase init && npx supabase start          # local
npx supabase db push                             # or: link a project, then push
# sign up once in the app, then:
psql "$DB_URL" -f supabase/seed.sql              # seeds "Semester canonical 12"
npm run db:types                                 # regenerate types/database.ts
```

Auth → URL configuration: Site URL = your origin; add `/auth/confirm` to redirect URLs. Email template link: `{{ .SiteURL }}/auth/confirm?token_hash={{ .TokenHash }}&type=email&next=/workflow-router`.

Run and verify

```
npm run dev
npm run typecheck && npm run lint && npm test && npm run build
```

Quality gate	Where it is held
Router works on local data	`lib/workflow-router/engine.ts` (no network) · `tests/routing.test.ts`
v0 forced for DB/auth/multi-tenant/Next.js/deploy	`v0Triggers` · 13 forced cases in `tests/routing.test.ts`
Markdown + JSON recommendation export	client download + `app/api/exports/recommendation`
Authenticated save	`app/workflow-router/actions.ts` (server recomputes; `getUser()`)
12 workflows per suite, run per platform/task	`benchmark_tasks unique(suite_id, number)` · `benchmark_runs`
Grades with evidence	`benchmark_grades unique(run_id, grader_id)` · `GradingForm` requires evidence
Weighted scores + leaderboards	`lib/benchmark/scoring.ts` · `tests/scoring.test.ts`
Persistence explicitly tested	DB check constraint + `RunSchema` + `persistenceCeiling` (local state ≤ 2)
RLS isolation	migrations 001/002 · `tests/RLS-TEST-PLAN.md`
No secret in the browser	only `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`; `lib/supabase/server.ts` imports `server-only`

Benchmark protocol (all platforms are manual — no automation is assumed)

1. Open a suite → task → pick a platform. The capability guard marks persistence tasks as **unsupported** on Claude Artifacts and ChatGPT Canvas with the reason; record the run anyway to document the gap.
2. **Claude Artifacts:** paste the exact prompt into claude.ai, publish the artifact, record the URL, the source as the manifest, console notes and duration.
3. **ChatGPT Canvas:** run the prompt in Canvas, export (Markdown/DOCX/PDF or code file), record the share link and file manifest.
4. **v0:** run the prompt, push to GitHub, deploy a preview, record repo + deployment URLs, the file manifest and the test log.
5. Persistence tasks (9–12): create a record, reload, and record survived/lost, the storage mode and a row id or log. Without evidence the persistence grade is capped at 1; browser-local at 2.
6. Two graders minimum per run; grade 0–5 on each criterion with evidence. Export CSV / Markdown from the suite page.

GitHub and Vercel

Push to GitHub → import in Vercel → add the two `NEXT_PUBLIC_SUPABASE_*` env vars (Production + Preview) → deploy. Protect `main`; require typecheck, lint, test and build on PRs; run `supabase db push` from CI with a deploy token, never from the browser.

Security / RLS

- Every exposed table has RLS on. Private rows: `owner_id = auth.uid()`. Shared rows: `visibility = 'organization'` **and** membership via `is_org_member` / `has_org_role` (SECURITY DEFINER, `search_path = public`) to avoid recursive policies.
- Viewers read; members write their own; owners/admins manage membership; only owners grant owner.
- Runs and grades inherit access from their suite (`can_read_suite` / `can_write_suite`).
- Mutations validate with Zod on the server and re-derive the recommendation; client UI state never authorises anything.

Claude Code toolkit

handoff/claude-code-toolkit/README.md

Semester · Claude Code design-system toolkit

Drop-in files for **harrisonjrubin7-cmyk/semester**, built from the brief in `uploads/provide comand to claud code to excute on this and.pdf` and checked against the repo (`lib/tokenexport.ts`, `docs/DESIGN-TOKENS.md`, `styles/hex.test.ts`, `design-tokens/semester.tokens.json`, `.claude/skills/`).

Copy the tree into the repository root — every path matches the repo layout. The three code files carry a `.txt` suffix so the design-system bundler here ignores them; drop it when copying:

```
for f in $(find . -name "*.txt"); do mv "$f" "${f%.txt}"; done
```

File	Purpose
`.claude/skills/audit-semester-design-sync/SKILL.md`	Read-only, repo-specific design-system audit (tokens, components, responsive, a11y, Figma parity)
`.claude/skills/build-semester-ui/SKILL.md`	Building UI with unity primitives, trust vocabulary, token rules and the finish checks
`.mcp.json`	Remote Figma MCP server (`type: http`), no credentials
`docs/design-system/FIGMA-MAPPING.md`	Template: ownership, rules, proposed Figma names ↔ verified exported paths, components, unresolved
`docs/design-system/SEMESTER-DESIGN-SYSTEM-BASELINE.md`	Phase 1 read-only baseline: architecture, source-of-truth map, strengths, gaps, risks, plan
`docs/design-system/README.md`	How to use the skills, Figma MCP, checks and report
`CLAUDE.md.design-system-section.md`	Section to append to the existing CLAUDE.md
`.claude/skills/create-semester-component/SKILL.md`	Spec-first shared component; rejects one-off wrappers
`.github/ci-design-system-steps.yml`	Three steps for the existing `ci.yml` build job + report artifact
`gitignore.addition`	`app/reports/`
`app/scripts/design-system-audit.mjs`	Zero-dependency audit: raw colours vs the hex ledger, undefined custom properties, tokens.css ↔ export sync, mapping resolution
`app/scripts/design-system-report.mjs`	Markdown report for CI (`--out`), exits with the audit status
`app/src/lib/designsystem.test.ts`	Proves each check fails on a violating fixture and the real tree is clean
`app/package.scripts.json`	`tokens:check`, `design-system:audit`, `:report`, `:check` to merge into `app/package.json`

Nothing here adds a dependency, a token source, Tailwind, Storybook or a package. `semester.tokens.json` stays generated by `npm run tokens:export`.

See `PROMPTS.md` for the exact Claude Code commands.

handoff/claude-code-toolkit/EXECUTE.md

Execute the five follow-ups in Claude Code

Paste into Claude Code at the repository root. It runs everything I could not run from Claude Design.

```
 Read handoff README.md, CLAUDE.md, docs/design-system/SEMESTER-DESIGN-SYSTEM-BASELINE.md. git fetch
origin main && git log --oneline -30 origin/main first.
Do not commit, push, open a PR, change secrets or write to Figma. Stop and report after each step that
fails.

STEP 1 — Install and run
- Copy the toolkit tree into the repo; for f in $(find . -name "*.txt" -path "*toolkit*"); do mv "$f"
"${f%.txt}"; done
- Append CLAUDE.md.design-system-section.md to CLAUDE.md (do not change existing sections). Add
gitignore.addition to .gitignore.
- Merge app/package.scripts.json into app/package.json scripts (keep existing scripts).
- From app/: npm run tokens:check; npm run design-system:audit; npx vitest run
src/lib/designsystem.test.ts; npx tsc -b; npm run lint; npm test.
- Fix only failures caused by the toolkit (e.g. a regex catching a false positive). Revert one check's
fix and watch its test go red, then restore. List pre-existing failures separately.

STEP 2 — Agree the raw-value baseline
- npm run design-system:baseline → app/design-system-baseline.json. Print totals per check (z-index,
duration, easing, shadow, font-size, radius) and the 10 largest files.
- Show me the table and wait for "approve baseline" before continuing.

STEP 3 — Figma (needs me)
- If /mcp shows figma connected and I give a file URL: read its variable collections and components
(read-only). Replace every "(proposed)" name in docs/design-system/FIGMA-MAPPING.md with the fetched
name, set Status per row (Match / Partial / Missing in Figma / Missing in code), move unmatched Figma
variables to Unresolved. Do not invent names. Run design-system:audit; 0 missing mappings.
- If not connected: say so and skip.

STEP 4 — Move inline z-index and durations onto tokens
- For each baseline entry with check z-index or duration in app/src (not look.ts, not LEDGER files):
  z-index 1→var(--layer-raised) 20→var(--layer-sticky) 21→var(--layer-chrome) 80→var(--layer-overlay)
90→var(--layer-menu) 100→var(--layer-skip) 1500→var(--layer-curtain); any other number: leave it and list
it with its reason.
  durations: transitions/animations → var(--motion-save|insert|panel|sheet|progress) by job (130ms
hover/press → var(--fast)); never --duration-* directly (those are not zeroed for reduced motion).
- Keep stacking.test.ts, motion.test.ts, a11y tests green. Re-run npm run design-system:baseline so the
baseline shrinks; the stale check enforces it.

STEP 5 — Make it a CI gate
- Insert .github/ci-design-system-steps.yml into .github/workflows/ci.yml build job after "npm run lint".
The committed baseline now makes any new raw value a failure.
- Run the full gate list from CLAUDE.md. Report: files changed, commands + results, baseline
before/after, remaining entries with reasons.
```

handoff/claude-code-toolkit/PROMPTS.md

Claude Code commands

0 · Set up (once, from the repo root)

```
 cd /path/to/semester
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
# or: claude plugin install figma@claude-plugins-official   (pick one route, not both)
claude
```

In Claude Code run `/mcp`, authorise Figma in the browser, confirm it shows connected.

1 · Audit (no edits)

```
 /audit-semester-design-sync app/src app/design-tokens/semester.tokens.json app/src/styles

You are auditing the existing Semester repository, not designing a generic new system.
First: git fetch origin main && git log --oneline -30 origin/main.
Read: CLAUDE.md, app/README.md, DESIGN-SYSTEM-GUIDE.md, docs/DESIGN-TOKENS.md,
docs/DESIGN-TOKEN-ARCHITECTURE.md, docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md, app/package.json,
app/src/styles/tokens.css, app/src/lib/look.ts, app/src/lib/tokenexport.ts (+ .test.ts),
app/design-tokens/semester.tokens.json, app/src/styles/*.test.ts, .claude/skills/.
Inspect: app/src/components/ (esp. unity/), app/src/screens/, app/src/styles/, App.tsx, main.tsx,
screens.tsx.
Do not modify files. Produce the report format in the skill:
1 architecture + source-of-truth map; 2 token pipeline look.ts/tokens.css → tokenexport.ts →
semester.tokens.json → Figma variables; 3 strengths to preserve; 4 drift risks (raw styles, hex,
duplicated component styles, hard-coded z-index/durations); 5 existing tests that enforce contracts;
6 exact missing files for the Claude Code + Figma MCP workflow only; 7 minimal phased plan;
8 table: priority · exact path · rationale · acceptance test.
Use evidence from actual files. Do not assume pnpm, monorepos, Storybook or Tailwind.
```

2 · Implement the toolkit (after you approve the audit)

```
 Implement the approved Semester Claude Code design-system toolkit.

Constraints:
- Preserve the Vite + React app under app/, tokens.css as the semantic authority, look.ts (tokensFor)
  as the value authority, tokenexport.ts as the export authority, semester.tokens.json as generated
output.
- Extend the existing tokenexport, style, accessibility, responsive and contract tests; don't replace
them.
- No Tailwind, Storybook, monorepo, new component package, or new dependency.
- Don't move components or rewrite App.tsx. Don't commit, push, open a PR or touch deployment config.

Create only (the drafts are in this hand-off folder — reconcile them with the audit, don't paste
blindly):
1 .claude/skills/audit-semester-design-sync/SKILL.md
2 .claude/skills/build-semester-ui/SKILL.md
3 .mcp.json (Figma remote MCP, no credentials)
4 docs/design-system/FIGMA-MAPPING.md
5 app/scripts/design-system-audit.mjs
```

```
6 app/scripts/design-system-report.mjs
7 app/package.json scripts: design-system:audit, design-system:report, design-system:check
8 app/src/lib/designsystem.test.ts proving: the export is generated and in sync; raw values outside
  the hex ledger fail; every Figma mapping row resolves; audit/report fail on violating fixtures and
  pass on valid ones.

Show the file-by-file plan before editing. Revert each fix under its test once to watch it go red.
After editing run, from app/: npx tsc -b && npm run lint && npm test && npm run test:shuffle &&
npm run design-system:check. Report changed files, exact commands, results, limitations and any
Figma steps left for me.
```

---

3 · Day-to-day

```
/build-semester-ui screens/Degree.tsx https://www.figma.com/design/…?node-id=…
```

```
cd app && npm run design-system:report -- --out design-system-report.md
```

---

handoff/claude-code-toolkit/CLAUDE.md.design-system-section.md

---

The design system, and where each value is decided

Purpose. Build interfaces native to Semester — calm, source-aware, accessible, dense without clutter. Not a generic SaaS dashboard.

Source of truth, in order. `app/src/lib/look.ts` (`tokensFor`) decides every value → `app/src/styles/tokens.css` names them by job → `app/src/lib/tokenexport.ts` exports them → `app/design-tokens/semester.tokens.json` is **generated**: never hand-edit it; `npm run tokens:export` rewrites it and `tokenexport.test.ts` fails on drift. The `styles/*.test.ts` files are design contracts; extend them, never loosen them to go green.

Before writing UI. Search `components/unity/`, `components/ui.tsx`, the nearest screen in `screens/`, and the relevant `styles/*.test.ts`. Reuse first. Use `/build-semester-ui`; a new shared component goes through `/create-semester-component`.

Raw values. Feature UI uses semantic variables only — no raw colours, spacing, radii, shadows, z-indexes, font sizes, durations or easing. Permitted: `look.ts`, the first `:root` of `app.css`, the `hex.test.ts` LEDGER (with a reason), generated output, test fixtures. `npm run design-system:audit` reports the rest by file:line.

Accessibility. Native elements; visible focus (`--focus-color`, clear of sticky chrome); icon-only controls named; label/hint/error associated; every state as glyph + word + tone; motion only through `--motion-*`; keep `a11y/*` and `taps.test.ts` green.

Responsive. Follow `breakpoints.test.ts`, `gutter.test.ts`, `density.test.ts`, `taps.test.ts`, `stacking.test.ts`. Keep the primary action at every width; nothing hover-only.

Figma. Figma MCP is read-only design intent. Map variables through `docs/design-system/FIGMA-MAPPING.md` to exported paths and components to existing patterns before writing anything. Missing Figma states never excuse missing loading/empty/error/disabled/permission/keyboard/focus/responsive behaviour. Record unresolved mappings in that file. Never write to Figma.

Done means. From `app/`: `npx tsc -b && npm run lint && npm test && npm run design-system:check` (plus `npm run tokens:export` if tokens changed). Report files changed, patterns reused, states and a11y covered, and gaps left.

Tooling cards

Module	Decides	Rendered by
provenance.ts	Source rank, conflicts, freshness label	SourceBadge, ProvenanceChips
nextaction.ts	Order of a student’s next actions	Today, NextSteps
access.ts	Role × screen, share expiry, break-glass	Role lens, route guards
freshness.ts	Lag → display / safe / destructive	Read models, lag simulator
consequence.ts	Undo / preview / two-person	Dialog, ActionPreview
aigateway.ts	Scope + policy before any model call	AIResponse
approvals.ts	Two-person duty state machine	Console approvals
scoring.ts	Weighted 0–5 grading	Workflow lab
pilot.ts	Launch readiness	Business gap analysis

No dependencies, no I/O, vitest proof per module. Files end in .txt; strip on copy to app/src/lib/core/.

tooling semester core

010 Core

core.institutions, core.profiles, audit.events

020 Community

groups, members, posts, reports, events, rsvps

030 Market

listings, offers, drops, bids

040 Ops

staff, accounts, invoices, social_posts, settings

supabase/migrations
*.sql.txt → .sql

→

app · lib
validation, 2 API routes

→

tests
RLS + unit

→

npx supabase db reset
then npm test

Status: written to the architecture guide. Not yet run, typechecked or tested.

tooling semester platform

Prototype · ui_kits/workflow-lab/

Router
6 inputs → tool, confidence, reasons, limits, escalation, prompts

Matrix
16-column feature matrix, 12 canonical workflows

Benchmark
Persistence tests, 0–5 grading, weighted /5 and /100, leaderboard, CSV/MD export

Saved on this device only; labelled prototype mode.

Production · handoff/workflow-router-app/

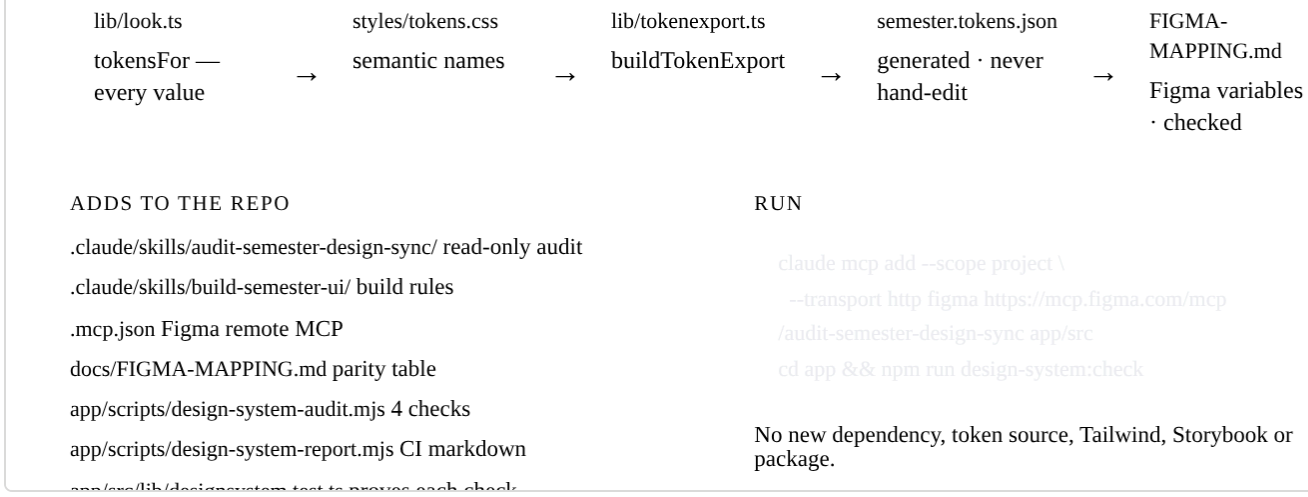
Stack
Next.js App Router, TypeScript strict, Tailwind, Supabase (@supabase/ssr, magic link), Zod, Server Actions

Data
001_core_workflow_router, 002_benchmark_grading; org-scoped RLS (owner · admin · member · viewer); seed with 12 workflows

Tests
vitest suites, RLS test plan

Run install.sh. Not yet typechecked, linted, tested or built.

tooling workflow router



tooling claude code

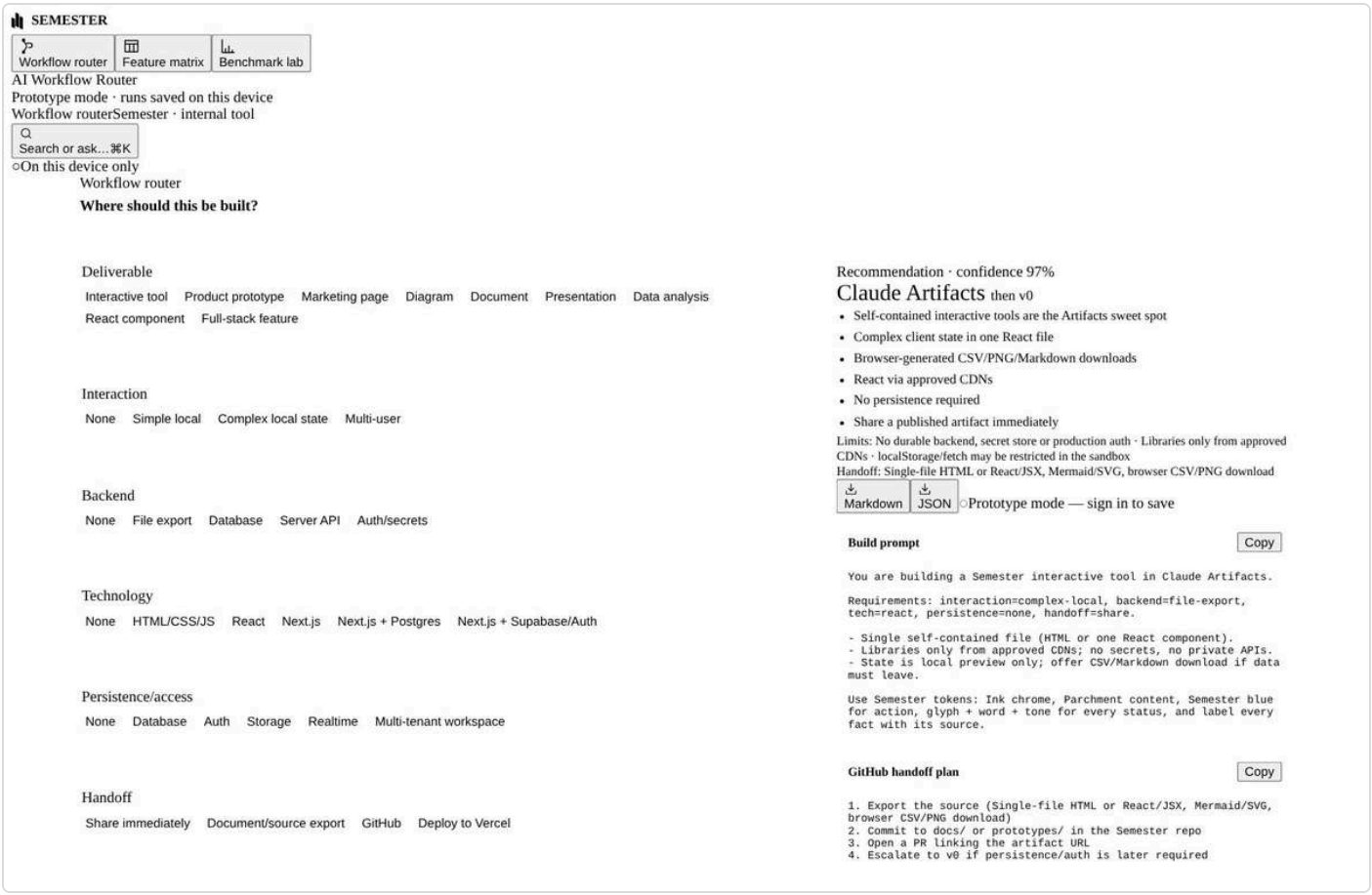
Stream	Built	Next
Brief	One-pager, FAQ, role sheets, overview deck	Real screenshot on slide 4
Guides	Architecture, developer, user	Admin manual, tutorials
Launch	Runbook, cutover plan, day-1/week-1 support	—
Secure	10 policies; HECVAT, SOC 2, pen test, insurance readiness	Stream index
Legal (drafts)	Terms, privacy, community, subprocessors	MSA, order form, DPA, cookie, accessibility
Build	—	Engineering playbook, QA and accessibility test plans

Rule: formal documents are doc_page files under docs/<stream>/, linked from docs/index.html. Legal stays draft until attorney review.

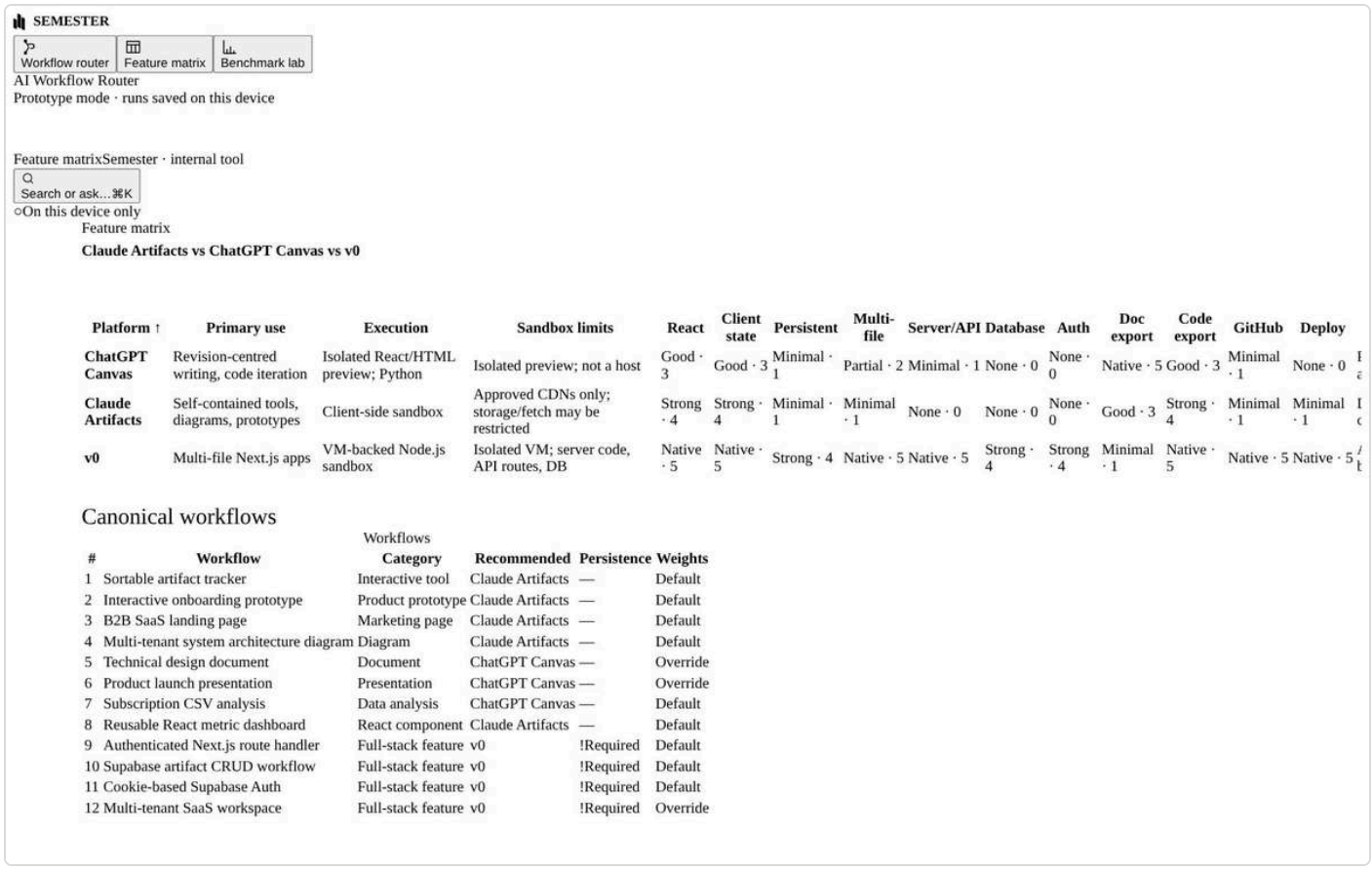
company documents

AI WORKFLOW ROUTER AND BENCHMARK LAB

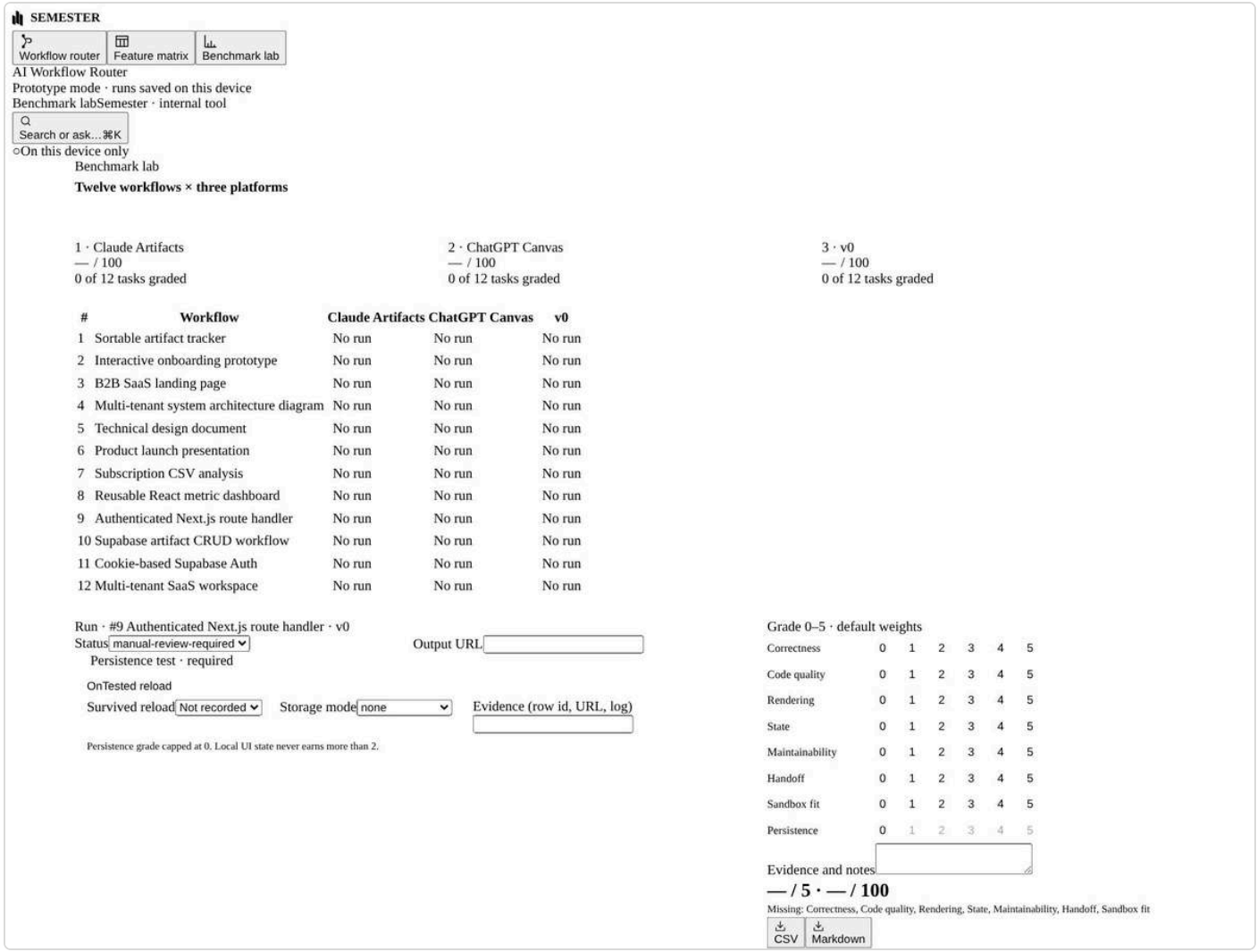
4 screens.



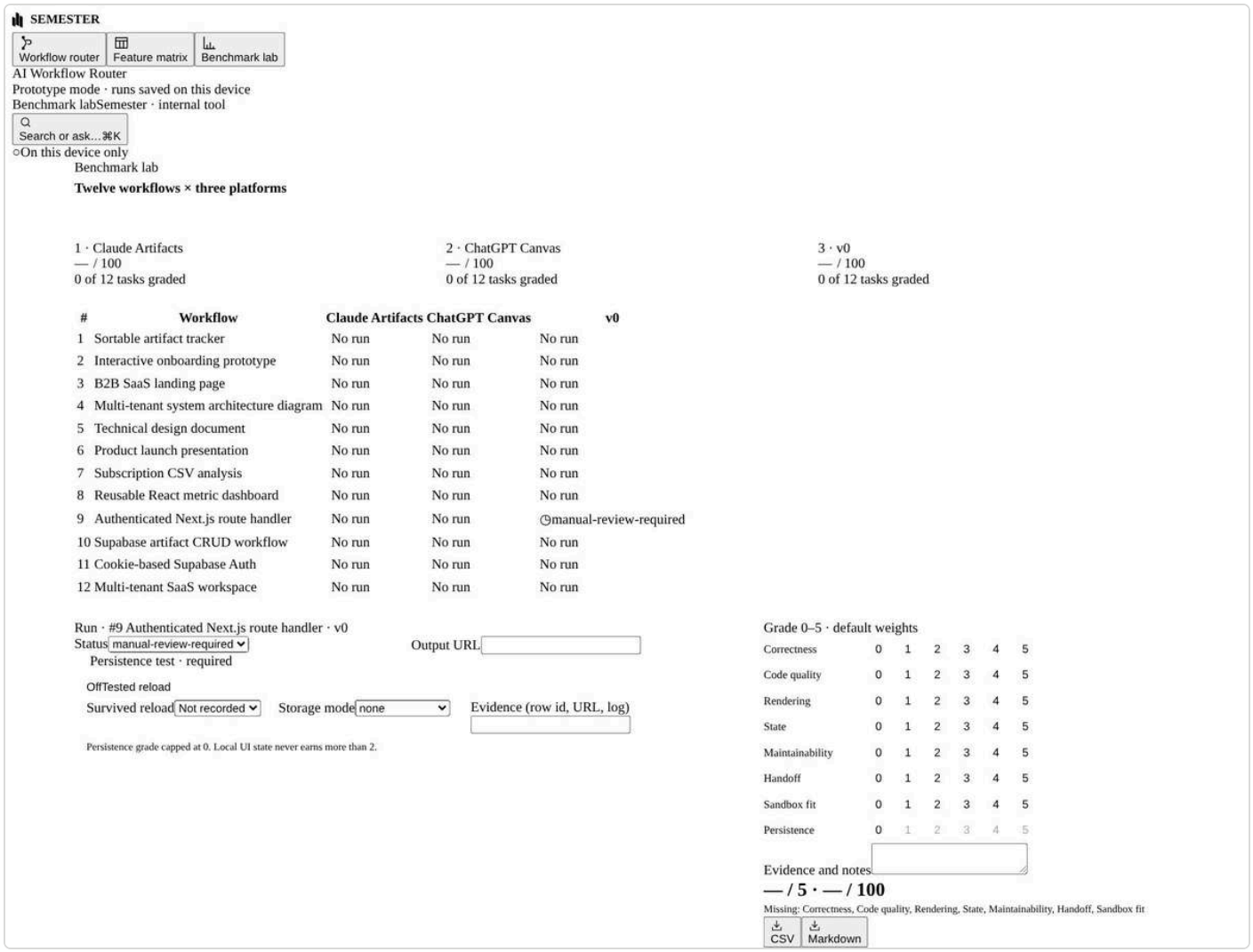
Opening view



Feature matrix



Benchmark lab



Benchmark lab › OnTested reload

Prototype notes

ui_kits/workflow-lab/README.md

AI Workflow Router & Benchmark Lab (prototype)

Interactive prototype of the app specified in the pasted brief, built from this design system. Router (6 inputs → primary/secondary, confidence, reasons, limits, escalation, handoff, prompts, GitHub plan, Supabase prompt, MD/JSON export), sortable 16-column feature matrix, the 12 canonical workflows, and a benchmark grid with per-run persistence test, 0–5 grading with the persistence ceiling, weighted scores, leaderboard and CSV/Markdown export.

Prototype mode: runs are saved to this browser only (localStorage) and labelled so. The production app — Next.js + Supabase with RLS — is in `handoff/workflow-router-app/`; `lab-engine.js` is a plain-JS port of its `lib/` and must be kept in step.

---

ui_kits/student/README.md

Student app UI kit

Desktop recreation of the Semester student workspace: dark Ink side rail, warm Parchment content, persistent system context bar with ⌘K palette.

Screens (click the rail): **Today** (index), **BIO 210** course workspace, **Assistant** (governed AI answer → action preview → confirm), **Family** (consent-driven sharing + access history), **Campus**.

Files: `StudentApp.jsx` shell + routing, `Today.jsx`, `Course.jsx`, `Assistant.jsx`, `Family.jsx`, `Campus.jsx`, `data.jsx` sample data. All primitives come from `window.SemesterDesignSystem_e9f3ba`.

Source: composed from the shared "unity" primitives in harrisonjrubin7-cmyk/semester (`app/src/components/unity/*`, `StartToday.tsx`, `desk/Sidebar.tsx`). The production app supports six navigations and three shells; this kit shows the desktop rail layout only.

---

ui_kits/staff/README.md

Staff workspaces UI kit

Faculty, Advisor, Registrar and Finance workspaces in one shell — switch roles from the rail. A dark scope bar names the role, tenant and identity on every screen.

- **Faculty** — gradebook with draft → release, regrade decisions. Source: `app/src/components/gradebook/InstructorBook.tsx`.
- **Advisor** — student-controlled meeting shares and authority scope. Source: `AdvisorMeeting.tsx`, `AdvisorSharedView.tsx`.
- **Registrar** — approval requests, capacity, record changes. Source: `enrollment/RegistrarDesk.tsx`, `institutional/OperationsStudio.tsx`.

- **Finance** — student account ledger and two-person requests. Source: `institutional/StudentAccounts.tsx`, `console/Approvals.tsx`.

`ui_kits/console/README.md`

Institution console UI kit

The enterprise control plane: dark environment bar (environment / scope / operator / MFA / session — the fields `CONTEXT_BAR` names in the source), Ink rail, Parchment content.

Screens: **Overview** (index), **Approvals** (two-person duty with ActionPreview + MFA confirm), **Integrations**, **Audit log**.

Source: harrisonjrubin7-cmyk/semester `app/src/components/console/*` (Approvals.tsx, ContextBar.tsx, Audit.tsx) and `institutional/IntegrationDashboard.tsx`. Copy for Approvals is lifted from Approvals.tsx.

`ui_kits/mobile/README.md`

Mobile UI kit

Two phones (Parchment and Ink grounds) with the five canonical destinations from the source — **Today, My Path, Search, Plan, Me** — on the labelled bottom tab bar. Each phone is independently clickable.

Files: `MobileScreens.jsx` (MToday, MPath, MSearch, MPlan, MMe), `MobileApp.jsx` (Phone frame + TabBar).

Note: per docs/BRAND-PLATFORM.md §3, no native iOS/Android binary exists; this is the responsive web app at phone width.

Prototype-to-codebase map

github.md

repo: harrisonjrubin7-cmyk/semester branch: main path: app/src

Last sync

date: 2026-10-05T03:38:34Z

Upstream since last sync

- `docs/design-system/` (FIGMA-MAPPING, README, BASELINE) and `workflow-lab/` (Next.js app) have landed.
- Not yet in repo: `.mcp.json`, the three new skills, `app/scripts/design-system-*`, baseline JSON.
- No `--source-*` / Semester-blue tokens in `app/src/styles` — this project's source palette remains a proposal.

Previous sync

date: 2026-10-05T00:49:21Z

Updated in this project

- Seven navigation areas and App directory from lib/navareas.ts + screens.tsx
- 14 new student screens (Calendar, Study, Write, Sheets, Decks, Mail, Connect, Settings, Privacy, Support, Community, Maps, Notifications, Me)
- TA, Student affairs, Security operations, Support queue; new External audiences kit

Sync history

- 2026-10-04T21:30:28Z — voice/naming, SourceLine, student + staff + mobile kits
- 2026-10-04T21:10:19Z — initial tokens, components, Student + Console kits

Screen map

Project screen	Repo files
ui_kits/student — Today	app/src/components/StartToday.tsx, components/unity/ObjectCard.tsx, unity/NextSteps.tsx
ui_kits/student — Course	components/unity/ContextBar.tsx, unity/Table.tsx, SourceBadge.tsx

Project screen	Repo files
ui_kits/student — Assistant	components/unity/States.tsx, unity/ActionPreview.tsx, lib/source.ts
ui_kits/student — Family	components/unity/Visibility.tsx, unity/DecisionTrail.tsx
ui_kits/student — Shell	components/desk/Sidebar.tsx, unity/SystemContextBar.tsx
ui_kits/console — Approvals	components/console/Approvals.tsx, console/Fields.tsx
ui_kits/console — Shell/Overview/Audit	components/console/ContextBar.tsx, console/Audit.tsx, institutional/IntegrationDashboard.tsx
ui_kits/student — Registration	app/src/components/enrollment/StudentRegistration.tsx
ui_kits/student — Grades	app/src/components/gradebook/StudentGrades.tsx
ui_kits/student — Advising	app/src/components/AdvisorMeeting.tsx
ui_kits/student — Degree, Billing, Career	docs/BRAND-PLATFORM.md, components/CareerEvidence.tsx (not read in full)
ui_kits/staff — Faculty	app/src/components/gradebook/InstructorBook.tsx
ui_kits/staff — Advisor	app/src/components/AdvisorMeeting.tsx, AdvisorSharedView.tsx
ui_kits/staff — Registrar, Finance	enrollment/RegistrarDesk.tsx, institutional/StudentAccounts.tsx
ui_kits/mobile	lib/look.ts (NAVS tabs), docs/BRAND-PLATFORM.md §1.8
ui_kits/student — App directory, areas	app/src/lib/navareas.ts, app/src/screens.tsx
ui_kits/student — Calendar, Study, Write, Sheets, Decks, Mail, Connect, Settings, Privacy, Support, Community, Maps, Notifications, Me	app/src/screens/* (names + areas only; layouts composed from shared primitives)
ui_kits/staff — TA, Student affairs	gradebook/InstructorBook.tsx, docs/BASIC-NEEDS-NAVIGATOR.md (not read)
ui_kits/console — Security, Support queue	console/BreakGlass.tsx, console/SupportQueue.tsx (not read in full)
ui_kits/external	docs/BRAND-PLATFORM.md §1.1, §3
handoff/claude-code-toolkit	lib/tokenexport.ts, docs/DESIGN-TOKENS.md, styles/hex.test.ts, design-tokens/semester.tokens.json, .claude/skills/
ui_kits/workflow-lab, handoff/workflow-router-app	uploads/list every artifact and thing for claud to genrate.md (spec), live site https://harrisonjrubin7-cmyk.github.io/semester/
ui_kits/architecture	uploads/arch-map.pdf, cqrs.pdf, sim.pdf, chatgpt.pdf (specs, not repo code)
tokens/*	app/src/styles/tokens.css, industry.css, unity.css, typefaces.css, lib/look.ts

What's next

NEXT-SESSION.md

Semester — next session brief

Start a new conversation in this project and paste: **"Read NEXT-SESSION.md and continue."**

Built (open ui_kits/master-control/index.html — every item is linked)

- Design system: tokens, 57 components, 68 cards, 7 templates (company, investor, institution, internal decks; student + console screens; pilot results report).
- Product: student app (64 screens, Tweaks, live Ask Semester), mobile, staff, console, external, investor portal.
- Maps of the whole: Master catalog (51 roles · 673 screens · 319 workflow steps (17 workflows) · 88 systems · 82 docs), Atlas, Role lens (22 roles), wireframe catalogue, forms/capabilities/interfaces (12 forms).
- Architecture: OS map, full stack & data (20 tables, flows, operations), read-model pipeline, lag simulator, AI gateway, phases.
- Company: business workspace, operating manual, operating pack, market research, compliance audit, company site, Vanderbilt one-pager, invitation email, campus map, 30 s demo video, company deck PPTX.
- Handoff: handoff/ (claude-code-toolkit, semester-core, workflow-router-app) — see handoff/README.md.

Done Oct 5 2026

- ui_kits/workflow-runner/: Semester OS map (21 layers with modules from uploads/Map Out entire Semester OS.pdf, 14-check control chain, journeys, 21 required shared workflows, 22-point completion definition) + runner for all 207 catalogued workflow steps (actor, hand-off, system, screen match, rendered step, audit log, auto-run), by-role view (49 roles), screen coverage and gap list. Linked from master-control. Batch C done: 7 added workflows (finance, campus, community & moderation, career/employer/alumni, family consent, developer platform, analytics & outcomes; 112 steps, labelled "Added · not in catalog yet") so every one of the 21 layers now has a runnable workflow. Next: fold the added workflows into catalog-data.js and write docs/master/*.md.
- Design System tab: 10 new cards (arch-*, tooling-semester-core/platform/workflow-router, company-documents, maps-of-the-whole, iconography-rules).
- docs/master/: all 13 catalogs + README, generated from catalog/runner/OS-map data (84 step screen gaps, 13 roles with no workflow, 419 screens no workflow uses — see SEMESTER_GAP_REGISTER.md).
- Templates from the linked Semester Platform project: templates/platform-shell, institution-os, command-center, role-onboarding (RoleOnboarding role prop → 13 role DCs), operating-map, finish-line, vision-deck. This design system wins on conflicts.
- handoff/execute/: 00-INDEX.md master prompt + 12 stream prompts (00 setup … 11 company site/grow), branch semester/, PR + report per stream.
- DONE: docs/grow/GTM Plan.html (+ index).

- DONE: full public site in ui_kits/company-site (10 pages, hash routes #/product … #/legal, responsive, forms, legal + accessibility statement).
- DONE: docs/legal MSA, Order Form, DPA (drafts, attorney review). Still: cookie notice, accessibility statement as a standalone doc.
- DONE: docs/run/Finance Model.html (setup lean $4,873 / base $12,430 / full $19,910; run-rate $565/mo).
- DONE: docs/run/Hiring Plan.html (10-role plan, 6 JDs, contractor briefs).
- DONE: docs/build/ Engineering Playbook, QA Test Plan, Accessibility Test Plan (+ index).
- DONE: docs/operate/ Operations Handbook (SLOs, error budget, on-call, alerts, monitoring, cadence) + Runbooks (R1–R8, macros).
- DONE: docs/legal Cookie Notice + Accessibility Statement; docs/secure/index.html.
- DONE: ui_kits/role-pages/ — one page per catalogued role (49 in catalog-data; brief said 51), linked from master control; workflow buttons open the runner at the step.
- DONE: ui_kits/gap-screens/ — the 84 step screen gaps added to the catalog (CATALOG.addedScreens; 673 screens total) and rendered in the shell. Regenerate docs/master/* to drop them from the gap register.
- DONE: batch A baseline — ui_kits/every-screen/ renders all 673 catalogued screens (archetype per name: dashboard/queue/list/builder/settings/detail/marketing) in six states; ◆ designed in a kit, ◖ from a workflow gap, ○ archetype only.
- NEXT: replace archetype screens with bespoke designs, highest-traffic first (B student, E registrar, K console); regenerate docs/master/*. EVERY ROLE pages (51) and batch A missing screens.
- Source project (done): /projects/7dd6738a-405e-41a6-9c02-ac359bb30017 (Semester Platform, Institution OS, Company Command Center, 14 role onboardings, Master Operating Map, Finish Line, Vision Deck).

Founder's latest ask (top priority)

"Every role in Semester. Finish, generate and produce everything I need to run, grow, build, operate, maintain, secure and launch Semester." Batch N is the master plan. Do it one stream per session, in this order, and link every output from master-control: 1. LAUNCH: COMPLETE. docs/launch/ has index, Launch Runbook, Cutover Plan and Support Plan (day-1/week-1). Next is BUILD (stream 3). SECURE has no docs/secure/index.html yet. Original list: launch runbook, go/no-go checklist, cutover plan, comms plan, day-1/week-1 support plan, rollback. 2. SECURE: the policy set at docs/secure/Security Policies.html is DONE. docs/secure/Security Readiness.html (HECVAT, SOC 2, pen test, insurance) is DONE too, so SECURE is complete. Next is BUILD. Original list: HECVAT Full answers draft, security policy set (access control, encryption, vendor management, incident response, BCP/DR, data retention, acceptable use), SOC 2 readiness tracker, pen-test scope, insurance checklist. 3. BUILD: engineering playbook (repo layout, branching, CI/CD on Vercel + Supabase, RLS test plan, environment matrix, release process), QA test plan per role, accessibility test plan (VPAT/ACR prep). 4. OPERATE / MAINTAIN: site operations console (batch E), on-call and SLOs, monitoring and alerts, SIS sync runbooks, backup and restore drills, dependency/patch cadence, support macros. 5. RUN: company operating cadence (weekly metrics, monthly close, quarterly planning), legal stack (LLC, contracts, DPA template, MSA, order form, privacy policy, terms), finance model (pre-revenue budget within the $5k–$20k setup range), hiring plan per role, a job description for each. 6. GROW: GTM plan (ICP, pipeline, pricing, case study from the pilot), the full public site (batch F), content calendar, partner program, fundraising data room index. 7. EVERY ROLE: one page per catalogued role (51), covering purpose, screens, permissions, workflows, KPIs and day-in-the-life. Render any missing screens (batch A). Build formal docs as doc_page files under docs//, and give each stream an index page.

Also done (Oct 2026)

- docs/guides: Architecture, Developer and User guides.
- handoff/semester-platform: migrations 010–040 (core, community, market, ops, all with RLS), validation, 2 API routes, RLS and unit tests. None of it has been run yet.
- Still to do: an admin MANUAL (operations manual for institution admins and support) and TUTORIALS (step-by-step, for students, staff and new developers).
- 0. Device and media capabilities, each permission-gated with a clear ask, a fallback, and a revoke path in Privacy settings:
- Camera: scan textbook ISBNs for listings, photograph items, ID check for campus verification (handled by the institution IdP where possible), video office hours.
- Microphone: voice notes, dictation in Write, voice questions to Ask Semester (transcribed on device where possible; audio never stored by default).
- Location: campus map, safe meet-up spots, event check-in, shuttle. Only while the app is in use, precise or approximate as the user chooses, never sold, and off by default.
- Image and video generation: for study visuals, club flyers and event promos, through the AI gateway with a content-safety filter, labelled AI-generated, with no real-person likeness and no student records.
- Notifications (push/email/SMS per channel), offline mode, PWA install, biometric sign-in on the device.
- Multi-user and multi-device: account switcher (student who is also a TA, staff in two roles), sessions list with remote sign-out, handoff between phone and desktop, shared family view only with student consent, delegated access with expiry.
- More personalization (expands H): themes (13 grounds × 11 accents from the app), density, text size, calm mode, dyslexia font, reduced motion, home layout builder, language, time format.
- Build: a Capabilities & Permissions screen in the student app, a device settings page, permission-prompt patterns as design-system cards, and an RLS/migration for sessions and delegations in handoff/semester-platform.
- LEGAL DRAFTS DONE: docs/legal has Terms, Privacy, Community Guidelines and Subprocessors, all marked as needing attorney review. docs/index.html is the library of every document. Still to do: MSA, order form, DPA template, cookie notice, accessibility statement.
- MANUAL + TUTORIALS (requested again): an admin and operations manual, plus step-by-step tutorials for students, staff and developers. Both go under docs/guides.

Founder's earlier ask

"Every screen rendered — capability, workflow, user role, business operations, site operations, and the actual site." Work in batches, one area per session. Each batch ends by linking new files from ui_kits/master-control/index.html: - A. Render every catalogued screen still missing, role by role (catalog = 51 roles · 673 screens). Use shared shells and components; group roughly 10–15 screens per file, one file per role. - B. Capability map rendered as screens: each capability → the screens and roles that use it. - C. DONE: ui_kits/workflow-runner/ — all 319 steps, clickable end to end, per role. - D. Business operations console (finance, sales/revops, CS, legal/compliance, hiring). No student records. - E. Site operations console (deploys, uptime, incidents, SIS sync, AI gateway, support queue, audit log). - F. The actual public site: full multi-

page marketing site (home, product, institutions, students, security/trust, pricing, about, contact, legal). Build on the existing company site. - G. DONE (Oct 2026): 9 logo SVGs in assets/ plus the components/LogoSystem/preview.html card. Outstanding: outline the wordmark text to paths. Original brief: More logos: logo variants (horizontal, stacked, mark-only, mono, reversed, favicon/app icon, co-brand lockup with partner institution). Add them to assets/ and give them a brand card. - H. Personalization capabilities: add them to the capability catalog and render them as screens. Covers learner profile and goals, adaptive study plan, pacing and reminders, content recommendations, accessibility and display preferences, notification preferences, AI tutor tone and level, dashboard layout, and institution-level theming. Every item has privacy controls (consent, view and reset my data, opt-out). AI never sees student records. - I. Update pass: audit catalog counts, master-control links, the readme and SKILL.md against what's built; fix anything stale or broken. - J. PARTLY DONE: ui_kits/ops/index.html has 12 consoles and is linked from master-control. Social and Finance are deep; the other 10 are one queue each and need more tabs. - K. Documents: every formal document (policies, FERPA/SOC 2/accessibility compliance docs, audit workflows, test plans, runbooks, DPA, incident response, AI use policy, data retention). Build them as doc_page documents in docs/, with an index. - L. PARTLY DONE: docs/brief/ has index, One-pager and FAQ. Role sheets are done too. The overview deck (templates/overview-deck/OverviewDeck.dc.html) is done too, so L is complete. Add a real screenshot on slide 4. Ops consoles now carry a "sample data" banner. Original brief: Product brief package to hand out: a one-pager, a 10-slide overview deck, an FAQ, and a role-by-role "what Semester does for you" sheet. Print/PDF ready, with one index page that links them all. - M. PARTLY DONE: student app at ui_kits/community. Staff side lives in the ops T&S console (health, marketplace ops, live drops, institution settings). Mentorship, clubs and leader tools are now built too. M is done apart from deeper polish. Started: ui_kits/community/index.html. Expand it to cover: - Social: campus-verified profiles controlled by the student, with visibility per field; follows and connections; a course and club feed; study groups; DMs with safety tools; events and RSVPs; mentorship matching. - Community: clubs and orgs, course communities, Q&A with endorsed answers, reputation that carries no academic weight, ambassador and peer-leader roles. - Live marketplace: textbooks, gear, housing and sublets, tutoring and services, and event tickets. Listings are campus-verified, with in-app offers and chat, safe meet-up spots, and live drops and auctions. Disputes and reports go to the moderator workspace. - Guardrails: no sale of graded work or exam material (academic integrity filter), the institution controls whether each feature is on, minors and age rules respected, moderation queue, AI pre-screen only on public posts, no student records in social or AI. Payments go through a processor; Semester never holds funds without a license. - Staff side: community health, moderation and marketplace ops tabs inside ui_kits/ops (T&S console). - (orig) J. Operations consoles, each fully built, interactive and rendered (one file each, shared lnk console shell, linked from master-control): - Social media control: unified inbox, compose/schedule calendar, approval queue, brand-safety and crisis mode, per-channel analytics, institution-partner co-posts. - Finance: ARR/MRR, invoices and AR aging, contracts and renewals, usage-based billing per institution, AI gateway cost, payroll summary, close checklist, audit export. - Also: marketing/campaigns, sales/RevOps pipeline, customer success health, support desk, trust and safety/moderation, legal/compliance (FERPA, DPAs, subprocessors), people/HR, IT/security (SOC 2 evidence), data/analytics, partnerships. - Each console needs real-feeling data, working filters and tabs, row detail drawers, and role permissions. Business data only; no student PII. Then continue with the list below.

Do next, in order

1. Export investor, institution and internal decks to PPTX (gen_pptx, same as company deck).
2. Deepen P1 workflow screens from the catalog gap register: registration readiness read model, support SLA/incident, SIS reconciliation.
3. Course Studio (faculty authoring) and moderator workspace screens — currently catalogued, not built.

4. Write the highest-value docs from the catalog's "Not started" list (AI_GOVERNANCE_POLICY, DATA_INVENTORY, INCIDENT_RESPONSE, PILOT_DELIVERY_FACTORY) as doc-page files.
5. Run handoff/claude-code-toolkit/EXECUTE.md in the repo (needs Claude Code, not this project).

Executable hand-off (Oct 2026)

- handoff/BUILD.md is the single entry point: rules, preflight, app target (Next.js; create web/ if app is Vite), install, stream order + gates, migration order 010–080, verify, human-only steps, definition of go. Scripts: handoff/preflight.sh, install.sh (no-overwrite, strips .txt), verify.sh (db reset, tsc, lint, unit, RLS/security/command, design-system, build, gitleaks). handoff/manifest.json. Seed (synthetic users with auth.identities), tests/helpers.ts, .env.test, .env.example added.
- Launch kit: sponsor weekly update email, incident notice email, staff training flier.

Security deepening (Oct 2026)

- Reviewed migrations 010–040: 11 findings (F2 critical: ops.staff self-insert as ceo; F5 high: profile institution_id change; F1, F4 high). Fixed in handoff/semester-platform/supabase/migrations/050_security_hardening.sql.txt, proven by tests/security.test.ts.txt; docs/secure/Platform Security Review.html; stream 01 updated.
- 060_command_functions: all community/market writes via audited api.* functions (validation, rate limits, integrity filter, campus checks), direct inserts revoked, listing-photo storage policies; tests/commands.test.ts.
- AI gateway hardened: content scan blocks records typed into prompts (lib/ai/redact.ts), route app/api/ai/route.ts, 070_ai_gateway (tenant policy, course rules, usage log without prompt content), 20-case red-team suite.
- 080: approved-source AI context, private storage buckets, audited delegations; security headers middleware; CI security job (gitleaks, audit, guards, security tests).
- templates/privacy-center: student privacy & data screen (AI by data type with institution prop, sharing revoke, sessions sign-out, access history, export, deletion hand-off).
- 7 governance components added (components/governance/). Launch kit imported (ui_kits/launch-kit + 5 templates).

Strategy (Oct 2026)

- docs/strategy/ (12 docs + README): competitive completion audit (20 areas), 19-dim competitive scorecard (~1.1/5), coherence test as release gate, implementation factory, mobile, AI differentiation + deterministic policy engine, trust advantage, 36-month plan. Competitive view added to ui_kits/platform-maturity.

Business, design & data systems (Oct 2026)

- docs/business/ (17 + 5 subsystem docs: finance, revenue, trust, customer, people; SoR map, risk register, cadence, KPI tree, backlog, top 50). Price book updated: Student Premium $8.99/mo or $69/yr, AI overage $30/1k requests, implementation $35k–$150k, marketplace 12%.

- docs/system/ (design 8, algorithms 7, workflows 2, data 4 + README with next 25). OPEN DECISION: PDF proposes indigo primary + emerald/amber/rose; system keeps Semester blue + stone hues until founder decides.

Platform maturity (Oct 2026)

- From uploads/what else is needed...pdf + Apple vs Google vs Microsoft...pdf: docs/platform/ (16 + 2 docs), ui_kits/platform-maturity/ (3 tiers, 12 pillars, 24-dim scorecard at ~1.2/5 design stage, API domains + contract, graph node fields + authority categories, 36-month roadmap, 7 stages, deployment ladder, moats/risks, next 50 actions), stream handoff/execute/13-platform-maturity.md.

Product principle (Oct 2026)

- Nobody has to leave Semester: ui_kits/one-place/ maps 45 jobs (students, staff, institutions, company) to modules; regulated or institution-owned systems (bank, processor, payroll, IdP, SIS during transition, GitHub/Vercel) are embedded, not replaced. Stream handoff/execute/12-one-place.md.

Rules to keep

- Nothing is "verified" without repo code, tests and release evidence.
- Company roles (investor, CS, revops) never see student records.
- Status, not certifications we don't hold. Cite every external number.

Founder facts

LLC (home state), solo founder, bootstrapping, pre-seed no revenue. Paid Vanderbilt pilot signed: real student records, DPA/FERPA addendum, HECVAT Full, SAML SSO, VPAT/ACR, cyber + E&O insurance required (not held); dates not set. Hosting Vercel + Supabase; AI never sees student records. Setup budget $5k–$20k. Vanderbilt name cleared for public use.

SEMESTER

Document library

Every formal Semester document. Each one prints or saves as a PDF. Legal documents are drafts until an attorney reviews them.

Product brief

Brief package
One-pager, FAQ, role sheets, deck

Guides

Architecture guide
For architects

Developer guide
For engineers

User guide
For every role

Operating manual
How to run the company

Company & system

docs/business
Company OS: 15 functions, 5 systems, KPI tree, cadence, backlog

docs/system
Brand, UI grammar, components, algorithms, workflows, data

Strategy

docs/strategy
Competitive audit, scorecard, moats, implementation factory, AI, trust, 36-month plan

Platform

docs/platform
Maturity model, 12 pillar standards, scorecard, APIs, 36-month roadmap

Platform maturity program
Interactive

Master catalogs

docs/master
13 generated catalogs: map, roles, screens, workflows, systems, gaps

OS map & workflow runner
Interactive version

Build

Engineering playbook
Repo to release

QA test plan
By role

Accessibility test plan
VPAT / ACR prep

Launch

Pilot launch runbook
Gates, timeline, rollback

Production cutover plan
Staging to production

Day-1 / week-1 support
Tiers, severities, review

Launch index
All launch documents

Run

Finance model
$5k–$20k setup, run-rate, runway

Hiring plan
Plan + 6 job descriptions

Run index
All run documents

Grow

GTM plan and price book
Offers, prices, pipeline

Grow index
All grow documents

Operate

Operations handbook
SLOs, on-call, alerts

Runbooks
SSO, SIS sync, restore, exposure, AI

Security

Security policy set
10 policies

Security readiness

Platform security review
11 RLS findings fixed

HECVAT, SOC 2, pen test, insurance

Secure index
All security documents

Legal (drafts)

Terms of Service
Users

Privacy Policy
FERPA-aligned

Community guidelines
Plain-language rules

Subprocessors
Vendor list

Cookie notice
Storage, no ad tracking

Accessibility statement
WCAG 2.2 AA status

Master Services Agreement
Institutions

Order form
Pilot + conversion

Data Protection Addendum
FERPA school-official terms

APPENDIX

Source code

The stylesheets, types, component sources and every engineering package, in full.

Token and component stylesheets

The generated token stylesheet and the component stylesheet, in full.

tokens.css · 251 lines

```
/* The Main Semester design system — generated from tokens.json */
:root, [data-theme="light"] {
  --ink-void: #040507;
  --ink-bg: #090a0e;
  --ink-panel: #12141a;
  --ink-hero: #191c23;
  --ink-raise: #22262f;
  --ink-fg: #eceef2;
  --parchment-void: #e8e4dc;
  --parchment-bg: #f4f1ea;
  --parchment-panel: #fbf9f4;
  --parchment-hero: #ffffff;
  --parchment-raise: #ffffff;
  --parchment-fg: #1b1a17;
  --graphite-bg: #16181c;
  --graphite-panel: #1f2229;
  --graphite-raise: #333843;
  --sterling: #d4d9e2;
  --sterling-bright: #f6f8fb;
  --sterling-deep: #9ca3b2;
  --sterling-shade: #4c5561;
  --brass: #d8c79a;
  --brass-bright: #f2e7c8;
  --brass-deep: #b09f76;
  --brass-shade: #695a33;
  --blue-100: #eef6ff;
  --blue-200: #d6ebff;
  --blue-300: #b5d9fd;
  --blue-400: #94bce3;
  --blue-500: #749dc4;
  --blue-600: #597ea3;
  --blue-700: #416180;
  --blue-800: #2c455d;
  --blue-900: #1d2d3d;
  --neutral-100: #f5f5f8;
  --neutral-200: #e7e7ea;
  --neutral-300: #d4d4d7;
  --neutral-400: #b7b7ba;
  --neutral-500: #98989b;
  --neutral-600: #7a7a7d;
  --neutral-700: #5d5d60;
  --neutral-800: #424244;
  --neutral-900: #2b2b2d;
  --official-ink: #2c455d;
  --official-tint: #b5d9fd;
  --connected-ink: #2d5c61;
  --connected-tint: #9fcbcd;
  --personal-ink: #4a4b4f;
  --personal-tint: #b9b9bd;
  --ai-ink: #544a7f;
  --ai-tint: #bcb2df;
  --pending-ink: #7a5310;
  --pending-tint: #e2c27a;
  --risk-ink: #a3301f;
  --risk-tint: #f0907a;
  --success-ink: #3e6141;
  --success-tint: #a9c9a3;
  --surface-sunken: var(--parchment-void);
  --surface-canvas: var(--parchment-bg);
  --surface-base: var(--parchment-panel);
  --surface-quiet: #f1ede4;
```

```
  --surface-raised: var(--parchment-hero);
  --surface-overlay: var(--parchment-raise);
  --text-primary: var(--parchment-fg);
  --text-secondary: rgba(27,26,23,.68);
  --text-tertiary: rgba(27,26,23,.52);
  --text-disabled: rgba(27,26,23,.52);
  --text-on-action: #ffffff;
  --text-link: var(--blue-700);
  --border-subtle: rgba(27,26,23,.09);
  --border-default: rgba(27,26,23,.16);
  --border-strong: rgba(27,26,23,.34);
  --action-primary: var(--blue-700);
  --action-primary-hover: var(--blue-800);
  --action-primary-press: var(--blue-900);
  --action-secondary: rgba(65,97,128,.08);
  --action-secondary-hover: rgba(65,97,128,.14);
  --focus-color: var(--blue-600);
  --accent-editorial: var(--brass-shade);
  --source-official: var(--official-ink);
  --source-official-line: rgba(44,69,93,.45);
  --source-official-wash: rgba(44,69,93,.08);
  --source-connected: var(--connected-ink);
  --source-connected-line: rgba(45,92,97,.45);
  --source-connected-wash: rgba(45,92,97,.08);
  --source-personal: var(--personal-ink);
  --source-personal-line: rgba(74,75,79,.40);
  --source-personal-wash: rgba(74,75,79,.06);
  --source-ai: var(--ai-ink);
  --source-ai-line: rgba(84,74,127,.45);
  --source-ai-wash: rgba(84,74,127,.08);
  --status-pending: var(--pending-ink);
  --status-pending-line: rgba(160,112,24,.50);
  --status-pending-wash: rgba(196,146,48,.12);
  --status-danger: var(--risk-ink);
  --status-danger-line: rgba(163,48,31,.45);
  --status-danger-wash: rgba(163,48,31,.08);
  --status-success: var(--success-ink);
  --status-success-line: rgba(62,97,65,.45);
  --status-success-wash: rgba(62,97,65,.08);
  --status-neutral: var(--text-secondary);
  --chrome-surface: var(--ink-bg);
  --chrome-surface-raised: var(--ink-panel);
  --chrome-text: var(--ink-fg);
  --chrome-text-secondary: rgba(236,238,242,.64);
  --chrome-line: rgba(236,238,242,.10);
  --chrome-active: rgba(148,188,227,.14);
  --scrim: rgba(4,5,7,.5);
  --brand-canvas: var(--surface-canvas);
  --brand-surface: var(--surface-base);
  --brand-ink: var(--text-primary);
  --brand-muted: var(--text-secondary);
  --brand-accent: var(--action-primary);
  --brand-focus: var(--focus-color);
  --lift-1: 0 1px 2px rgba(0,0,0,.4);
  --lift-2: 0 1px 2px rgba(0,0,0,.45),0 6px 18px rgba(0,0,0,.35);
  --lift-3: 0 1px 3px rgba(0,0,0,.5),0 12px 36px rgba(0,0,0,.5);
  --shadow-sm: 0 1px 2px rgba(43,43,45,.14);
  --shadow-md: 0 3px 10px rgba(43,43,45,.16);
  --shadow-lg: 0 12px 32px rgba(43,43,45,.22);
}
[data-theme="dark"] {
  --surface-sunken: var(--ink-void);
  --surface-canvas: var(--ink-bg);
  --surface-base: var(--ink-panel);
  --surface-quiet: #161820;
  --surface-raised: var(--ink-hero);
  --surface-overlay: var(--ink-raise);
  --text-primary: var(--ink-fg);
  --text-secondary: rgba(236,238,242,.64);
  --text-tertiary: rgba(236,238,242,.42);
  --text-disabled: rgba(236,238,242,.42);
  --text-on-action: #08090c;
  --text-link: var(--blue-300);
  --border-subtle: rgba(236,238,242,.08);
  --border-default: rgba(236,238,242,.14);
  --border-strong: rgba(236,238,242,.30);
  --action-primary: var(--blue-400);
```

```
  --action-primary-hover: var(--blue-300);
  --action-primary-press: var(--blue-500);
  --action-secondary: rgba(148,188,227,.10);
  --action-secondary-hover: rgba(148,188,227,.18);
  --focus-color: var(--blue-400);
  --accent-editorial: var(--brass);
  --source-official: var(--official-tint);
  --source-official-line: rgba(181,217,253,.40);
  --source-official-wash: rgba(181,217,253,.08);
  --source-connected: var(--connected-tint);
  --source-connected-line: rgba(159,203,205,.40);
  --source-connected-wash: rgba(159,203,205,.08);
  --source-personal: var(--personal-tint);
  --source-personal-line: rgba(185,185,189,.34);
  --source-personal-wash: rgba(185,185,189,.06);
  --source-ai: var(--ai-tint);
  --source-ai-line: rgba(188,178,223,.42);
  --source-ai-wash: rgba(188,178,223,.09);
  --status-pending: var(--pending-tint);
  --status-pending-line: rgba(226,194,122,.45);
  --status-pending-wash: rgba(226,194,122,.09);
  --status-danger: var(--risk-tint);
  --status-danger-line: rgba(240,144,122,.45);
  --status-danger-wash: rgba(240,144,122,.09);
  --status-success: var(--success-tint);
  --status-success-line: rgba(169,201,163,.42);
  --status-success-wash: rgba(169,201,163,.08);
  --status-neutral: var(--text-secondary);
  --chrome-surface: var(--ink-bg);
  --chrome-surface-raised: var(--ink-panel);
  --chrome-text: var(--ink-fg);
  --brand-canvas: var(--surface-canvas);
  --brand-surface: var(--surface-base);
  --brand-ink: var(--text-primary);
  --brand-muted: var(--text-secondary);
  --brand-accent: var(--action-primary);
  --brand-focus: var(--focus-color);
}
:root {
  --space-1: 8px;
  --space-2: 16px;
  --space-3: 24px;
  --space-4: 32px;
  --space-5: 40px;
  --space-6: 48px;
  --space-8: 64px;
  --space-10: 80px;
  --space-12: 96px;
  --page-pad: 18px;
  --chrome-pad: 18px;
  --r-sm: 3px;
  --r-md: 6px;
  --r-lg: 10px;
  --shape-pill: 999px;
  --rail-w: 248px;
  --rail-w-compact: 72px;
  --layout-measure: 100%;
  --layout-operational: 1180px;
  --target-min: 24px;
  --target-compact: 32px;
  --target-icon: 40px;
  --target-primary: 44px;
  --focus-ring-width: 2px;
  --focus-ring-offset: 2px;
  --blur-chrome: 14px;
  --layer-base: 0;
  --layer-raised: 1;
  --layer-sticky: 20;
  --layer-chrome: 21;
  --layer-overlay: 80;
  --layer-menu: 90;
  --layer-skip: 100;
  --layer-curtain: 1500;
  --leading-none: 1;
  --leading-display-xl: 1.08;
  --leading-display-lg: 1.1;
  --leading-display: 1.15;
```

```
  --leading-display-sm: 1.2;
  --leading-display-xs: 1.25;
  --leading-tight: 1.3;
  --leading-tight-plus: 1.35;
  --leading-normal: 1.45;
  --leading-relaxed: 1.5;
  --leading-relaxed-plus: 1.55;
  --leading-loose: 1.6;
  --tracking-heading: -0.015em;
  --tracking-kicker: 0.08em;
  --tracking-caps: 0.1em;
  --tracking-wordmark: 0.22em;
  --font-display: 'Cinzel','Barlow Condensed',Georgia,serif;
  --font-heading: 'Barlow Condensed',system-ui,sans-serif;
  --font-body: 'Barlow',system-ui,sans-serif;
  --font-mono: ui-monospace,'SF Mono',Menlo,Consolas,monospace;
}
.role-editorial { font-family: var(--font-display); font-size: 42px; line-height: 1.1; font-weight: 500; }
.role-display { font-family: var(--font-heading); font-size: 26px; line-height: 1.15; font-weight: 600; letter-spacing:
-0.015em; }
.role-page { font-family: var(--font-heading); font-size: 24px; line-height: 1.2; font-weight: 600; letter-spacing: -0.015em; }
.role-section { font-family: var(--font-heading); font-size: 20px; line-height: 1.2; font-weight: 600; letter-spacing:
-0.015em; }
.role-card-title { font-family: var(--font-heading); font-size: 18px; line-height: 1.25; font-weight: 600; letter-spacing:
-0.015em; }
.role-body { font-family: var(--font-body); font-size: 16px; line-height: 1.55; font-weight: 400; }
.role-control { font-family: var(--font-heading); font-size: 14px; line-height: 1.2; font-weight: 600; letter-spacing:
-0.015em; }
.role-compact { font-family: var(--font-body); font-size: 12px; line-height: 1.45; font-weight: 400; }
.role-label { font-family: var(--font-body); font-size: 11px; line-height: 1.3; font-weight: 500; }
.role-caption { font-family: var(--font-body); font-size: 11px; line-height: 1.45; font-weight: 400; }
.role-kicker { font-family: var(--font-body); font-size: 10.5px; line-height: 1.3; font-weight: 500; letter-spacing: 0.08em; }
.role-numeric { font-family: var(--font-heading); font-size: 22px; line-height: 1.15; font-weight: 600; letter-spacing:
-0.015em; }
@font-face { font-family: "Barlow"; font-style: normal; font-weight: 400; font-display: swap; src: url("fonts/Barlow-400-
latin.woff2") format("woff2"); }
@font-face { font-family: "Barlow"; font-style: normal; font-weight: 500; font-display: swap; src: url("fonts/Barlow-500-
latin.woff2") format("woff2"); }
@font-face { font-family: "Barlow"; font-style: normal; font-weight: 700; font-display: swap; src: url("fonts/Barlow-700-
latin.woff2") format("woff2"); }
@font-face { font-family: "Barlow Condensed"; font-style: normal; font-weight: 400; font-display: swap; src:
url("fonts/BarlowCondensed-400-latin.woff2") format("woff2"); }
@font-face { font-family: "Barlow Condensed"; font-style: normal; font-weight: 600; font-display: swap; src:
url("fonts/BarlowCondensed-600-latin.woff2") format("woff2"); }
@font-face { font-family: "Cinzel"; font-style: normal; font-weight: 400 600; font-display: swap; src: url("fonts/Cinzel-
latin.woff2") format("woff2"); }
```

components/bundle.css · 388 lines

```
/* Semester — component and base styles. Token values live in tokens.json (compiled to tokens.css); this sheet keeps what
tokens.json cannot hold: computed scales, aliases, motion, media overrides, base and component rules. */

/* ── from tokens/colors.css ── */
/* Semester colour.
 * Primitives come from the app's measured grounds and accents (app/src/lib/look.ts):
 *   Ink (default dark) and Parchment (default light) ramps, Sterling/Brass metals, and the Industry blue ramp
 *   which this system names "Semester blue" for interactive focus and trust.
 * Source-status hues are desaturated stones drawn toward the brief's semantics (official navy, connected teal,
 * personal graphite, AI violet, pending amber, risk red, success green). Every state also carries a word + glyph.
 * Default scope = warm off-white content (Parchment). Chrome (rail, tab bar, console header) is always Ink.
 * [data-theme="dark"] flips content to Ink. */
:root{
  --chrome-metal:linear-gradient(180deg,#f6f8fb 0%,#d4d9e2 45%,#9ca3b2 100%);/* @kind color */
  }

@media (prefers-contrast:more){
  :root{--text-secondary:rgba(27,26,23,.9);--text-tertiary:rgba(27,26,23,.76);--border-default:rgba(27,26,23,.34);--border-
subtle:rgba(27,26,23,.2)}
  [data-theme="dark"]{--text-secondary:rgba(236,238,242,.9);--text-tertiary:rgba(236,238,242,.76);--border-
default:rgba(236,238,242,.34);--border-subtle:rgba(236,238,242,.2)}
}

/* ── from tokens/typography.css ── */
/* Semester type. Barlow for product reading, Barlow Condensed for headings/labels/controls,
 * Cinzel for editorial display moments only, system mono for identifiers and timestamps.
```

```
 * Sizes are the app's scale (app/src/styles/app.css) and respond to --text-scale. */
:root{
  --font-heading-weight:600;
  --font-editorial:var(--font-display);
  --font-product:var(--font-body);
  --type-3xs:calc(9px * var(--text-scale,1));
  --type-2xs:calc(10px * var(--text-scale,1));
  --type-2xs-plus:calc(10.5px * var(--text-scale,1));
  --type-xs:calc(11px * var(--text-scale,1));
  --type-xs-plus:calc(11.5px * var(--text-scale,1));
  --type-sm:calc(12px * var(--text-scale,1));
  --type-sm-plus:calc(12.5px * var(--text-scale,1));
  --type-control:calc(14px * var(--text-scale,1));
  --type-base:calc(16px * var(--text-scale,1));
  --type-base-plus:calc(16.5px * var(--text-scale,1));
  --type-md:calc(17px * var(--text-scale,1));
  --type-lg:calc(18px * var(--text-scale,1));
  --type-display-xs:calc(19px * var(--text-scale,1));
  --type-display-sm:calc(20px * var(--text-scale,1));
  --type-display:calc(22px * var(--text-scale,1));
  --type-display-lg:calc(24px * var(--text-scale,1));
  --type-xl:calc(26px * var(--text-scale,1));
  --type-2xl:calc(28px * var(--text-scale,1));
  --type-3xl:calc(32px * var(--text-scale,1));
  --type-4xl:calc(42px * var(--text-scale,1));
  --type-5xl:calc(56px * var(--text-scale,1));
  --type-role-editorial:var(--type-4xl);
  --type-role-display:var(--type-xl);
  --type-role-page:var(--type-display-lg);
  --type-role-section:var(--type-display-sm);
  --type-role-card-title:var(--type-lg);
  --type-role-body:var(--type-base);
  --type-role-compact:var(--type-sm);
  --type-role-label:var(--type-xs);
  --type-role-kicker:var(--type-2xs-plus);
  --type-role-numeric:var(--type-display);
  --type-role-control:var(--type-control);
  --type-role-caption:var(--type-xs);
  --layout-reading:66ch;/* @kind other */
}

/* ── from tokens/spacing.css ── */
/* Spacing. --sp-1…7 are the app's density-aware 2px steps for compact UI;
 * --space-* is the 8px macro grid for layout. --density: 1 comfortable, .86 snug, .74 tight. */
:root{
  --sp-1:calc(2px * var(--density,1));/* @kind other */--sp-2:calc(4px * var(--density,1));--sp-3:calc(6px * var(--
density,1));--sp-4:calc(8px * var(--density,1));
  --sp-5:calc(10px * var(--density,1));--sp-6:calc(12px * var(--density,1));--sp-7:calc(16px * var(--density,1));
  --layout-gutter:var(--page-pad);
  --control-height-compact:var(--target-compact);--control-height-standard:var(--target-primary);
  }
@media (min-width:760px){:root{--page-pad:20px;--chrome-pad:20px}}
@media (min-width:1100px){:root{--page-pad:30px;--chrome-pad:30px}}

/* ── from tokens/effects.css ── */
/* Shape, elevation, motion, layers — values from app/src/styles/app.css, tokens.css, industry.css. */
:root{
  --shape-control:var(--r-sm);--shape-card:var(--r-md);--shape-surface:var(--r-lg);--shape-sheet:var(--r-lg);/* Lifts. Dark
grounds: --lift-*; light grounds: ink-tinted --shadow-*. Border first, shadow second. */
  --elevation-none:none;--elevation-raised:var(--shadow-sm);--elevation-floating:var(--shadow-md);--elevation-modal:var(--
shadow-lg);
  --ease:cubic-bezier(.22,1,.36,1);/* @kind other */
  --ease-standard:var(--ease);/* @kind other */
  --ease-emphasized:cubic-bezier(.2,.8,.2,1);/* @kind other */
  --duration-instant:0ms;/* @kind other */--duration-fast:130ms;/* @kind other */--duration-standard:180ms;/* @kind other */--
duration-slow:240ms;/* @kind other */--duration-sheet:280ms;/* @kind other */
  --fast:130ms var(--ease);/* @kind other */
  --motion-save:var(--duration-standard) var(--ease-standard);/* @kind other */
  --motion-insert:var(--duration-standard) var(--ease-standard);/* @kind other */
  --motion-panel:var(--duration-slow) var(--ease-emphasized);/* @kind other */
  --motion-sheet:var(--duration-sheet) var(--ease-emphasized);/* @kind other */
  --motion-progress:var(--duration-standard) linear;/* @kind other */
}
[data-theme="dark"]{--elevation-raised:var(--lift-1);--elevation-floating:var(--lift-2);--elevation-modal:var(--lift-3)}
@media (prefers-reduced-motion:reduce){:root{--fast:0ms linear;/* @kind other */--motion-save:0ms linear;/* @kind other */--
motion-insert:0ms linear;/* @kind other */--motion-panel:0ms linear;/* @kind other */--motion-sheet:0ms linear;/* @kind other
*/--motion-progress:0ms linear;/* @kind other */}}
```

```
:root[data-calm='still'],:root[data-calm='calm']{--fast:0ms linear;/* @kind other */--motion-save:0ms linear;/* @kind other */--motion-insert:0ms linear;/* @kind other */--motion-panel:0ms linear;/* @kind other */--motion-sheet:0ms linear;/* @kind other */--motion-progress:0ms linear;/* @kind other */}

/* ── from tokens/base.css ── */
/* Base element styles — what a Semester page looks like before any component. */
*,*::before,*::after{box-sizing:border-box}
html{-webkit-text-size-adjust:100%}
body{margin:0;background:var(--surface-canvas);color:var(--text-primary);font-family:var(--font-body);font-size:var(--type-base);line-height:var(--leading-relaxed-plus);-webkit-font-smoothing:antialiased;text-rendering:optimizeLegibility}
h1,h2,h3,h4,h5,h6{font-family:var(--font-heading);font-weight:var(--font-heading-weight);line-height:var(--leading-display-sm);letter-spacing:var(--tracking-heading);margin:0;text-wrap:balance}
h1{font-size:var(--type-role-page)}h2{font-size:var(--type-role-section)}h3{font-size:var(--type-lg)}h4{font-size:var(--type-md)}
p{margin:0;text-wrap:pretty}
a{color:var(--text-link);text-underline-offset:3px;text-decoration-thickness:1px}
a:hover{color:var(--text-primary)}
:focus{outline:none}
:focus-visible{outline:var(--focus-ring-width) solid var(--focus-color);outline-offset:var(--focus-ring-offset)}
::selection{background:rgba(116,157,196,.3)}
button{font:inherit;color:inherit}
.sr-only{position:absolute!important;width:1px;height:1px;padding:0;margin:-1px;overflow:hidden;clip:rect(0,0,0,0);white-space:nowrap;border:0}
.nums{font-variant-numeric:tabular-nums}
.mono{font-family:var(--font-mono);font-size:.92em}
.kicker{font-family:var(--font-body);font-size:var(--type-role-kicker);font-weight:500;letter-spacing:var(--tracking-kicker);text-transform:uppercase;color:var(--text-secondary)}
.editorial{font-family:var(--font-display);font-weight:500;letter-spacing:.01em;line-height:var(--leading-display-lg)}
.chrome-text{background:var(--chrome-metal);-webkit-background-clip:text;background-clip:text;color:transparent}
.link-quiet{color:var(--text-secondary);text-decoration:underline dotted;text-underline-offset:3px;font-size:var(--type-sm);background:none;border:0;padding:0;cursor:pointer}
.link-quiet:hover{color:var(--text-primary)}
.skip-link{position:absolute;left:var(--sp-4);top:-60px;z-index:var(--layer-skip);padding:var(--sp-4) var(--sp-6);background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--r-md)}
.skip-link:focus{top:var(--sp-4)}

/* ── from tokens/components.css ── */
/* Semester component classes. Drawn from semantic tokens only; used by the React components in components/.
 * Ported from app/src/styles/unity.css + industry.css, re-pointed at this system's tokens. */

/* ── Buttons ── */
.btn{display:inline-flex;align-items:center;justify-content:center;gap:calc(6px * var(--density,1));min-height:var(--target-primary);padding:var(--sp-3) calc(14px * var(--density,1));font-family:var(--font-heading);font-weight:var(--font-heading-weight);font-size:var(--type-control);letter-spacing:.02em;line-height:1.2;color:var(--text-primary);background:transparent;border:1px solid var(--border-default);border-radius:var(--shape-control);cursor:pointer;text-decoration:none;white-space:nowrap;transition:background var(--fast),border-color var(--fast),color var(--fast)}
.btn svg{display:block;flex:none}
.btn:hover{background:var(--action-secondary)}
.btn:active{background:var(--action-secondary-hover)}
.btn:disabled,.btn[aria-disabled='true']{opacity:.45;cursor:not-allowed}
.btn-primary{background:var(--action-primary);border-color:var(--action-primary);color:var(--text-on-action)}
.btn-primary:hover{background:var(--action-primary-hover);border-color:var(--action-primary-hover)}
.btn-primary:active{background:var(--action-primary-press)}
.btn-ghost{border-color:transparent;color:var(--text-link);padding-inline:var(--sp-4)}
.btn-danger{color:var(--status-danger);border-color:var(--status-danger-line)}
.btn-danger:hover{background:var(--status-danger-wash)}
.btn-sm{min-height:var(--target-compact);padding:var(--sp-2) var(--sp-5);font-size:var(--type-sm-plus)}
.btn-block{width:100%}
.btn-icon{width:var(--target-icon);height:var(--target-icon);min-height:0;padding:0;border-color:transparent;color:var(--text-secondary)}
.btn-icon:hover{color:var(--text-primary)}
.btn-icon.btn-sm{width:var(--target-compact);height:var(--target-compact)}

/* ── Forms ── */
.field{display:grid;gap:calc(5px * var(--density,1))}
.field-label{font-size:var(--type-sm);font-weight:500;color:var(--text-primary)}
.field-hint{font-size:var(--type-xs);color:var(--text-secondary);line-height:var(--leading-normal)}
.field-error{font-size:var(--type-xs);color:var(--status-danger);display:flex;gap:var(--sp-2)}
.input{width:100%;min-height:var(--target-primary);padding:calc(6px * var(--density,1)) calc(10px * var(--density,1));font:inherit;font-size:var(--type-control);color:var(--text-primary);caret-color:var(--action-primary);background:var(--surface-raised);border:1px solid var(--border-default);border-radius:var(--shape-control);transition:border-color var(--fast)}
.input::placeholder{color:var(--text-tertiary)}
.input:hover{border-color:var(--border-strong)}
.input:focus-visible{border-color:var(--focus-color);outline-offset:0}
.input[aria-invalid='true']{border-color:var(--status-danger-line)}
.input:disabled{background:var(--surface-sunken);color:var(--text-disabled);cursor:not-allowed}
```

```
textarea.input{min-height:90px;resize:vertical;line-height:var(--leading-normal)}
select.input{appearance:none;padding-inline-end:34px;background-image:linear-gradient(45deg,transparent 50%,var(--text-
secondary) 50%),linear-gradient(135deg,var(--text-secondary) 50%,transparent 50%);background-position:calc(100% - 16px)
50%,calc(100% - 12px) 50%;background-size:4px 4px;background-repeat:no-repeat}
.seg{display:inline-flex;overflow:hidden;border:1px solid var(--border-default);border-radius:var(--shape-
control);background:var(--surface-raised)}
.seg-opt{display:inline-flex;align-items:center;gap:var(--sp-3);min-height:var(--target-compact);padding:calc(7px * var(--
density,1)) calc(12px * var(--density,1));font-size:var(--type-sm-plus);cursor:pointer;background:none;border:0;color:var(--
text-secondary)}
.seg-opt+.seg-opt{border-left:1px solid var(--border-default)}
.seg-opt[aria-pressed='true'],.seg-opt[aria-checked='true']{background:var(--text-primary);color:var(--surface-canvas)}
.seg-opt:not([aria-pressed='true']):not([aria-checked='true']):hover{background:var(--action-secondary);color:var(--text-
primary)}
.choice{display:flex;align-items:flex-start;gap:var(--sp-5);min-height:var(--target-primary);padding-block:var(--sp-3);font-
size:var(--type-base);color:var(--text-primary);cursor:pointer}
.choice input{width:20px;height:20px;margin:2px 0 0;flex:none;accent-color:var(--action-primary)}
.choice-about{display:block;font-size:var(--type-sm);line-height:var(--leading-normal);color:var(--text-secondary)}
.switch{display:inline-flex;align-items:center;gap:var(--sp-5);min-height:var(--target-primary);cursor:pointer;font-size:var(--
type-base)}
.switch-track{position:relative;width:36px;height:20px;flex:none;border-radius:var(--shape-pill);background:var(--surface-
sunken);border:1px solid var(--border-strong);transition:background var(--fast)}
.switch-track::after{content:'';position:absolute;top:2px;left:2px;width:14px;height:14px;border-radius:50%;background:var(--
text-secondary);transition:transform var(--fast),background var(--fast)}
.switch[aria-checked='true'] .switch-track{background:var(--action-primary);border-color:var(--action-primary)}
.switch[aria-checked='true'] .switch-track::after{transform:translateX(16px);background:var(--text-on-action)}
.switch-state{font-size:var(--type-xs);color:var(--text-secondary);min-width:2.5em}

/* ── Status + provenance ── */
.status-chip{display:inline-flex;align-items:baseline;gap:var(--sp-2);padding:var(--sp-1) var(--sp-4);font-size:var(--type-
xs);line-height:var(--leading-tight);color:var(--status-neutral);border:1px solid var(--border-default);border-radius:var(--
shape-control);white-space:nowrap}
.status-glyph{font-variant-numeric:tabular-nums}
.status-chip[data-tone='pending'],.status-chip[data-tone='attention']{color:var(--status-pending);border-color:var(--status-
pending-line);background:var(--status-pending-wash)}
.status-chip[data-tone='danger']{color:var(--status-danger);border-color:var(--status-danger-line);background:var(--status-
danger-wash)}
.status-chip[data-tone='success']{color:var(--status-success);border-color:var(--status-success-line)}
.status-chip[data-tone='official']{color:var(--source-official);border-color:var(--source-official-line);background:var(--
source-official-wash)}
.status-chip[data-tone='connected']{color:var(--source-connected);border-color:var(--source-connected-line)}
.status-chip[data-tone='personal']{color:var(--source-personal);border-color:var(--source-personal-line)}
.status-chip[data-tone='ai']{color:var(--source-ai);border-color:var(--source-ai-line);background:var(--source-ai-wash)}
.status-chip[data-fill='solid']{border-width:1px;border-style:solid}
.status-chip[data-fill='dashed']{border-style:dashed}
.prov{display:inline-flex;flex-wrap:wrap;align-items:baseline;gap:var(--sp-2)}
.prov-age{font-size:var(--type-xs);color:var(--text-secondary)}
.source-badge{display:inline-flex;flex-wrap:wrap;align-items:baseline;gap:var(--sp-3);font-size:var(--type-xs);color:var(--
text-secondary)}
.source-report{background:none;border:0;padding:0;font-size:var(--type-xs);color:var(--text-link);text-
decoration:underline;text-underline-offset:3px;cursor:pointer}
.save-state{display:inline-flex;align-items:baseline;gap:var(--sp-2);font-size:var(--type-xs);color:var(--text-
secondary);transition:color var(--motion-save)}
.save-state[data-tone='success']{color:var(--text-primary)}
.save-state[data-tone='pending']{color:var(--status-pending)}
.save-state[data-tone='danger']{color:var(--status-danger)}

/* ── AI disclosure ── */
.ai-frame{display:grid;gap:var(--sp-6);padding:var(--sp-7) var(--sp-7) var(--sp-6);border:1px solid var(--source-ai-
line);border-radius:var(--shape-card);background:var(--surface-raised)}
.ai-head{display:flex;flex-wrap:wrap;align-items:center;justify-content:space-between;gap:var(--sp-4)}
.ai-body{font-size:var(--type-base);line-height:var(--leading-relaxed-plus);max-width:var(--layout-reading)}
.ai-body p+p{margin-top:var(--sp-5)}
.ai-cite{display:inline-flex;align-items:center;justify-content:center;min-width:18px;height:18px;padding:0 4px;margin-
inline:2px;font-size:var(--type-2xs);font-variant-numeric:tabular-nums;vertical-align:2px;border:1px solid var(--border-
default);border-radius:var(--r-sm);color:var(--text-secondary)}
.ai-sources{display:grid;gap:var(--sp-2);margin:0;padding:var(--sp-5) 0 0;list-style:none;border-top:1px solid var(--border-
subtle)}
.ai-sources li{display:flex;gap:var(--sp-4);align-items:baseline;font-size:var(--type-sm);color:var(--text-secondary)}
.ai-meta{display:flex;flex-wrap:wrap;gap:var(--sp-3) var(--sp-7);font-size:var(--type-xs);color:var(--text-secondary)}
.ai-meta b{font-weight:500;color:var(--text-primary)}

/* ── Surfaces ── */
.object-card{display:flex;flex-direction:column;gap:var(--sp-3);padding:var(--sp-7);background:var(--surface-base);border:1px
solid var(--border-default);border-radius:var(--shape-card)}
.object-card-eyebrow{font-size:var(--type-2xs-plus);letter-spacing:var(--tracking-kicker);text-transform:uppercase;color:var(--
text-secondary);display:flex;gap:var(--sp-3);align-items:baseline}
.object-card-title{margin:0;font-family:var(--font-body);font-size:var(--type-lg);line-height:var(--leading-display-sm);font-
```

```
weight:600;letter-spacing:0;color:var(--text-primary)}
.object-card-why{font-size:var(--type-base);line-height:var(--leading-normal);color:var(--text-secondary)}
.object-card-meta{font-size:var(--type-sm);color:var(--text-secondary);font-variant-numeric:tabular-nums}
.object-card-actions,.object-card-foot,.state-actions,.dialog-actions{display:flex;flex-wrap:wrap;align-items:center;gap:var(--
sp-4) var(--sp-6)}
.object-card-actions{margin-top:var(--sp-3)}
.object-card[data-tone='pending']{border-color:var(--status-pending-line)}
.object-card[data-tone='danger']{border-color:var(--status-danger-line)}
.context-bar{display:flex;flex-direction:column;gap:var(--sp-3);padding:var(--sp-6) var(--sp-7);background:var(--surface-
base);border:1px solid var(--border-subtle);border-radius:var(--shape-card)}
.context-bar-title{margin:0;font-family:var(--font-body);font-weight:600;letter-spacing:0;font-size:var(--type-display-
xs);line-height:var(--leading-display-xs)}
.context-bar-states{display:flex;flex-wrap:wrap;gap:var(--sp-3)}
.next-steps{display:grid;gap:var(--sp-3);margin:0;padding:0;list-style:none}
.next-step{display:flex;align-items:center;gap:var(--sp-6);width:100%;min-height:var(--target-primary);padding:var(--sp-5)
var(--sp-6);text-align:left;background:var(--surface-base);color:var(--text-primary);border:1px solid var(--border-
default);border-radius:var(--shape-card);font:inherit;cursor:pointer;transition:border-color var(--fast),background var(--
fast)}
.next-step:hover{border-color:var(--border-strong);background:var(--surface-raised)}
.next-step-text{display:grid;gap:2px;flex:1;min-width:0}
.next-step-label{font-size:var(--type-md)}
.next-step-why{font-size:var(--type-sm);color:var(--text-secondary)}
.action-preview{display:grid;gap:var(--sp-5)}
.action-preview-facts{display:grid;grid-template-columns:minmax(0,1fr);gap:var(--sp-1);margin:0}
.action-preview-facts dt{font-size:var(--type-xs);color:var(--text-secondary);margin-top:var(--sp-4)}
.action-preview-facts dd{margin:0;font-size:var(--type-base);line-height:var(--leading-normal)}
.action-preview-caveat{font-size:var(--type-sm);color:var(--text-secondary)}
.dialog-scrim{position:fixed;inset:0;z-index:var(--layer-overlay);display:grid;place-items:center;padding:var(--space-
2);background:var(--scrim)}
.dialog{width:min(480px,100%);display:flex;flex-direction:column;gap:var(--sp-7);padding:calc(24px * var(--
density,1));background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--shape-sheet);box-
shadow:var(--elevation-modal);animation:dialog-in var(--motion-sheet)}
.dialog-title{font-family:var(--font-heading);font-size:var(--type-display-sm)}
.dialog-actions{justify-content:flex-end;padding-top:var(--sp-4);border-top:1px solid var(--border-subtle)}
@keyframes dialog-in{from{opacity:0;transform:translateY(8px)}}

/* ── States ── */
.state{display:flex;flex-direction:column;gap:var(--sp-3);padding:var(--sp-6) var(--sp-7);border:1px solid var(--border-
default);border-radius:var(--shape-card);font-size:var(--type-base);line-height:var(--leading-normal);color:var(--text-
primary)}
.state-title{font-size:var(--type-md);font-weight:600;display:flex;gap:var(--sp-3);align-items:baseline}
.state-body,.state-note,.state-ref{margin:0;color:var(--text-secondary)}
.state-note,.state-ref{font-size:var(--type-sm)}
.state[data-kind='error']{border-color:var(--status-danger-line);background:var(--status-danger-wash)}
.state[data-kind='error'] .status-glyph{color:var(--status-danger)}
.state[data-kind='success'] .status-glyph{color:var(--status-success)}
.state[data-kind='offline']{border-color:var(--status-pending-line);background:var(--status-pending-wash);flex-
direction:row;gap:var(--sp-4);font-size:var(--type-sm)}
.state[data-kind='permission']{border-style:dashed}
.state[data-kind='empty']{border-style:dashed;align-items:flex-start;padding:var(--sp-7) var(--sp-7)}
.state-progress-head{display:flex;justify-content:space-between;gap:var(--sp-4);font-size:var(--type-sm)}
.state-progress-bar{width:100%;height:6px;appearance:none;border:0;border-radius:var(--shape-
pill);overflow:hidden;background:var(--surface-sunken)}
.state-progress-bar::-webkit-progress-bar{background:var(--surface-sunken)}
.state-progress-bar::-webkit-progress-value{background:var(--action-primary);transition:width var(--motion-progress)}
.state-progress-bar::-moz-progress-bar{background:var(--action-primary)}
.state-steps{list-style:none;margin:0;padding:0;display:flex;flex-direction:column;gap:var(--sp-3);font-size:var(--type-
base);color:var(--text-secondary)}
.state-steps li{display:flex;gap:var(--sp-4)}
.state-steps [data-state='working'],.state-steps [data-state='done']{color:var(--text-primary)}
.state-steps [data-state='failed']{color:var(--status-danger)}
.state-loading{display:flex;flex-direction:column;gap:var(--sp-6)}
.state-loading-bar{border-radius:var(--shape-card);background:var(--surface-sunken)}
.notice{margin:0;padding:var(--sp-3) var(--sp-4);border:1px solid var(--border-default);border-radius:var(--r-
md);background:var(--surface-base);font-size:var(--type-sm);line-height:var(--leading-relaxed)}
.notice-head{font-family:var(--font-heading);letter-spacing:.06em;text-transform:uppercase;font-size:var(--type-xs);margin-
right:var(--sp-3)}

/* ── Navigation ── */
.rail{display:flex;flex-direction:column;gap:2px;width:var(--rail-w);padding:var(--sp-7) var(--sp-5);background:var(--chrome-
surface);color:var(--chrome-text);border-right:1px solid var(--chrome-line);height:100%;overflow:auto}
.rail-brand{display:flex;align-items:center;gap:var(--sp-5);padding:var(--sp-3) var(--sp-5) var(--sp-7)}
.rail-label{padding:var(--sp-7) var(--sp-5) var(--sp-3);font-size:var(--type-2xs-plus);letter-spacing:var(--tracking-
kicker);text-transform:uppercase;color:var(--chrome-text-secondary)}
.rail-row{display:flex;align-items:center;gap:var(--sp-6);width:100%;min-height:40px;padding:var(--sp-3) var(--sp-
5);background:none;border:0;border-radius:var(--r-md);color:var(--chrome-text-secondary);font-family:var(--font-body);font-
size:var(--type-control);text-align:left;cursor:pointer;transition:background var(--fast),color var(--fast)}
```

```
.rail-row:hover{color:var(--chrome-text);background:rgba(236,238,242,.05)}
.rail-row[aria-current='page']{color:var(--chrome-text);background:var(--chrome-active)}
.rail-row[aria-current='page'] svg{color:var(--blue-300)}
.rail-count{margin-left:auto;font-size:var(--type-xs);font-variant-numeric:tabular-nums;color:var(--chrome-text-secondary)}
.rail-foot{margin-top:auto;padding:var(--sp-7) var(--sp-5) 0;font-size:var(--type-xs);color:var(--chrome-text-secondary);line-
height:var(--leading-normal);border-top:1px solid var(--chrome-line)}
.tabbar{display:flex;justify-content:space-around;align-items:stretch;padding:var(--sp-2) var(--sp-3) max(var(--sp-3),env(safe-
area-inset-bottom));background:rgba(9,10,14,.94);border-top:1px solid var(--chrome-line);color:var(--chrome-text-secondary)}
@supports (backdrop-filter:blur(1px)){.tabbar{background:rgba(9,10,14,.86);backdrop-filter:blur(var(--blur-chrome));-webkit-
backdrop-filter:blur(var(--blur-chrome))}}
.tab{flex:1;display:flex;flex-direction:column;align-items:center;justify-content:center;gap:3px;min-height:var(--target-
primary);padding:var(--sp-2) 0;background:none;border:0;color:inherit;font-size:var(--type-2xs-plus);letter-
spacing:.02em;cursor:pointer}
.tab[aria-current='page']{color:var(--chrome-text)}
.tab[aria-current='page'] svg{color:var(--blue-300)}
.syscontext{display:flex;flex-wrap:wrap;align-items:center;justify-content:space-between;gap:var(--sp-4);min-height:var(--
target-primary);padding:var(--sp-2) var(--chrome-pad);border-bottom:1px solid var(--border-subtle);background:var(--surface-
base);color:var(--text-secondary);font-size:var(--type-xs);line-height:var(--leading-tight)}
.syscontext-group{display:flex;align-items:center;gap:var(--sp-5);min-width:0}
.syscontext-term{color:var(--text-primary);font-variant-numeric:tabular-nums}
.syscontext-search{display:flex;align-items:center;gap:var(--sp-4);min-height:32px;min-width:240px;padding:0 var(--sp-
5);border:1px solid var(--border-default);border-radius:var(--r-md);background:var(--surface-raised);color:var(--text-
tertiary);font-size:var(--type-sm);cursor:pointer}
.kbd{display:inline-flex;align-items:center;min-width:18px;height:18px;padding:0 5px;font-family:var(--font-mono);font-
size:var(--type-2xs);color:var(--text-secondary);border:1px solid var(--border-default);border-bottom-width:2px;border-
radius:var(--r-sm);background:var(--surface-raised)}
.palette{width:min(620px,100%);background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--
shape-sheet);box-shadow:var(--elevation-modal);overflow:hidden}
.palette-input{display:flex;align-items:center;gap:var(--sp-5);padding:0 var(--sp-7);border-bottom:1px solid var(--border-
subtle)}
.palette-input input{flex:1;min-height:52px;border:0;background:none;font:inherit;font-size:var(--type-md);color:var(--text-
primary);outline:none}
.palette-group{padding:var(--sp-4) var(--sp-4) var(--sp-3)}
.palette-group-label{padding:var(--sp-3) var(--sp-5);font-size:var(--type-2xs-plus);letter-spacing:var(--tracking-kicker);text-
transform:uppercase;color:var(--text-secondary)}
.palette-option{display:flex;align-items:center;gap:var(--sp-6);width:100%;min-height:var(--target-primary);padding:var(--sp-3)
var(--sp-5);border:0;border-inline-start:3px solid transparent;background:none;text-align:left;font:inherit;font-size:var(--
type-control);color:var(--text-primary);cursor:pointer;border-radius:0 var(--r-sm) var(--r-sm) 0}
.palette-option[aria-selected='true']{border-inline-start-color:var(--focus-color);background:var(--surface-quiet)}
.palette-option-detail{margin-left:auto;font-size:var(--type-xs);color:var(--text-secondary)}
.palette-foot{display:flex;gap:var(--sp-7);padding:var(--sp-4) var(--sp-7);border-top:1px solid var(--border-subtle);font-
size:var(--type-xs);color:var(--text-secondary)}

/* ── Data ── */
.table-wrap{overflow-x:auto;border:1px solid var(--border-default);border-radius:var(--shape-card);background:var(--surface-
base)}
.table{width:100%;border-collapse:collapse;font-size:var(--type-control)}
.table th{text-align:left;font-weight:500;font-size:var(--type-xs);letter-spacing:var(--tracking-kicker);text-
transform:uppercase;color:var(--text-secondary);padding:var(--sp-5) var(--sp-6);border-bottom:1px solid var(--border-
default);white-space:nowrap;background:var(--surface-quiet)}
.table td{padding:var(--sp-5) var(--sp-6);border-bottom:1px solid var(--border-subtle);vertical-align:baseline}
.table tbody tr:last-child td{border-bottom:0}
.table tbody tr:hover{background:var(--action-secondary)}
.table td.num,.table th.num{text-align:right;font-variant-numeric:tabular-nums}
.fields{display:grid;grid-template-columns:minmax(120px,max-content) minmax(0,1fr);gap:var(--sp-4) var(--sp-7);margin:0;font-
size:var(--type-sm)}
.fields dt{color:var(--text-secondary)}
.fields dd{margin:0;color:var(--text-primary)}
.trail{list-style:none;margin:0;padding:0;display:grid}
.trail-item{display:grid;grid-template-columns:14px minmax(0,1fr);gap:var(--sp-6);padding-bottom:var(--sp-7);position:relative}
.trail-item::before{content:'';position:absolute;left:6px;top:14px;bottom:0;width:1px;background:var(--border-default)}
.trail-item:last-child::before{display:none}
.trail-dot{width:13px;height:13px;margin-top:4px;border-radius:50%;border:1.5px solid var(--border-strong);background:var(--
surface-canvas)}
.trail-item[data-state='done'] .trail-dot{background:var(--text-primary);border-color:var(--text-primary)}
.trail-item[data-state='current'] .trail-dot{border-color:var(--focus-color);box-shadow:0 0 0 3px var(--action-secondary-
hover)}
.trail-item[data-state='blocked'] .trail-dot{border-color:var(--status-danger)}
.trail-title{font-size:var(--type-control);font-weight:500}
.trail-meta{font-size:var(--type-xs);color:var(--text-secondary);font-variant-numeric:tabular-nums}

/* ── Expanded set ── */
.combo{position:relative;display:grid;gap:var(--sp-2)}
.combo-list{position:absolute;inset-inline:0;top:100%;z-index:var(--layer-menu);margin:var(--sp-2) 0 0;padding:var(--sp-
2);list-style:none;max-height:260px;overflow-y:auto;background:var(--surface-overlay);border:1px solid var(--border-
default);border-radius:var(--shape-surface);box-shadow:var(--elevation-floating)}
.combo-option{display:grid;gap:var(--sp-1);min-height:var(--target-primary);padding:var(--sp-3) var(--sp-4);border-inline-
```

```
start:3px solid transparent;cursor:pointer;font-size:var(--type-control)}
.combo-option[aria-selected='true']{border-inline-start-color:var(--focus-color);background:var(--surface-quiet);font-
weight:600}
.combo-detail{font-size:var(--type-xs);color:var(--text-secondary);font-weight:400}
.visibility{display:flex;flex-direction:column;gap:var(--sp-3)}
.visibility-said{margin:0;font-size:var(--type-sm);line-height:var(--leading-normal);color:var(--text-
primary);display:flex;flex-wrap:wrap;gap:var(--sp-3);align-items:baseline}
.visibility fieldset{border:0;margin:0;padding:0;display:grid;gap:var(--sp-2)}
.sheet-scrim{position:fixed;inset:0;z-index:var(--layer-overlay);display:flex;align-items:flex-end;justify-
content:center;background:var(--scrim)}
.sheet{width:100%;max-width:620px;max-height:88%;overflow:auto;padding:var(--sp-7) calc(20px * var(--density,1)) calc(24px *
var(--density,1));background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--shape-sheet)
var(--shape-sheet) 0 0;box-shadow:var(--elevation-modal);animation:dialog-in var(--motion-sheet)}
.sheet.is-inline{max-height:none;overflow:visible;border-radius:var(--shape-sheet);box-shadow:var(--elevation-
floating);animation:none}
.sheet-grip{width:36px;height:4px;border-radius:2px;background:var(--border-strong);margin:0 auto var(--sp-6)}
.sheet-head{display:flex;align-items:center;justify-content:space-between;gap:var(--sp-4);margin-bottom:var(--sp-4)}
.source-list{display:grid;gap:var(--sp-1);margin:0}
.source-list dt{margin-top:var(--sp-6);font-size:var(--type-2xs-plus);letter-spacing:var(--tracking-kicker);text-
transform:uppercase;color:var(--text-secondary)}
.source-list dt:first-child{margin-top:0}
.source-list dd{margin:0;font-size:var(--type-base);line-height:var(--leading-normal)}
.toast{display:inline-flex;align-items:center;gap:var(--sp-6);min-height:var(--target-primary);padding:var(--sp-3) var(--sp-4)
var(--sp-3) var(--sp-7);background:var(--ink-raise);color:var(--ink-fg);border-radius:var(--r-md);box-shadow:var(--lift-
2);font-size:var(--type-sm)}
.toast .btn{color:var(--blue-300);border-color:transparent;min-height:var(--target-compact)}
.toast .btn:hover{background:rgba(148,188,227,.12)}
.focus-bar{display:inline-flex;flex-wrap:wrap;align-items:center;gap:var(--sp-5);padding:var(--sp-3) var(--sp-4) var(--sp-3)
var(--sp-7);background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--shape-pill);box-
shadow:var(--elevation-floating);font-size:var(--type-sm)}
.focus-timer{font-family:var(--font-heading);font-weight:600;font-size:var(--type-lg);font-variant-numeric:tabular-nums}
.screen-guide{padding-top:var(--sp-6);border-top:1px solid var(--border-subtle)}
.screen-guide summary{cursor:pointer;font-size:var(--type-sm);color:var(--text-secondary);min-height:var(--target-min);list-
style:none;display:flex;gap:var(--sp-3);align-items:center}
.screen-guide summary::-webkit-details-marker{display:none}
.screen-guide[open] summary{color:var(--text-primary)}
.screen-guide dl{margin:var(--sp-5) 0 0;max-width:var(--layout-reading)}
.screen-guide dt{margin-top:var(--sp-5);font-weight:600;font-size:var(--type-control)}
.screen-guide dd{margin:var(--sp-1) 0 0;color:var(--text-secondary);font-size:var(--type-control);line-height:var(--leading-
relaxed)}
.open-in{display:flex;flex-wrap:wrap;align-items:center;gap:var(--sp-3)}
.open-in-label{font-size:var(--type-xs);color:var(--text-secondary);margin-right:var(--sp-2)}
.pill-soft{display:inline-flex;align-items:center;gap:var(--sp-3);min-height:var(--target-compact);padding:0 var(--sp-
6);background:var(--action-secondary);border:1px solid transparent;border-radius:var(--shape-control);font-size:var(--type-sm-
plus);color:var(--text-primary);cursor:pointer}
.pill-soft:hover{background:var(--action-secondary-hover)}
.quick-actions{display:flex;flex-wrap:wrap;gap:var(--sp-4)}
.quick-action{display:inline-flex;align-items:center;gap:var(--sp-4);min-height:var(--target-primary);padding:0 var(--sp-7) 0
var(--sp-6);background:var(--surface-base);border:1px solid var(--border-default);border-radius:var(--shape-card);font-
size:var(--type-control);color:var(--text-primary);cursor:pointer}
.quick-action:hover{border-color:var(--border-strong);background:var(--surface-raised)}
.avatar{display:inline-flex;align-items:center;justify-content:center;flex:none;border-radius:var(--r-lg);background:var(--
action-secondary);border:1px solid var(--border-strong);color:var(--text-primary);font-family:var(--font-heading);letter-
spacing:.04em;line-height:1;overflow:hidden}
.metric{display:grid;gap:var(--sp-3);padding:var(--sp-6) var(--sp-7);border:1px solid var(--border-default);border-radius:var(-
-shape-card);background:var(--surface-base)}
.metric-value{font-family:var(--font-heading);font-weight:600;font-size:var(--type-2xl);line-height:1;font-variant-
numeric:tabular-nums}
.metric-delta{font-size:var(--type-xs);color:var(--text-secondary)}
.chart{display:grid;gap:var(--sp-5)}
.chart-title{font-size:var(--type-md);font-weight:600}
.chart-bars{display:grid;gap:var(--sp-4)}
.chart-row{display:grid;grid-template-columns:120px minmax(0,1fr) 64px;gap:var(--sp-5);align-items:center;font-size:var(--type-
sm)}
.chart-track{height:14px;background:var(--surface-sunken);border-radius:2px;overflow:hidden}
.chart-bar{height:100%;border-radius:2px}
.chart-bar[data-enc='solid']{background:var(--text-primary)}
.chart-bar[data-enc='outline']{background:transparent;box-shadow:inset 0 0 0 1.5px var(--text-primary)}
.chart-bar[data-enc='hatched']{background:repeating-linear-gradient(135deg,var(--text-primary) 0 2px,transparent 2px 6px);box-
shadow:inset 0 0 0 1px var(--text-primary)}
.chart-bar[data-enc='dotted']{background:radial-gradient(var(--status-danger) 1.2px,transparent 1.4px) 0 0/6px 6px;box-
shadow:inset 0 0 0 1px var(--status-danger)}
.chart-foot{font-size:var(--type-xs);color:var(--text-secondary);display:flex;flex-wrap:wrap;gap:var(--sp-3) var(--sp-7)}
.legend-swatch{display:inline-block;width:18px;height:10px;vertical-align:-1px;margin-right:var(--sp-3);border-radius:1px}

@media (forced-colors:active){.status-chip,.object-card,.context-bar,.next-step,.dialog,.palette,.state{border:1px solid
CanvasText}.status-chip[data-fill='solid']{border-width:3px}}
```

```
/* governance: PolicyBadge, ConsentBadge, HealthBadge, ApprovalBanner, ErrorSummary, SupportHandoff, PageHeader */
.gov-badge{display:inline-flex;align-items:baseline;gap:var(--sp-2);padding:var(--sp-1) var(--sp-4);font:inherit;font-
size:var(--type-xs);line-height:var(--leading-tight);color:var(--text-primary);background:transparent;border:1px solid var(--
border-default);border-radius:var(--shape-control);white-space:nowrap}
button.gov-badge{cursor:pointer}button.gov-badge:hover{background:var(--action-secondary)}
.gov-glyph{font-variant-numeric:tabular-nums;color:var(--text-secondary)}.gov-sub{color:var(--text-secondary);font-size:var(--
type-xs)}
.gov-link{all:unset;cursor:pointer;color:var(--action-primary);text-decoration:underline;margin-left:var(--sp-2)}
.gov-badge[data-kind='consent'][data-state='shared']{border-color:var(--status-pending-line)}
.gov-badge[data-kind='consent'][data-state='required']{color:var(--status-pending);border-color:var(--status-pending-
line);background:var(--status-pending-wash)}
.health-badge{display:inline-flex;align-items:baseline;gap:var(--sp-2);padding:var(--sp-1) var(--sp-4);font-size:var(--type-
xs);border:1px solid var(--border-default);border-radius:var(--shape-control)}
.health-badge b{font-weight:600}.health-badge[data-tone='success']{color:var(--status-success);border-color:var(--status-
success-line)}
.health-badge[data-tone='pending']{color:var(--status-pending);border-color:var(--status-pending-line);background:var(--status-
pending-wash)}
.health-badge[data-tone='danger']{color:var(--status-danger);border-color:var(--status-danger-line);background:var(--status-
danger-wash)}
.approval-banner{display:flex;gap:var(--sp-5);align-items:center;padding:var(--sp-5) var(--sp-6);border:1px solid var(--status-
pending-line);background:var(--status-pending-wash);border-radius:var(--shape-card);flex-wrap:wrap}
.approval-banner[data-state='approved']{border-color:var(--status-success-line);background:var(--status-success-
wash)}.approval-banner[data-state='rejected']{border-color:var(--status-danger-line);background:var(--status-danger-wash)}
.approval-body{display:grid;gap:2px;flex:1;min-width:200px}.approval-actions{display:flex;gap:var(--sp-3)}
.error-summary{padding:var(--sp-5) var(--sp-6);border:1px solid var(--status-danger-line);background:var(--status-danger-
wash);border-radius:var(--shape-card);color:var(--status-danger)}
.error-summary ul{margin:var(--sp-3) 0 0;padding-left:var(--sp-7)}.error-summary a{color:var(--status-danger)}
.support-handoff{display:grid;gap:var(--sp-3);padding:var(--sp-6);border:1px solid var(--border-default);border-radius:var(--
shape-card);background:var(--surface-base)}
.handoff-office{font-family:var(--font-heading);font-size:var(--type-lg)}.handoff-reason{margin:0;color:var(--text-secondary)}
.handoff-meta{display:grid;grid-template-columns:auto 1fr;gap:var(--sp-1) var(--sp-5);margin:0;font-size:var(--type-
sm)}.handoff-meta dt{color:var(--text-secondary)}.handoff-meta dd{margin:0}
.page-header{display:flex;justify-content:space-between;align-items:flex-end;gap:var(--sp-6);flex-wrap:wrap;padding-
bottom:var(--sp-7);border-bottom:1px solid var(--border-subtle)}
.page-header h1{margin:var(--sp-2) 0 0;text-wrap:pretty}.page-header-purpose{margin:var(--sp-2) 0 0;color:var(--text-
secondary);max-width:66ch}.page-header-source{margin-top:var(--sp-3)}.page-header-actions{display:flex;gap:var(--sp-3);flex-
wrap:wrap}
```

Component types

Every component's props, as TypeScript declarations.

components/index.d.ts · 415 lines

```
// Types for every component in window.SemesterDesignSystem_e9f3ba, gathered from components/<Name>/<Name>.d.ts.

// ---- AIResponse
import * as React from 'react';
/**
 * A governed assistant answer — labelled AI-assisted with data scope, numbered sources, confidence, request cost and policy.
 * @startingPoint section="AI" subtitle="A governed assistant answer — labelled AI-assisted with data scope, numbered sou"
viewport="700x400"
 */
export interface AIResponseProps { children: React.ReactNode; scope?: string[]; sources?: { title: string; kind?: string }[];
confidence?: string; cost?: string; policy?: string; actions?: React.ReactNode; model?: string; }
export declare function AIResponse(props: AIResponseProps): JSX.Element;

// ---- ActionPreview
import * as React from 'react';
/**
 * What is about to happen, said before it happens — What happens / Exactly what / What stays / Taking it back.
 */
export interface ActionPreviewProps { subject?: string; says: React.ReactNode; exactly?: React.ReactNode; doesNotChange?:
React.ReactNode; recovery: { kind: "undo" | "request" | "none"; how?: string }; subjectTo?: React.ReactNode; provenance?:
React.ReactNode; }
export declare function ActionPreview(props: ActionPreviewProps): JSX.Element;

// ---- ApprovalBanner
import * as React from 'react';
export interface ApprovalBannerProps { state?: "required" | "pending" | "approved" | "rejected"; what: string; approver?:
string; requestedBy?: string; onRequest?: () => void; onApprove?: () => void; onReject?: () => void; }
export declare function ApprovalBanner(props: ApprovalBannerProps): JSX.Element;

// ---- Avatar
import * as React from 'react';
/** Initials in a tinted square that follows the corner setting; profile glyph when no name. */
export interface AvatarProps { name?: string; size?: number; }
export declare function Avatar(props: AvatarProps): JSX.Element;

// ---- Button
import * as React from 'react';
/**
 * Action button — one primary per decision region; 44px tall by default, condensed-caps-free label in Barlow Condensed 600.
 * @startingPoint section="Actions" subtitle="Action button — one primary per decision region; 44px tall by default, condensed"
viewport="700x400"
 */
export interface ButtonProps { variant?: "primary" | "secondary" | "ghost" | "danger"; size?: "md" | "sm"; icon?: string;
iconAfter?: string; block?: boolean; loading?: boolean; disabled?: boolean; children?: React.ReactNode; onClick?: (e: any) =>
void; type?: "button" | "submit"; }
export declare function Button(props: ButtonProps): JSX.Element;

// ---- Choice
import * as React from 'react';
/**
 * Radio/checkbox row with a sentence explaining the consequence — used for visibility and sharing choices.
 */
export interface ChoiceProps { type?: "radio" | "checkbox"; label: string; about?: string; checked?: boolean; onChange?: (e:
any) => void; name?: string; value?: string; disabled?: boolean; }
export declare function Choice(props: ChoiceProps): JSX.Element;

// ---- Combobox
import * as React from 'react';
/** Text box with a filtered listbox beneath — course search, people, rooms. */
export interface ComboboxProps { label: string; hint?: string; options: { label: string; detail?: string }[]; value?: string;
onChange?: (q: string) => void; onSelect?: (o: any) => void; placeholder?: string; defaultOpen?: boolean; }
export declare function Combobox(props: ComboboxProps): JSX.Element;

// ---- CommandPalette
```

```
import * as React from 'react';
/**
 * Global search and command palette (⌘K) with keyboard selection.
 */
export interface CommandPaletteProps { groups: { label: string; items: { label: string; icon?: string; detail?: string }[] }[];
placeholder?: string; onSelect?: (item: any) => void; }
export declare function CommandPalette(props: CommandPaletteProps): JSX.Element;

// ---- ConsentBadge
import * as React from 'react';
export interface ConsentBadgeProps { state?: "private" | "shared" | "expired" | "revoked" | "required"; recipient?: string;
expires?: string; onManage?: () => void; }
export declare function ConsentBadge(props: ConsentBadgeProps): JSX.Element;

// ---- ContextBar
import * as React from 'react';
/**
 * The object a screen is about, with its states and actions.
 */
export interface ContextBarProps { kicker?: string; title: string; states?: React.ReactNode; actions?: React.ReactNode; }
export declare function ContextBar(props: ContextBarProps): JSX.Element;

// ---- DataTable
import * as React from 'react';
/**
 * Crisp comparison table with caps headers, hairline rows, tabular numerals.
 * @startingPoint section="Data" subtitle="Crisp comparison table with caps headers, hairline rows, tabular numerals."
viewport="700x400"
 */
export interface DataTableProps { columns: { key: string; label: string; numeric?: boolean; render?: (row: any) =>
React.ReactNode }[]; rows: any[]; caption?: string; }
export declare function DataTable(props: DataTableProps): JSX.Element;

// ---- DateField
import * as React from 'react';
/** Native date/time input with the chosen date spoken back unambiguously. */
export interface DateFieldProps { label: string; hint?: string; value?: string; onChange?: (e: any) => void; withTime?:
boolean; said?: string; }
export declare function DateField(props: DateFieldProps): JSX.Element;

// ---- DecisionTrail
import * as React from 'react';
/**
 * Auditable timeline of who did what, when, and under which authority.
 */
export interface DecisionTrailProps { items: { title: string; who?: string; when?: string; detail?: string; state?: "done" |
"current" | "waiting" | "blocked" }[]; label?: string; }
export declare function DecisionTrail(props: DecisionTrailProps): JSX.Element;

// ---- Dialog
import * as React from 'react';
/**
 * Modal confirmation — focus lands on Cancel, Escape closes, names object + consequence + recovery.
 */
export interface DialogProps { open?: boolean; title: string; children?: React.ReactNode; confirmLabel?: string; cancelLabel?:
string; onConfirm?: () => void; onCancel?: () => void; destructive?: boolean; inline?: boolean; }
export declare function Dialog(props: DialogProps): JSX.Element;

// ---- EmptyState
import * as React from 'react';
/**
 * Explains why a list is empty and offers one relevant next action.
 * @startingPoint section="States" subtitle="Explains why a list is empty and offers one relevant next action."
viewport="700x400"
 */
export interface EmptyStateProps { title: string; body?: string; action?: React.ReactNode; }
export declare function EmptyState(props: EmptyStateProps): JSX.Element;

// ---- ErrorState
import * as React from 'react';
/**
 * Plain-language failure with a required recovery action and an optional support reference.
 */
export interface ErrorStateProps { title: string; body: string; recoverLabel?: string; onRecover?: () => void; secondaryLabel?:
string; onSecondary?: () => void; reference?: string; }
export declare function ErrorState(props: ErrorStateProps): JSX.Element;
```

```
// ---- ErrorSummary
import * as React from 'react';
export interface ErrorSummaryProps { errors: { field: string; message: string }[]; title?: string; }
export declare const ErrorSummary: (props: ErrorSummaryProps & { ref?: any }) => JSX.Element | null;

// ---- Fields
import * as React from 'react';
/**
 * Label/value definition list for records and context.
 */
export interface FieldsProps { items: { field: string; value: React.ReactNode; mono?: boolean }[]; label?: string; }
export declare function Fields(props: FieldsProps): JSX.Element;

// ---- FocusBar
import * as React from 'react';
/** Floating pill during a focus session: time left, task, Pause, End. */
export interface FocusBarProps { task: string; remaining?: string; onPause?: () => void; onEnd?: () => void; breakDue?: string;
}
export declare function FocusBar(props: FocusBarProps): JSX.Element;

// ---- HealthBadge
import * as React from 'react';
export interface HealthBadgeProps { band?: "healthy" | "watch" | "atrisk" | "unknown"; score?: number; driver?: string;
updated?: string; }
export declare function HealthBadge(props: HealthBadgeProps): JSX.Element;

// ---- Icon
import * as React from 'react';
/**
 * Semester line glyph (Lucide-like, 24 grid, 1.5 stroke) from the app's own icon set — 108 names, see assets/icons/.
 * @startingPoint section="Glyph" subtitle="Semester line glyph (Lucide-like, 24 grid, 1.5 stroke) from the app's own icon s"
viewport="700x400"
 */
export interface IconProps { name: string; size?: number; label?: string; strokeWidth?: number; style?: React.CSSProperties;
className?: string; }
export declare function Icon(props: IconProps): JSX.Element;
export declare const ICON_NAMES: string[];

// ---- IconButton
import * as React from 'react';
/**
 * Icon-only control with a required accessible name (also its tooltip).
 */
export interface IconButtonProps { icon: string; label: string; size?: "md" | "sm"; pressed?: boolean; onClick?: (e: any) =>
void; }
export declare function IconButton(props: IconButtonProps): JSX.Element;

// ---- LoadingState
import * as React from 'react';
/**
 * Still structural placeholders plus a spoken "Loading …" — no spinner or shimmer.
 */
export interface LoadingStateProps { what?: string; bars?: number[]; }
export declare function LoadingState(props: LoadingStateProps): JSX.Element;

// ---- MetricTile
import * as React from 'react';
/** One figure worth reading first, with unit, source and age. */
export interface MetricTileProps { label: string; value: string; unit?: string; source?: string; updated?: string; delta?:
string; }
export declare function MetricTile(props: MetricTileProps): JSX.Element;

// ---- NextSteps
import * as React from 'react';
/**
 * Ordered list of next safe actions, each with one line of why.
 */
export interface NextStepsProps { steps: { label: string; why?: string; icon?: string; aside?: React.ReactNode; onClick?: () =>
void }[]; }
export declare function NextSteps(props: NextStepsProps): JSX.Element;

// ---- Notice
import * as React from 'react';
/**
 * One-line service or context notice (role=status).
 */
export interface NoticeProps { head?: string; children: React.ReactNode; }
```

```
export declare function Notice(props: NoticeProps): JSX.Element;

// ---- ObjectCard
import * as React from 'react';
/**
 * The universal decision card: eyebrow → title → provenance → why → metadata → one primary + one quiet secondary.
 * @startingPoint section="Surfaces" subtitle="The universal decision card: eyebrow → title → provenance → why → metadata →
one" viewport="700x400"
 */
export interface ObjectCardProps { kind?: string; statuses?: string[]; title: string; provenance?: { origin: string; by?:
string; cues?: string[]; age?: string }; explanation?: string; metadata?: string; primary?: { label: string; onClick?: () =>
void }; secondary?: { label: string; onClick?: () => void }; tone?: "pending" | "danger"; level?: 2 | 3 | 4; children?:
React.ReactNode; }
export declare function ObjectCard(props: ObjectCardProps): JSX.Element;

// ---- OfflineStrip
import * as React from 'react';
/**
 * Offline strip that says what still works and that changes are queued.
 */
export interface OfflineStripProps { queued?: number; syncs?: boolean; }
export declare function OfflineStrip(props: OfflineStripProps): JSX.Element;

// ---- OpenIn
import * as React from 'react';
/** "Open in" row — the way from one object to every place it matters (Plan, Study, Calendar). */
export interface OpenInProps { targets: string[]; about: string; onOpen?: (t: string) => void; }
export declare function OpenIn(props: OpenInProps): JSX.Element;

// ---- PageHeader
import * as React from 'react';
export interface PageHeaderProps { kicker?: string; title: string; purpose?: string; source?: React.ReactNode; actions?:
React.ReactNode; }
export declare function PageHeader(props: PageHeaderProps): JSX.Element;

// ---- PermissionNotice
import * as React from 'react';
/**
 * States what changed about access, why, and where it is controlled.
 */
export interface PermissionNoticeProps { changed: string; why: string; controlLabel?: string; onControl?: () => void; }
export declare function PermissionNotice(props: PermissionNoticeProps): JSX.Element;

// ---- PolicyBadge
import * as React from 'react';
export interface PolicyBadgeProps { name: string; version?: string; owner?: string; onOpen?: () => void; }
export declare function PolicyBadge(props: PolicyBadgeProps): JSX.Element;

// ---- ProgressState
import * as React from 'react';
/**
 * A determinate wait (upload, import) with real <progress>, percentage in words, Cancel/Retry.
 */
export interface ProgressStateProps { label: string; done?: number; total?: number; note?: string; failed?: string; onCancel?:
() => void; onRetry?: () => void; }
export declare function ProgressState(props: ProgressStateProps): JSX.Element;

// ---- ProvenanceChart
import * as React from 'react';
/** Bar chart that encodes provenance: solid official/connected, outline yours, hatched estimated/AI, dotted stale. */
export interface ProvenanceChartProps { title: string; rows: { label: string; value: number; source?: string }[]; unit?:
string; max?: number; source?: string; demo?: boolean; }
export declare function ProvenanceChart(props: ProvenanceChartProps): JSX.Element;

// ---- ProvenanceChips
import * as React from 'react';
/**
 * A fact's origin chip plus at most two cues and its age, spoken as one sentence.
 */
export interface ProvenanceChipsProps { origin: string; by?: string; cues?: string[]; age?: string; }
export declare function ProvenanceChips(props: ProvenanceChipsProps): JSX.Element;

// ---- QuickActions
import * as React from 'react';
/** Row of 2–5 labelled starting actions for empty and onboarding states. */
export interface QuickActionsProps { actions: { label: string; icon?: string; onClick?: () => void }[]; }
export declare function QuickActions(props: QuickActionsProps): JSX.Element;
```

```
// ---- SaveState
import * as React from 'react';
/**
 * The quiet "Saving… → Saved" line near the work, in a polite live region.
 */
export interface SaveStateProps { status?: string; }
export declare function SaveState(props: SaveStateProps): JSX.Element;

// ---- ScreenGuide
import * as React from 'react';
/** "About this screen" disclosure at the foot of every screen — what it is for, where its data comes from, who decides. */
export interface ScreenGuideProps { items: { q: string; a: string }[]; open?: boolean; }
export declare function ScreenGuide(props: ScreenGuideProps): JSX.Element;

// ---- SectionLabel
import * as React from 'react';
/**
 * The small uppercase kicker that heads a section, with an optional right-aligned count.
 */
export interface SectionLabelProps { children: React.ReactNode; aside?: React.ReactNode; as?: "h2" | "h3" | "h4"; style?:
React.CSSProperties; }
export declare function SectionLabel(props: SectionLabelProps): JSX.Element;

// ---- Segmented
import * as React from 'react';
/**
 * 2–4 mutually exclusive view choices (e.g. Cards / Rows / Timeline).
 */
export interface SegmentedProps { options: (string | { value: string; label: string })[]; value: string; onChange?: (v: string)
=> void; label: string; }
export declare function Segmented(props: SegmentedProps): JSX.Element;

// ---- Select
import * as React from 'react';
/**
 * Native select drawn in Semester's palette.
 */
export interface SelectProps { label: string; hint?: string; options: (string | { value: string; label: string })[]; value?:
string; onChange?: (e: any) => void; id?: string; }
export declare function Select(props: SelectProps): JSX.Element;

// ---- Sheet
import * as React from 'react';
/** Bottom sheet on mobile, window on desktop; the home of "Source & details". Also exports `SourceDetails`. */
export interface SheetProps { title: string; children?: React.ReactNode; onClose?: () => void; inline?: boolean; actions?:
React.ReactNode; }
export declare function Sheet(props: SheetProps): JSX.Element;
export declare function SourceDetails(props: { items: { label: string; value: React.ReactNode }[] }): JSX.Element;

// ---- SideRail
import * as React from 'react';
/**
 * Desktop side rail on dark Ink chrome — persistent, quiet, labelled.
 * @startingPoint section="Navigation" subtitle="Desktop side rail on dark Ink chrome — persistent, quiet, labelled."
viewport="700x400"
 */
export interface SideRailProps { groups: { label?: string; items: { id: string; label: string; icon: string; count?: number }[]
}[]; current?: string; onNavigate?: (id: string) => void; foot?: React.ReactNode; brand?: boolean; }
export declare function SideRail(props: SideRailProps): JSX.Element;

// ---- SourceBadge
import * as React from 'react';
/**
 * Where a fact came from, beside the fact — controlled vocabulary (glyph + word + tone).
 * @startingPoint section="Trust & provenance" subtitle="Where a fact came from, beside the fact — controlled vocabulary (glyph
+ word + " viewport="700x400"
 */
export interface SourceBadgeProps { source: "official" | "connected" | "imported" | "personal" | "ai" | "estimated" | "review"
| "stale" | "external" | "sample"; updated?: string; by?: string; onReport?: () => void; }
export declare function SourceBadge(props: SourceBadgeProps): JSX.Element;

// ---- SourceLine
import * as React from 'react';
/** The Source Line — Semester's signature device: a thin editorial rule from a statement to its source chip. */
export interface SourceLineProps { children: React.ReactNode; source?: string; by?: string; when?: string }
export declare function SourceLine(props: SourceLineProps): JSX.Element;
```

```
// ---- StatusChip
import * as React from 'react';
/**
 * One status chip — glyph + word + tone from the shared status table.
 */
export interface StatusChipProps { status: string; label?: string; }
export declare function StatusChip(props: StatusChipProps): JSX.Element;

// ---- StepStatus
import * as React from 'react';
/**
 * A process in named steps — shows the AI or system working instead of faking certainty.
 */
export interface StepStatusProps { steps: { label: string; state: "waiting" | "working" | "done" | "failed" }[]; label?:
string; }
export declare function StepStatus(props: StepStatusProps): JSX.Element;

// ---- SuccessState
import * as React from 'react';
/**
 * Respectful acknowledgement that points at what is next — no confetti, streaks or ranks.
 */
export interface SuccessStateProps { title: string; body?: string; nextLabel?: string; onNext?: () => void; }
export declare function SuccessState(props: SuccessStateProps): JSX.Element;

// ---- SupportHandoff
import * as React from 'react';
export interface SupportHandoffProps { office: string; reason?: string; channel?: string; hours?: string; status?: "ready" |
"sent" | "received" | "resolved"; reference?: string; onStart?: () => void; }
export declare function SupportHandoff(props: SupportHandoffProps): JSX.Element;

// ---- Switch
import * as React from 'react';
/**
 * On/off setting that speaks its state as a word beside the track.
 */
export interface SwitchProps { checked?: boolean; onChange?: (v: boolean) => void; label: string; disabled?: boolean; }
export declare function Switch(props: SwitchProps): JSX.Element;

// ---- SystemContextBar
import * as React from 'react';
/**
 * Persistent top bar: location, term, ⌘K search, sync state, quiet tools.
 */
export interface SystemContextBarProps { location?: string; term?: string; sync?: string; onSearch?: () => void; actions?:
React.ReactNode; }
export declare function SystemContextBar(props: SystemContextBarProps): JSX.Element;

// ---- TabBar
import * as React from 'react';
/**
 * Mobile bottom navigation for the five canonical destinations, always labelled.
 */
export interface TabBarProps { items: { id: string; label: string; icon: string }[]; current?: string; onNavigate?: (id:
string) => void; }
export declare function TabBar(props: TabBarProps): JSX.Element;

// ---- TextField
import * as React from 'react';
/**
 * Labelled input/textarea with associated hint and error — the label is always visible.
 * @startingPoint section="Forms" subtitle="Labelled input/textarea with associated hint and error — the label is always vis"
viewport="700x400"
 */
export interface TextFieldProps { label: string; hint?: string; error?: string; multiline?: boolean; id?: string; value?:
string; placeholder?: string; onChange?: (e: any) => void; disabled?: boolean; }
export declare function TextField(props: TextFieldProps): JSX.Element;

// ---- UndoToast
import * as React from 'react';
/** One-sentence confirmation with Undo for reversible work — instead of an "are you sure?" dialog. */
export interface UndoToastProps { message: string; onUndo?: () => void; undoLabel?: string; }
export declare function UndoToast(props: UndoToastProps): JSX.Element;

// ---- VisibilityPicker
import * as React from 'react';
```

```
/** "Who can see this?" — the one privacy pattern, with the same four answers in every module. */
export interface VisibilityPickerProps { value?: "only-me" | "course" | "collaborators" | "portfolio"; onChange?: (v: string)
=> void; allowed?: string[]; locked?: boolean; origin?: "student-entered" | "institution-provided" | "connected-system";
onLearnMore?: () => void; }
export declare function VisibilityPicker(props: VisibilityPickerProps): JSX.Element;

// ---- Wordmark
import * as React from 'react';
/**
 * The Semester lockup — three-slab mark + SEMESTER in Barlow Condensed caps; also exports `Mark`.
 */
export interface WordmarkProps { size?: number; metal?: boolean; showWord?: boolean; style?: React.CSSProperties; }
export declare function Wordmark(props: WordmarkProps): JSX.Element;
export declare function Mark(props: { size?: number; style?: React.CSSProperties; className?: string }): JSX.Element;
```

Component sources

The JSX source of every component in the bundle.

components/core/Avatar.jsx · 10 lines

```
import React from 'react';
import { Icon } from './Icon.jsx';
const initials = (n = '') => n.trim().split(/\s+/).filter(Boolean).slice(0, 2).map((p) => p[0].toUpperCase()).join('');
/** Initials in a tinted square that follows the corner setting — never a photo, never derived from an email. Empty name →
profile glyph. */
export function Avatar({ name = '', size = 40 }) {
  const l = initials(name);
  if (!l) return <Icon name="profile" size={size} />;
  return <span className="avatar" aria-hidden="true" style={{ width: size, height: size, fontSize: Math.round(size * 0.4) }}>
{l}</span>;
}
```

components/core/Button.jsx · 14 lines

```
import React from 'react';
import { Icon } from './Icon.jsx';
/** One visually dominant (primary) action per decision region. */
export function Button({ variant = 'secondary', size = 'md', icon, iconAfter, block = false, loading = false, disabled,
children, type = 'button', ...rest }) {
  const cls = ['btn', variant === 'primary' && 'btn-primary', variant === 'ghost' && 'btn-ghost', variant === 'danger' && 'btn-
danger', size === 'sm' && 'btn-sm', block && 'btn-block'].filter(Boolean).join(' ');
  return (
    <button type={type} className={cls} disabled={disabled || loading} aria-busy={loading || undefined} {...rest}>
      {icon && <Icon name={icon} size={size === 'sm' ? 15 : 17} />}
      <span>{loading ? 'Working…' : children}</span>
      {iconAfter && <Icon name={iconAfter} size={size === 'sm' ? 15 : 17} />}
    </button>
  );
}
```

components/core/Icon.jsx · 13 lines

```
import React from 'react';
import { ICON_SHAPES } from '../icons/iconShapes.js';
/** Semester glyph — Lucide-like, 24 grid, stroke 1.5, round caps. Decorative unless `label` is given. */
export function Icon({ name, size = 19, label, strokeWidth = 1.5, style, className }) {
  const shapes = ICON_SHAPES[name] || [];
  return React.createElement('svg', {
    width: size, height: size, viewBox: '0 0 24 24', fill: 'none', stroke: 'currentColor', strokeWidth,
    strokeLinecap: 'round', strokeLinejoin: 'round', className, style: { display: 'block', flex: 'none', ...style },
    'aria-hidden': label ? undefined : 'true', role: label ? 'img' : undefined, 'aria-label': label,
  }, shapes.map((s, i) => s.c ? React.createElement('circle', { key: i, cx: s.c[0], cy: s.c[1], r: s.c[2] }) :
React.createElement('path', { key: i, d: s.d })));
}
export const ICON_NAMES = Object.keys(ICON_SHAPES);
```

components/core/IconButton.jsx · 11 lines

```
import React from 'react';
import { Icon } from './Icon.jsx';
/** Icon-only control. `label` is required — it is the accessible name and the tooltip. */
export function IconButton({ icon, label, size = 'md', pressed, ...rest }) {
  return (
    <button type="button" className={'btn btn-icon' + (size === 'sm' ? ' btn-sm' : '')} aria-label={label} title={label} aria-
pressed={pressed} {...rest}>
      <Icon name={icon} size={size === 'sm' ? 16 : 19} />
    </button>
```

```
  );
}
```

components/core/SectionLabel.jsx · 12 lines

```
import React from 'react';
/** The small caps kicker over a section, with an optional count or aside on the right. */
export function SectionLabel({ children, aside, as = 'h2', style }) {
  const H = as;
  return (
    <div style={{ display: 'flex', alignItems: 'baseline', justifyContent: 'space-between', gap: 'var(--sp-4)', marginBottom:
'var(--sp-4)', ...style }}>
      <H className="kicker" style={{ margin: 0, fontFamily: 'var(--font-body)', fontWeight: 500 }}>{children}</H>
      {aside != null && <span className="kicker nums" style={{ letterSpacing: 0 }}>{aside}</span>}
    </div>
  );
}
```

components/core/Wordmark.jsx · 23 lines

```
import React from 'react';
import { MARK_VIEWBOX, MARK_PATHS } from './markPaths.js';
/** The Semester mark (three slabs) — flat currentColor silhouette. */
export function Mark({ size = 24, style, className }) {
  const [, , w, h] = MARK_VIEWBOX.split(' ').map(Number);
  return (
    <svg width={(size * w) / h} height={size} viewBox={MARK_VIEWBOX} fill="currentColor" aria-hidden="true" focusable="false"
className={className} style={{ display: 'block', flex: 'none', ...style }}>
      {MARK_PATHS.map((d) => <path key={d} d={d} />)}
    </svg>
  );
}
/** Mark + SEMESTER lockup. `metal` paints the brushed sterling used on dark chrome. */
export function Wordmark({ size = 20, metal = false, showWord = true, style }) {
  return (
    <span aria-label="Semester" role="img" style={{ display: 'inline-flex', alignItems: 'center', gap: size * 0.5, color: metal
? 'var(--sterling)' : 'currentColor', ...style }}>
      <Mark size={size} />
      {showWord && (
        <span aria-hidden="true" className={metal ? 'chrome-text' : undefined} style={{ fontFamily: 'var(--font-heading)',
fontWeight: 600, textTransform: 'uppercase', letterSpacing: 'var(--tracking-wordmark)', fontSize: size * 0.82, lineHeight: 1,
paddingTop: 1 }}>Semester</span>
      )}
    </span>
  );
}
```

components/data/DataTable.jsx · 14 lines

```
import React from 'react';
/** Crisp comparison table. Use only when comparing columns; the region (not the page) scrolls on compact screens. */
export function DataTable({ columns = [], rows = [], caption }) {
  return (
    <div className="table-wrap" role="region" aria-label={caption} tabIndex={0}>
      <table className="table">
        {caption && <caption className="sr-only">{caption}</caption>}
        <thead><tr>{columns.map((c) => <th key={c.key} className={c.numeric ? 'num' : undefined} scope="col">{c.label}</th>)}
</tr></thead>
        <tbody>{rows.map((r, i) => <tr key={r.id || i}>{columns.map((c) => <td key={c.key} className={c.numeric ? 'num' :
undefined}>{c.render ? c.render(r) : r[c.key]}</td>)}</tr>)}</tbody>
      </table>
    </div>
  );
}
```

components/data/DecisionTrail.jsx · 15 lines

```
import React from 'react';
/** Auditable timeline: who did what, when, under which authority. States: done, current, waiting, blocked. */
export function DecisionTrail({ items = [], label = 'Decision trail' }) {
```

```
  return (
    <ol className="trail" aria-label={label}>
      {items.map((it, i) => (
        <li key={i} className="trail-item" data-state={it.state || 'done'} aria-current={it.state === 'current' ? 'step' :
undefined}>
          <span className="trail-dot" aria-hidden="true"></span>
          <div><div className="trail-title">{it.title}</div><div className="trail-meta">{[it.who,
it.when].filter(Boolean).join(' · ')}</div>{it.detail && <div style={{ fontSize: 'var(--type-sm)', color: 'var(--text-
secondary)', marginTop: 'var(--sp-2)' }}>{it.detail}</div>}</div>
        </li>
      ))}
    </ol>
  );
}
```

components/data/Fields.jsx · 10 lines

```
import React from 'react';
/** Label/value definition list — the console's record and context-bar layout. */
export function Fields({ items = [], label }) {
  return (
    <dl className="fields" aria-label={label}>
      {items.map((it) => <React.Fragment key={it.field}><dt>{it.field}</dt><dd className={it.mono ? 'mono' : undefined}>
{it.value}</dd></React.Fragment>)}
    </dl>
  );
}
```

components/data/MetricTile.jsx · 14 lines

```
import React from 'react';
import { SourceBadge } from '../trust/SourceBadge.jsx';
/** One figure worth reading first, with its unit, its source and how old it is. Never an outcome claim. */
export function MetricTile({ label, value, unit, source, updated, delta }) {
  return (
    <div className="metric">
      <div className="kicker">{label}</div>
      <div><span className="metric-value">{value}</span>{unit && <span style={{ marginLeft: 6, fontSize: 'var(--type-sm)',
color: 'var(--text-secondary)' }}>{unit}</span>}</div>
      {delta && <div className="metric-delta">{delta}</div>}
      {source && <SourceBadge source={source} updated={updated} />}
    </div>
  );
}
```

components/data/ProvenanceChart.jsx · 24 lines

```
import React from 'react';
const ENC = { official: 'solid', connected: 'solid', personal: 'outline', estimated: 'hatched', ai: 'hatched', stale: 'dotted'
};
const SAID = { solid: 'Official or connected', outline: 'Yours', hatched: 'Estimated or AI-derived', dotted: 'Out of date' };
/** Bar chart that encodes provenance, not just value: solid = official/connected, outline = yours, hatched = estimated/AI,
dotted = stale. Every bar labelled directly. */
export function ProvenanceChart({ title, rows = [], unit = '', max, source, demo = false }) {
  const m = max || Math.max(...rows.map((r) => r.value)) || 1;
  const used = [...new Set(rows.map((r) => ENC[r.source] || 'solid'))];
  return (
    <figure className="chart" style={{ margin: 0 }}>
      <figcaption className="chart-title">{title}</figcaption>
      <div className="chart-bars">
        {rows.map((r) => (
          <div className="chart-row" key={r.label}>
            <span>{r.label}</span>
            <div className="chart-track" aria-hidden="true"><div className="chart-bar" data-enc={ENC[r.source] || 'solid'}
style={{ width: (r.value / m) * 100 + '%' }}></div></div>
            <span className="nums" style={{ textAlign: 'right' }}>{r.value}{unit}</span>
          </div>
        ))}
      </div>
      <div className="chart-foot">{used.map((e) => <span key={e}><span className="chart-bar legend-swatch" data-enc={e}></span>
{SAID[e]}</span>)}{source && <span>Source: {source}</span>}{demo && <span>Illustrative demo data</span>}</div>
```

```
    </figure>
  );
}
```

components/feedback/EmptyState.jsx · 12 lines

```
import React from 'react';
/** Say why it is empty, and offer one relevant next action. */
export function EmptyState({ title, body, action }) {
  return (
    <div className="state" data-kind="empty">
      <div className="state-title">{title}</div>
      {body && <p className="state-body">{body}</p>}
      {action && <div className="state-actions">{action}</div>}
    </div>
  );
}
```

components/feedback/ErrorState.jsx · 16 lines

```
import React from 'react';
/** What went wrong in plain words, and the way out. A recovery action is required. */
export function ErrorState({ title, body, recoverLabel = 'Try again', onRecover, secondaryLabel, onSecondary, reference }) {
  return (
    <div className="state" data-kind="error" role="alert">
      <div className="state-title"><span className="status-glyph" aria-hidden="true">!</span>{title}</div>
      <p className="state-body">{body}</p>
      <div className="state-actions">
        <button type="button" className="btn btn-primary" onClick={onRecover}>{recoverLabel}</button>
        {secondaryLabel && <button type="button" className="btn btn-ghost" onClick={onSecondary}>{secondaryLabel}</button>}
      </div>
      {reference && <p className="state-ref nums">Reference: <span className="mono">{reference}</span></p>}
    </div>
  );
}
```

components/feedback/LoadingState.jsx · 11 lines

```
import React from 'react';
/** Still structural placeholders + a spoken sentence. No spinner, no shimmer. */
export function LoadingState({ what = 'this screen', bars = [24, 14, 72, 72] }) {
  return (
    <div className="state-loading" aria-busy="true">
      <p className="sr-only" role="status">Loading {what}…</p>
      {bars.map((h, i) => <div key={i} className="state-loading-bar" style={{ height: h, width: i === 1 ? '40%' : i === 0 ?
'62%' : '100%' }} aria-hidden="true"></div>)}
    </div>
  );
}
```

components/feedback/Notice.jsx · 6 lines

```
import React from 'react';
/** A one-line service or context notice. role=status — quiet, never a giant red banner. */
export function Notice({ head, children }) {
  return <p className="notice" role="status">{head && <strong className="notice-head">{head}</strong>}{children}</p>;
}
```

components/feedback/OfflineStrip.jsx · 7 lines

```
import React from 'react';
/** Says what still works and that work is queued. */
export function OfflineStrip({ queued = 0, syncs = true }) {
  const tail = !syncs ? '; everything is kept on this device.' : queued > 0 ? ` — ${queued} change${queued === 1 ? '' : 's'}
will sync when you are back.` : '; changes sync when you are back.';
  return <div className="state" data-kind="offline" role="status"><span className="status-glyph" aria-hidden="true">⊘</span>
```

```
<span>You are offline. You can keep working{tail}</span></div>;
}
```

components/feedback/PermissionNotice.jsx · 12 lines

```
import React from 'react';
/** What changed about access, why, who controls it, and where to change it. */
export function PermissionNotice({ changed, why, controlLabel, onControl }) {
  return (
    <div className="state" data-kind="permission" role="status">
      <div className="state-title"><span className="status-glyph" aria-hidden="true">⊙</span>{changed}</div>
      <p className="state-body">{why}</p>
      {controlLabel && <button type="button" className="link-quiet" style={{ alignSelf: 'flex-start' }} onClick={onControl}>
{controlLabel}</button>}
    </div>
  );
}
```

components/feedback/ProgressState.jsx · 17 lines

```
import React from 'react';
/** A determinate wait — an upload, an import. Real <progress>, percentage in words, Cancel/Retry. */
export function ProgressState({ label, done = 0, total = 100, note, failed, onCancel, onRetry }) {
  const pct = total > 0 ? Math.min(100, Math.round((done / total) * 100)) : 0;
  return (
    <div className="state">
      <div className="state-progress-head"><span>{label}</span><span className="nums">{failed ? 'Stopped' : pct + '%'}</span>
</div>
      {failed ? <p className="state-body" role="alert">{failed}</p> : <progress className="state-progress-bar" value={done}
max={total} aria-label={label}></progress>}
      {note && <p className="state-note">{note}</p>}
      <div className="state-actions">
        {failed && onRetry && <button type="button" className="btn btn-primary btn-sm" onClick={onRetry}>Retry</button>}
        {!failed && onCancel && pct < 100 && <button type="button" className="btn btn-ghost btn-sm" onClick=
{onCancel}>Cancel</button>}
      </div>
    </div>
  );
}
```

components/feedback/SaveState.jsx · 8 lines

```
import React from 'react';
import { STATUSES } from '../trust/vocabulary.js';
/** A quiet save/sync line near the work — not a toast. Polite live region; offline/conflict interrupt. */
export function SaveState({ status = 'saved' }) {
  const s = STATUSES[status] || STATUSES.unknown;
  return <span className="save-state" data-tone={s.tone} role={s.urgent ? 'alert' : 'status'} aria-live={s.urgent ? 'assertive'
: 'polite'}><span className="status-glyph" aria-hidden="true">{s.glyph}</span>{s.label}</span>;
}
```

components/feedback/StepStatus.jsx · 16 lines

```
import React from 'react';
const GLYPH = { waiting: '○', working: '…', done: '✓', failed: '!' };
const SAID = { waiting: 'waiting', working: 'in progress', done: 'done', failed: 'failed' };
/** A process in named steps — "Gathering sources → Drafting → Ready". Shows the work instead of faking certainty. */
export function StepStatus({ steps = [], label = 'Progress' }) {
  return (
    <ol className="state-steps" aria-label={label}>
      {steps.map((s) => (
        <li key={s.label} data-state={s.state} aria-current={s.state === 'working' ? 'step' : undefined}>
          <span className="status-glyph" aria-hidden="true" style={{ width: '1em', textAlign: 'center' }}>{GLYPH[s.state]}
</span>{s.label}<span className="sr-only"> — {SAID[s.state]}</span>
        </li>
      ))}
    </ol>
  );
}
```

components/feedback/SuccessState.jsx · 12 lines

```
import React from 'react';
/** Acknowledge, then point at what is next. Respectful, never celebratory. */
export function SuccessState({ title, body, nextLabel, onNext }) {
  return (
    <div className="state" data-kind="success" role="status">
      <div className="state-title"><span className="status-glyph" aria-hidden="true">✓</span>{title}</div>
      {body && <p className="state-body">{body}</p>}
      {nextLabel && <p className="state-body">Next: <button type="button" className="link-quiet" style={{ fontSize: 'inherit'
}} onClick={onNext}>{nextLabel}</button></p>}
    </div>
  );
}
```

components/forms/Choice.jsx · 11 lines

```
import React from 'react';
/** Radio or checkbox with a label and an optional sentence explaining the consequence. */
export function Choice({ type = 'radio', label, about, checked, onChange, name, value, disabled }) {
  return (
    <label className="choice" style={disabled ? { opacity: 0.5, cursor: 'not-allowed' } : undefined}>
      <input type={type} name={name} value={value} checked={checked} onChange={onChange} disabled={disabled} />
      <span>{label}{about && <span className="choice-about">{about}</span>}</span>
    </label>
  );
}
```

components/forms/Combobox.jsx · 22 lines

```
import React, { useId, useState } from 'react';
/** Text box with a filtered list beneath — course search, people, rooms. Arrow keys move an edge + weight, not colour alone.
*/
export function Combobox({ label, hint, options = [], value = '', onChange, onSelect, placeholder, defaultOpen = false }) {
  const id = useId(); const [q, setQ] = useState(value); const [open, setOpen] = useState(defaultOpen); const [sel, setSel] =
useState(0);
  const shown = options.filter((o) => (o.label + ' ' + (o.detail || '')).toLowerCase().includes(q.toLowerCase())).slice(0, 8);
  const pick = (o) => { setQ(o.label); setOpen(false); onSelect && onSelect(o); };
  return (
    <div className="combo">
      <label className="field-label" htmlFor={id}>{label}</label>
      {hint && <span className="field-hint">{hint}</span>}
      <input id={id} className="input" role="combobox" aria-expanded={open} aria-controls={id + '-list'} aria-
autocomplete="list" value={q} placeholder={placeholder}
        onChange={(e) => { setQ(e.target.value); setOpen(true); setSel(0); onChange && onChange(e.target.value); }} onFocus={()
=> setOpen(true)}
        onKeyDown={(e) => { if (e.key === 'ArrowDown') { e.preventDefault(); setSel((s) => Math.min(shown.length - 1, s + 1));
} if (e.key === 'ArrowUp') { e.preventDefault(); setSel((s) => Math.max(0, s - 1)); } if (e.key === 'Enter' && shown[sel])
pick(shown[sel]); if (e.key === 'Escape') setOpen(false); }} />
      {open && shown.length > 0 && (
        <ul className="combo-list" id={id + '-list'} role="listbox">
          {shown.map((o, i) => <li key={o.label} role="option" aria-selected={i === sel} className="combo-option" onMouseEnter=
{() => setSel(i)} onMouseDown={(e) => { e.preventDefault(); pick(o); }}>{o.label}{o.detail && <span className="combo-detail">
{o.detail}</span>}</li>)}
        </ul>
      )}
    </div>
  );
}
```

components/forms/DateField.jsx · 14 lines

```
import React, { useId } from 'react';
/** Native date (and optional time) input with a visible label and the date spoken back unambiguously. */
export function DateField({ label, hint, value, onChange, withTime = false, said }) {
  const id = useId();
  return (
    <div className="field">
      <label className="field-label" htmlFor={id}>{label}</label>
      {hint && <span className="field-hint">{hint}</span>}
      <input id={id} className="input nums" type={withTime ? 'datetime-local' : 'date'} value={value} onChange={onChange} />
```

```
      {said && <span className="field-hint nums">{said}</span>}
    </div>
  );
}
```

components/forms/Segmented.jsx · 13 lines

```
import React from 'react';
/** 2–4 mutually exclusive views (Cards / Rows / Timeline). */
export function Segmented({ options = [], value, onChange, label }) {
  return (
    <div className="seg" role="radiogroup" aria-label={label}>
      {options.map((o) => {
        const v = typeof o === 'string' ? o : o.value; const l = typeof o === 'string' ? o : o.label;
        return <button key={v} type="button" role="radio" aria-checked={v === value} className="seg-opt" onClick={() =>
onChange && onChange(v)}>{l}</button>;
      })}
    </div>
  );
}
```

components/forms/Select.jsx · 15 lines

```
import React, { useId } from 'react';
/** Native select, drawn in Semester's palette. Never imitate a select with a menu. */
export function Select({ label, hint, options = [], id, ...rest }) {
  const auto = useId(); const fid = id || auto;
  return (
    <div className="field">
      <label className="field-label" htmlFor={fid}>{label}</label>
      {hint && <span className="field-hint">{hint}</span>}
      <select id={fid} className="input" {...rest}>
        {options.map((o) => typeof o === 'string' ? <option key={o} value={o}>{o}</option> : <option key={o.value} value=
{o.value}>{o.label}</option>)}
      </select>
    </div>
  );
}
```

components/forms/Switch.jsx · 12 lines

```
import React from 'react';
/** On/off setting. The state is spoken as a word beside the track, never carried by colour alone. */
export function Switch({ checked = false, onChange, label, disabled }) {
  return (
    <button type="button" role="switch" aria-checked={checked} className="switch" disabled={disabled} onClick={() => onChange
&& onChange(!checked)} style={{ background: 'none', border: 0, padding: 0, color: 'var(--text-primary)', textAlign: 'left' }}>
      <span className="switch-track" aria-hidden="true"></span>
      <span className="switch-state" aria-hidden="true">{checked ? 'On' : 'Off'}</span>
      <span>{label}</span>
    </button>
  );
}
```

components/forms/TextField.jsx · 15 lines

```
import React, { useId } from 'react';
/** Labelled text input. Persistent visible label; hint and error are programmatically associated. */
export function TextField({ label, hint, error, multiline = false, id, ...rest }) {
  const auto = useId(); const fid = id || auto; const hid = fid + '-hint'; const eid = fid + '-err';
  const El = multiline ? 'textarea' : 'input';
  return (
    <div className="field">
      <label className="field-label" htmlFor={fid}>{label}</label>
      {hint && <span className="field-hint" id={hid}>{hint}</span>}
      <El id={fid} className="input" aria-invalid={error ? 'true' : undefined} aria-describedby={[hint && hid, error &&
eid].filter(Boolean).join(' ') || undefined} {...rest} />
      {error && <span className="field-error" id={eid}><span aria-hidden="true">!</span>{error}</span>}
    </div>
```

```
  );
}
```

components/forms/VisibilityPicker.jsx · 25 lines

```
import React, { useId } from 'react';
const AUDIENCES = [
  { id: 'only-me', label: 'Only me', about: 'Nobody else can see it.' },
  { id: 'course', label: 'Course', about: 'People enrolled in the course it is attached to.' },
  { id: 'collaborators', label: 'Selected collaborators', about: 'Only the people you add by name.' },
  { id: 'portfolio', label: 'Public portfolio', about: 'Anyone with the link to your portfolio.' },
];
const ORIGIN = { 'student-entered': 'Entered by you', 'institution-provided': 'Provided by your institution', 'connected-
system': 'From a system you connected' };
/** "Who can see this?" — the one privacy pattern, same four answers in every module. Only offers audiences the module can
honour. */
export function VisibilityPicker({ value = 'only-me', onChange, allowed = ['only-me', 'course', 'collaborators', 'portfolio'],
locked = false, origin = 'student-entered', onLearnMore }) {
  const name = useId(); const chosen = AUDIENCES.find((a) => a.id === value) || AUDIENCES[0];
  return (
    <div className="visibility">
      {locked ? <p className="visibility-said"><span className="kicker">Who can see this</span>{chosen.label} — {chosen.about}
</p> : (
        <fieldset><legend className="kicker" style={{ marginBottom: 'var(--sp-2)' }}>Who can see this?</legend>
          {AUDIENCES.filter((a) => allowed.includes(a.id)).map((a) => (
            <label key={a.id} className="choice"><input type="radio" name={name} checked={value === a.id} onChange={() =>
onChange && onChange(a.id)} /><span>{a.label}<span className="choice-about">{a.about}</span></span></label>
          ))}
        </fieldset>
      )}
      <p className="visibility-said"><span className="kicker">Data source</span>{ORIGIN[origin]} · <button type="button"
className="link-quiet" onClick={onLearnMore}>Learn more</button></p>
    </div>
  );
}
```

components/governance/ApprovalBanner.jsx · 8 lines

```
import React from 'react';
/** Shows where a consequential change is in its approval path. */
export function ApprovalBanner({ state = 'required', what, approver, requestedBy, onRequest, onApprove, onReject }) {
  const L = { required: ['?', 'Needs approval'], pending: ['…', 'Waiting for approval'], approved: ['✓', 'Approved'], rejected:
['⊘', 'Not approved'] };
  const [g, t] = L[state] || L.required;
  return <div className="approval-banner" data-state={state} role="status"><span className="gov-glyph" aria-hidden="true">{g}
</span><div className="approval-body"><b>{t}: {what}</b><span className="gov-sub">{state === 'pending' && approver ? 'With ' +
approver : state === 'required' ? 'A second person must approve before this applies.' : requestedBy ? 'Requested by ' +
requestedBy : ''}</span></div><div className="approval-actions">{state === 'required' && onRequest && <button type="button"
className="btn btn-sm btn-primary" onClick={onRequest}>Request approval</button>}{state === 'pending' && onApprove && <button
type="button" className="btn btn-sm btn-primary" onClick={onApprove}>Approve</button>}{state === 'pending' && onReject &&
<button type="button" className="btn btn-sm" onClick={onReject}>Reject</button>}</div></div>;
}
```

components/governance/ConsentBadge.jsx · 8 lines

```
import React from 'react';
/** Shows who can see something because the student shared it, until when. */
export function ConsentBadge({ state = 'private', recipient, expires, onManage }) {
  const L = { private: ['○', 'Only you'], shared: ['⇄', 'Shared with ' + (recipient || 'someone')], expired: ['⊘', 'Share
expired'], revoked: ['⊘', 'Share revoked'], required: ['?', 'Needs your consent'] };
  const [g, t] = L[state] || L.private;
  return <span className="gov-badge" data-kind="consent" data-state={state}><span className="gov-glyph" aria-hidden="true">{g}
</span><span>{t}</span>{expires && state === 'shared' && <span className="gov-sub">· until {expires}</span>}{onManage &&
<button type="button" className="gov-link" onClick={onManage}>Manage</button>}</span>;
}
```

components/governance/ErrorSummary.jsx · 7 lines

```
import React from 'react';
/** Form error summary: count, each error linked to its field. Focus moves here on submit. */
```

```
export const ErrorSummary = React.forwardRef(function ErrorSummary({ errors = [], title }, ref) {
  if (!errors.length) return null;
  return <div className="error-summary" role="alert" tabIndex={-1} ref={ref}><b>! {title || (errors.length + (errors.length > 1
? ' fields need' : ' field needs') + ' attention')}</b><ul>{errors.map((e) => <li key={e.field}><a href={'#' + e.field}
onClick={(ev) => { const el = document.getElementById(e.field); if (el) { ev.preventDefault(); el.focus(); } }}>{e.message}</a>
</li>)}</ul></div>;
});
```

components/governance/HealthBadge.jsx · 8 lines

```
import React from 'react';
/** Health band with the score's top driver — for tenants, integrations, customers. */
export function HealthBadge({ band = 'healthy', score, driver, updated }) {
  const L = { healthy: ['✓', 'Healthy', 'success'], watch: ['?', 'Watch', 'pending'], atrisk: ['!', 'At risk', 'danger'],
unknown: ['…', 'Not enough data', 'neutral'] };
  const [g, t, tone] = L[band] || L.unknown;
  return <span className="health-badge" data-tone={tone}><span className="gov-glyph" aria-hidden="true">{g}</span><b>{t}</b>
{score != null && <span className="nums">{score}</span>}{driver && <span className="gov-sub">· {driver}</span>}{updated &&
<span className="gov-sub">· {updated}</span>}</span>;
}
```

components/governance/PageHeader.jsx · 6 lines

```
import React from 'react';
/** Page head: kicker, one H1, what the page helps you do, source line, primary action. */
export function PageHeader({ kicker, title, purpose, source, actions }) {
  return <header className="page-header"><div className="page-header-text">{kicker && <div className="kicker">{kicker}</div>}
<h1>{title}</h1>{purpose && <p className="page-header-purpose">{purpose}</p>}{source && <div className="page-header-source">
{source}</div>}</div>{actions && <div className="page-header-actions">{actions}</div>}</header>;
}
```

components/governance/PolicyBadge.jsx · 7 lines

```
import React from 'react';
/** Names the policy that governs what is shown, with its owner and version. */
export function PolicyBadge({ name, version, owner, onOpen }) {
  const text = <><span className="gov-glyph" aria-hidden="true">§</span><span>{name}{version ? ' v' + version : ''}</span>
{owner && <span className="gov-sub">· {owner}</span>}</>;
  return onOpen ? <button type="button" className="gov-badge" data-kind="policy" onClick={onOpen} aria-label={'Policy: ' + name
+ (version ? ' version ' + version : '') + (owner ? ', owned by ' + owner : '')}>{text}</button> : <span className="gov-badge"
data-kind="policy">{text}</span>;
}
```

components/governance/SupportHandoff.jsx · 7 lines

```
import React from 'react';
/** Hands a question to the office that owns it; Semester tracks status only. */
export function SupportHandoff({ office, reason, channel, hours, status = 'ready', reference, onStart }) {
  const L = { ready: 'Not started', sent: 'Sent', received: 'Received by the office', resolved: 'Resolved' };
  return <div className="support-handoff"><div className="gov-sub">Owned by</div><b className="handoff-office">{office}</b>
{reason && <p className="handoff-reason">{reason}</p>}<dl className="handoff-meta">{channel && <><dt>How</dt><dd>{channel}</dd>
</>}{hours && <><dt>Hours</dt><dd>{hours}</dd></>}<dt>Status</dt><dd>{L[status] || status}{reference ? ' · ' + reference : ''}
</dd></dl>{status === 'ready' && onStart && <button type="button" className="btn btn-sm btn-primary" onClick={onStart}>
{'Contact ' + office}</button>}<p className="gov-sub">The office decides in its own system. Semester shows the status and never
edits it.</p></div>;
}
```

components/navigation/CommandPalette.jsx · 29 lines

```
import React, { useState } from 'react';
import { Icon } from '../core/Icon.jsx';
/** Global search + command palette. Arrow keys move an edge + weight, not colour alone. */
export function CommandPalette({ groups = [], placeholder = 'Search Semester or type a command', onSelect }) {
  const [q, setQ] = useState(''); const [sel, setSel] = useState(0);
  const flat = []; const shown = groups.map((g) => ({ ...g, items: g.items.filter((i) =>
i.label.toLowerCase().includes(q.toLowerCase())) })).filter((g) => g.items.length);
  shown.forEach((g) => g.items.forEach((i) => flat.push(i)));
  const key = (e) => { if (e.key === 'ArrowDown') { e.preventDefault(); setSel((s) => Math.min(flat.length - 1, s + 1)); } if
```

```
(e.key === 'ArrowUp') { e.preventDefault(); setSel((s) => Math.max(0, s - 1)); } if (e.key === 'Enter' && flat[sel] &&
onSelect) onSelect(flat[sel]); };
  let n = -1;
  return (
    <div className="palette" role="dialog" aria-label="Command palette">
      <div className="palette-input"><Icon name="search" size={19} style={{ color: 'var(--text-secondary)' }} /><input value=
{q} onChange={(e) => { setQ(e.target.value); setSel(0); }} onKeyDown={key} placeholder={placeholder} aria-label={placeholder}
/><span className="kbd">esc</span></div>
      <div role="listbox">
        {shown.map((g) => (
          <div className="palette-group" key={g.label} role="group" aria-label={g.label}>
            <div className="palette-group-label">{g.label}</div>
            {g.items.map((it) => { n++; const i = n; return (
              <button key={it.label} type="button" role="option" aria-selected={i === sel} className="palette-option"
onMouseEnter={() => setSel(i)} onClick={() => onSelect && onSelect(it)}>
                {it.icon && <Icon name={it.icon} size={17} style={{ color: 'var(--text-secondary)' }} />}<span>{it.label}
</span>{it.detail && <span className="palette-option-detail">{it.detail}</span>}
              </button>); })}
          </div>
        ))}
        {flat.length === 0 && <div style={{ padding: 'var(--sp-7)', color: 'var(--text-secondary)', fontSize: 'var(--type-sm)'
}}>Nothing matches “{q}”. Try a course code, a person, or “ask”.</div>}
      </div>
      <div className="palette-foot"><span><span className="kbd">↑</span> <span className="kbd">↓</span> move</span><span><span
className="kbd">↵</span> open</span><span>Results respect your privacy settings</span></div>
    </div>
  );
}
```

components/navigation/SideRail.jsx · 23 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
import { Wordmark } from '../core/Wordmark.jsx';
/** Desktop side rail — persistent, quiet, always on dark chrome. Groups: [{label?, items:[{id,label,icon,count}]}]. */
export function SideRail({ groups = [], current, onNavigate, foot, brand = true }) {
  return (
    <nav className="rail" aria-label="Semester">
      {brand && <div className="rail-brand"><Wordmark size={18} metal /></div>}
      {groups.map((g, gi) => (
        <React.Fragment key={gi}>
          {g.label && <div className="rail-label">{g.label}</div>}
          {g.items.map((it) => (
            <button key={it.id} type="button" className="rail-row" aria-current={current === it.id ? 'page' : undefined}
onClick={() => onNavigate && onNavigate(it.id)}>
              <Icon name={it.icon} size={19} /><span>{it.label}</span>{it.count != null && <span className="rail-count">
{it.count}</span>}
            </button>
          ))}
        </React.Fragment>
      ))}
      {foot && <div className="rail-foot">{foot}</div>}
    </nav>
  );
}
```

components/navigation/SystemContextBar.jsx · 19 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
import { SaveState } from '../feedback/SaveState.jsx';
/** Persistent top bar: where you are, the term, global search (⌘K), sync state, and quiet tools. */
export function SystemContextBar({ location, term, sync = 'synced', onSearch, actions }) {
  return (
    <div className="syscontext">
      <div className="syscontext-group">
        {location && <span style={{ color: 'var(--text-primary)', fontSize: 'var(--type-sm)' }}>{location}</span>}
        {term && <span className="syscontext-term">{term}</span>}
      </div>
      <button type="button" className="syscontext-search" onClick={onSearch} aria-label="Search or run a command">
        <Icon name="search" size={15} /><span style={{ flex: 1, textAlign: 'left' }}>Search or ask…</span><span
className="kbd">⌘K</span>
      </button>
```

```
      <div className="syscontext-group"><SaveState status={sync} />{actions}</div>
    </div>
  );
}
```

components/navigation/TabBar.jsx · 15 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
/** Mobile bottom navigation — the five canonical destinations, labelled. */
export function TabBar({ items = [], current, onNavigate }) {
  return (
    <nav className="tabbar" aria-label="Semester">
      {items.map((it) => (
        <button key={it.id} type="button" className="tab" aria-current={current === it.id ? 'page' : undefined} onClick={() =>
onNavigate && onNavigate(it.id)}>
          <Icon name={it.icon} size={21} /><span>{it.label}</span>
        </button>
      ))}
    </nav>
  );
}
```

components/overlays/FocusBar.jsx · 14 lines

```
import React from 'react';
/** Floating bar during a focus session — what you're working on, time left, and the way out. Only frosted surface besides the
tab bar. */
export function FocusBar({ task, remaining = '24:10', onPause, onEnd, breakDue }) {
  return (
    <div className="focus-bar" role="region" aria-label="Focus session">
      <span className="focus-timer" aria-label={'Time left ' + remaining}>{remaining}</span>
      <span style={{ color: 'var(--text-secondary)' }}>{task}</span>
      {breakDue && <span style={{ color: 'var(--text-primary)' }}>{breakDue}</span>}
      <button type="button" className="btn btn-sm" onClick={onPause}>Pause</button>
      <button type="button" className="btn btn-sm btn-ghost" onClick={onEnd}>End session</button>
    </div>
  );
}
```

components/overlays/Sheet.jsx · 19 lines

```
import React from 'react';
import { IconButton } from '../core/IconButton.jsx';
/** Bottom sheet (mobile) / window (desktop). Default use: "Source & details" — origin, freshness, used in, limitations. */
export function Sheet({ title, children, onClose, inline = false, actions }) {
  const box = (
    <div className={'sheet' + (inline ? ' is-inline' : '')} role="dialog" aria-modal={!inline} aria-label={title}>
      {!inline && <div className="sheet-grip" aria-hidden="true"></div>}
      <div className="sheet-head"><h2 style={{ fontSize: 'var(--type-display-xs)', fontFamily: 'var(--font-body)', fontWeight:
600, letterSpacing: 0 }}>{title}</h2>{onClose && <IconButton icon="close" label="Close" onClick={onClose} />}</div>
      {children}
      {actions && <div className="dialog-actions" style={{ marginTop: 'var(--sp-7)' }}>{actions}</div>}
    </div>
  );
  return inline ? box : <div className="sheet-scrim" onClick={(e) => e.target === e.currentTarget && onClose && onClose()}>
{box}</div>;
}
/** The Source & details definition list used inside a Sheet. */
export function SourceDetails({ items = [] }) {
  return <dl className="source-list">{items.map((it) => <React.Fragment key={it.label}><dt>{it.label}</dt><dd>{it.value}</dd>
</React.Fragment>)}</dl>;
}
```

components/overlays/UndoToast.jsx · 6 lines

```
import React from 'react';
/** Reversible work gets an undo, not a confirmation. One sentence + Undo; polite live region; never auto-hides under 6s. */
export function UndoToast({ message, onUndo, undoLabel = 'Undo' }) {
  return <div className="toast" role="status" aria-live="polite"><span>{message}</span>{onUndo && <button type="button"
```

```
className="btn btn-sm" onClick={onUndo}>{undoLabel}</button>}</div>;
}
```

components/surfaces/ActionPreview.jsx · 19 lines

```
import React from 'react';
const RECOVERY = { undo: 'You can undo this.', request: 'This can be reversed on request: ', none: 'This can't be undone.' };
/** What a person is about to do, said before they do it. Recovery is required. */
export function ActionPreview({ subject, says, exactly, doesNotChange, recovery = { kind: 'undo' }, subjectTo, provenance }) {
  return (
    <div className="action-preview">
      {subject && <p style={{ margin: 0 }}><strong>{subject}</strong></p>}
      {provenance}
      <dl className="action-preview-facts">
        <dt>What happens</dt><dd>{says}</dd>
        {exactly && <><dt>Exactly what</dt><dd>{exactly}</dd></>}
        {doesNotChange && <><dt>What stays the same</dt><dd>{doesNotChange}</dd></>}
        <dt>Taking it back</dt><dd>{RECOVERY[recovery.kind]}{recovery.how ? ' ' + recovery.how : ''}</dd>
      </dl>
      {subjectTo && <p className="action-preview-caveat">{subjectTo}</p>}
    </div>
  );
}
```

components/surfaces/ContextBar.jsx · 13 lines

```
import React from 'react';
/** The object a screen is about, with its states and actions — one bar, not a card inside a card. */
export function ContextBar({ kicker, title, states, actions }) {
  return (
    <section className="context-bar">
      {kicker && <div className="kicker">{kicker}</div>}
      <h2 className="context-bar-title">{title}</h2>
      {states && <div className="context-bar-states">{states}</div>}
      {actions && <div className="object-card-actions" style={{ marginTop: 'var(--sp-2)' }}>{actions}</div>}
    </section>
  );
}
```

components/surfaces/Dialog.jsx · 19 lines

```
import React, { useEffect, useRef } from 'react';
/** Modal confirmation. Focus lands on Cancel; Escape closes; consequential actions name object, consequence, recovery. */
export function Dialog({ open = true, title, children, confirmLabel = 'Confirm', cancelLabel = 'Cancel', onConfirm, onCancel,
destructive = false, inline = false }) {
  const cancelRef = useRef(null);
  useEffect(() => { if (open && !inline && cancelRef.current) cancelRef.current.focus(); }, [open, inline]);
  if (!open) return null;
  const box = (
    <div className="dialog" role="dialog" aria-modal={!inline} aria-label={title} onKeyDown={(e) => { if (e.key === 'Escape' &&
onCancel) onCancel(); }}>
      <h2 className="dialog-title">{title}</h2>
      <div>{children}</div>
      <div className="dialog-actions">
        <button ref={cancelRef} type="button" className="btn" onClick={onCancel}>{cancelLabel}</button>
        <button type="button" className={destructive ? 'btn btn-danger' : 'btn btn-primary'} onClick={onConfirm}>{confirmLabel}
</button>
      </div>
    </div>
  );
  return inline ? box : <div className="dialog-scrim" onClick={(e) => { if (e.target === e.currentTarget && onCancel)
onCancel(); }}>{box}</div>;
}
```

components/surfaces/NextSteps.jsx · 20 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
/** Ordered list of next safe actions, each with one line of why. */
export function NextSteps({ steps = [] }) {
```

```
  return (
    <ul className="next-steps">
      {steps.map((s) => (
        <li key={s.label}>
          <button type="button" className="next-step" onClick={s.onClick}>
            {s.icon && <Icon name={s.icon} size={19} style={{ color: 'var(--text-secondary)' }} />}
            <span className="next-step-text"><span className="next-step-label">{s.label}</span>{s.why && <span className="next-
step-why">{s.why}</span>}</span>
            {s.aside}
            <Icon name="chevronRight" size={17} style={{ color: 'var(--text-tertiary)' }} />
          </button>
        </li>
      ))}
    </ul>
  );
}
```

components/surfaces/ObjectCard.jsx · 28 lines

```
import React, { useId } from 'react';
import { STATUSES } from '../trust/vocabulary.js';
import { ProvenanceChips } from '../trust/ProvenanceChips.jsx';
/**
 * The universal object card — one rhythm for every kind of thing:
 * eyebrow (kind · status) → title → provenance → why it matters → metadata → one primary + one quiet secondary.
 */
export function ObjectCard({ kind = 'Task', statuses = [], title, provenance, explanation, metadata, primary, secondary, tone,
level = 3, children }) {
  const hid = useId(); const H = 'h' + level;
  const glyphs = statuses.map((s) => (STATUSES[s] || {}).glyph).filter(Boolean).join(' ');
  return (
    <article className="object-card" data-tone={tone} aria-labelledby={hid}>
      <div className="object-card-eyebrow">{glyphs && <span className="status-glyph" aria-hidden="true">{glyphs}</span>}<span>
{[kind, ...statuses.map((s) => (STATUSES[s] || { label: s }).label)].join(' · ')}</span></div>
      <H id={hid} className="object-card-title">{title}</H>
      {provenance && <div><ProvenanceChips {...provenance} /></div>}
      {explanation && <p className="object-card-why">{explanation}</p>}
      {metadata && <p className="object-card-meta">{metadata}</p>}
      {children}
      {(primary || secondary) && (
        <div className="object-card-actions">
          {primary && <button type="button" className="btn btn-primary" onClick={primary.onClick}>{primary.label}</button>}
          {secondary && <button type="button" className="link-quiet" onClick={secondary.onClick}>{secondary.label}</button>}
        </div>
      )}
    </article>
  );
}
```

components/surfaces/OpenIn.jsx · 12 lines

```
import React from 'react';
/** "Open in" — the way from one thing to every place it matters. Words, never icons alone; wraps on a phone. */
export function OpenIn({ targets = [], about, onOpen }) {
  if (!targets.length) return null;
  return (
    <div className="open-in" role="group" aria-label={'Open ' + about + ' in'}>
      <span className="open-in-label" aria-hidden="true">Open in</span>
      {targets.map((t) => <button key={t} type="button" className="pill-soft" onClick={() => onOpen && onOpen(t)}>{t}
</button>)}
    </div>
  );
}
```

components/surfaces/QuickActions.jsx · 7 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
/** A row of 2–5 labelled starting actions ("Add a syllabus", "Connect a calendar"). Never icon-only. */
export function QuickActions({ actions = [] }) {
  return <div className="quick-actions">{actions.map((a) => <button key={a.label} type="button" className="quick-action"
```

```
onClick={a.onClick}>{a.icon && <Icon name={a.icon} size={18} style={{ color: 'var(--text-secondary)' }} />}{a.label}</button>)}
</div>;
}
```

components/surfaces/ScreenGuide.jsx · 11 lines

```
import React from 'react';
/** "About this screen" — a quiet disclosure at the foot of every screen: what it's for, where the data comes from, who to ask.
*/
export function ScreenGuide({ items = [], open = false }) {
  return (
    <details className="screen-guide" open={open}>
      <summary><span aria-hidden="true">ⓘ</span>About this screen</summary>
      <dl>{items.map((it) => <React.Fragment key={it.q}><dt>{it.q}</dt><dd>{it.a}</dd></React.Fragment>)}</dl>
    </details>
  );
}
```

components/trust/AIResponse.jsx · 31 lines

```
import React from 'react';
import { SourceBadge } from './SourceBadge.jsx';
/**
 * A governed assistant answer: always labelled AI-assisted, shows the data scope it was allowed to read,
 * numbered sources, stated uncertainty, request cost, and the actions that need human confirmation.
 */
export function AIResponse({ children, scope = [], sources = [], confidence, cost, policy, actions, model = 'Semester
assistant' }) {
  return (
    <article className="ai-frame" aria-label="AI-assisted answer">
      <div className="ai-head">
        <SourceBadge source="ai" updated={model} />
        {policy && <span className="status-chip" data-tone="pending"><span className="status-glyph" aria-hidden="true">§</span>
{policy}</span>}
      </div>
      <div className="ai-body">{children}</div>
      {sources.length > 0 && (
        <ol className="ai-sources" aria-label="Sources">
          {sources.map((s, i) => (
            <li key={i}><span className="ai-cite">{i + 1}</span><span style={{ color: 'var(--text-primary)' }}>{s.title}</span>
{s.kind && <SourceBadge source={s.kind} />}</li>
          ))}
        </ol>
      )}
      <div className="ai-meta">
        {scope.length > 0 && <span>Allowed to read <b>{scope.join(', ')}</b></span>}
        {confidence && <span>Confidence <b>{confidence}</b></span>}
        {cost && <span>Request cost <b className="nums">{cost}</b></span>}
      </div>
      {actions && <div className="state-actions">{actions}</div>}
    </article>
  );
}
```

components/trust/ProvenanceChips.jsx · 19 lines

```
import React from 'react';
import { SOURCES, STATUSES } from './vocabulary.js';
/** Origin chip, then at most two cues, then age. Read to assistive tech as one sentence. */
export function ProvenanceChips({ origin = 'official', by, cues = [], age }) {
  const o = SOURCES[origin] || SOURCES.external;
  const shown = cues.slice(0, 2).map((c) => ({ key: c, ...(STATUSES[c] || { label: c, glyph: '·', tone: 'neutral' }) }));
  const sentence = [o.label + (by ? ' by ' + by : ''), ...shown.map((c) => c.label.toLowerCase()), age].filter(Boolean).join(',
');
  return (
    <span className="prov" data-origin={origin} title={o.meaning}>
      <span className="prov" aria-hidden="true">
        <span className="status-chip" data-tone={o.tone} data-fill={o.fill}><span className="status-glyph">{o.glyph}</span>
{o.short}{by ? ' · ' + by : ''}</span>
        {shown.map((c) => <span key={c.key} className="status-chip" data-tone={c.tone}><span className="status-glyph">{c.glyph}
</span>{c.label}</span>)}
```

```
        {age && <span className="prov-age">{age}</span>}
      </span>
      <span className="sr-only">{sentence}</span>
    </span>
  );
}
```

components/trust/SourceBadge.jsx · 15 lines

```
import React from 'react';
import { SOURCES } from './vocabulary.js';
/** Where a fact came from and how old it is, beside the fact. Text first — never icon or colour alone. */
export function SourceBadge({ source = 'official', updated, by, onReport }) {
  const s = SOURCES[source] || SOURCES.external;
  return (
    <span className="source-badge" data-source={source} title={s.meaning}>
      <span className="status-chip" data-tone={s.tone} data-fill={s.fill}><span className="status-glyph" aria-hidden="true">
{s.glyph}</span>{s.label}{by ? ' · ' + by : ''}</span>
      <span className="sr-only">{s.meaning}</span>
      {updated && <span>{updated}</span>}
      {onReport && <button type="button" className="source-report" onClick={onReport}>Report incorrect information</button>}
    </span>
  );
}
```

components/trust/SourceLine.jsx · 17 lines

```
import React from 'react';
import { SOURCES } from './vocabulary.js';
/** The signature device: a thin editorial line from a statement to its source chip. Only where a real source exists. */
export function SourceLine({ children, source = 'official', by, when }) {
  const s = SOURCES[source] || SOURCES.external;
  return (
    <div style={{ display: 'grid', gap: 'var(--sp-4)' }}>
      <div>{children}</div>
      <div style={{ display: 'flex', alignItems: 'center', gap: 'var(--sp-4)' }} title={s.meaning}>
        <span aria-hidden="true" style={{ flex: '0 0 48px', height: 1, background: 'var(--accent-editorial)' }}></span>
        <span aria-hidden="true" style={{ width: 5, height: 5, borderRadius: '50%', background: 'var(--accent-editorial)',
marginLeft: -6 }}></span>
        <span className="status-chip" data-tone={s.tone} data-fill={s.fill}><span className="status-glyph" aria-hidden="true">
{s.glyph}</span>{[s.short, by, when].filter(Boolean).join(' · ')}</span>
      </div>
    </div>
  );
}
```

components/trust/StatusChip.jsx · 8 lines

```
import React from 'react';
import { STATUSES } from './vocabulary.js';
/** One status, drawn one way: glyph + word + tone. */
export function StatusChip({ status = 'pending', label }) {
  const s = STATUSES[status] || { label: status, glyph: '·', tone: 'neutral' };
  return <span className="status-chip" data-tone={s.tone}><span className="status-glyph" aria-hidden="true">{s.glyph}</span>
{label || s.label}</span>;
}
```

components/trust/vocabulary.js · 42 lines

```
// The one trust + status vocabulary. Ported from semester app/src/lib/source.ts and lib/status.ts.
// Every state = glyph + word + tone. Tone is the least of the three.
export const SOURCES = {
  official:  { label: 'Institution verified', short: 'Official', glyph: '◆', tone: 'official', fill: 'solid', meaning:
'Confirmed by your institution's own system.' },
  connected: { label: 'Connected', short: 'Connected', glyph: '⇄', tone: 'connected', fill: 'outline', meaning: 'Synced from a
connected account or feed. Semester has not checked it with your institution.' },
  imported:  { label: 'Imported', short: 'Imported', glyph: '↓', tone: 'connected', fill: 'outline', meaning: 'Copied in from a
file or feed. Semester has not checked it with your institution.' },
  personal:  { label: 'Student entered', short: 'Yours', glyph: '○', tone: 'personal', fill: 'outline', meaning: 'You typed
this in. Semester has not checked it with your institution.' },
```

```
  made:      { label: 'Made here', short: 'Made here', glyph: '○', tone: 'personal', fill: 'outline', meaning: 'Semester built
this from sources you gave it — a rule-based extraction, not an AI draft.' },
  ai:        { label: 'AI-assisted', short: 'AI-assisted', glyph: '✦', tone: 'ai', fill: 'outline', meaning: 'Written with
Semester's assistant. It is not an official answer — check anything you act on.' },
  estimated: { label: 'Estimated', short: 'Estimated', glyph: '≈', tone: 'pending', fill: 'outline', meaning: 'Worked out by
Semester from what it knows. Not an official figure.' },
  review:    { label: 'Needs review', short: 'Review', glyph: '?', tone: 'pending', fill: 'outline', meaning: 'Something about
this looks wrong or out of date. Check it before relying on it.' },
  stale:     { label: 'Unavailable or stale', short: 'Stale', glyph: '!', tone: 'danger', fill: 'outline', meaning: 'Semester
cannot confirm that this information is current.' },
  external:  { label: 'External', short: 'External', glyph: '↗', tone: 'neutral', fill: 'outline', meaning: 'From a source
outside your institution and outside Semester, such as a web page.' },
  sample:    { label: 'Sample', short: 'Sample', glyph: '◌', tone: 'neutral', fill: 'dashed', meaning: 'Illustrative data. Not
live.' },
};
export const STATUSES = {
  saving: { label: 'Saving…', glyph: '…', tone: 'neutral' },
  saved: { label: 'Saved', glyph: '✓', tone: 'success' },
  syncing: { label: 'Syncing', glyph: '↻', tone: 'neutral' },
  synced: { label: 'Synced', glyph: '✓', tone: 'success' },
  offline: { label: 'Offline', glyph: '⊘', tone: 'pending', urgent: true },
  queued: { label: 'Queued', glyph: '⋯', tone: 'neutral' },
  conflict: { label: 'Conflict needs review', glyph: '⇄', tone: 'danger', urgent: true },
  'sync-error': { label: 'Sync trouble', glyph: '!', tone: 'danger', urgent: true },
  'needs-confirmation': { label: 'Needs confirmation', glyph: '?', tone: 'pending' },
  'action-required': { label: 'Action required', glyph: '!', tone: 'pending' },
  pending: { label: 'Pending approval', glyph: '◷', tone: 'pending' },
  approved: { label: 'Approved', glyph: '✓', tone: 'success' },
  rejected: { label: 'Rejected', glyph: '×', tone: 'danger' },
  verified: { label: 'Verified', glyph: '✓', tone: 'success' },
  hold: { label: 'Hold on account', glyph: '!', tone: 'danger' },
  'faculty-approved': { label: 'Faculty-approved', glyph: '◆', tone: 'success' },
  'course-provided': { label: 'Course-provided', glyph: '◇', tone: 'official' },
  'updated-today': { label: 'Updated today', glyph: '↻', tone: 'neutral' },
  draft: { label: 'Draft', glyph: '○', tone: 'neutral' },
  live: { label: 'Live', glyph: '●', tone: 'success' },
  beta: { label: 'Preview', glyph: '◌', tone: 'neutral' },
  'device-only': { label: 'On this device only', glyph: '○', tone: 'neutral' },
  'signed-out': { label: 'Not signed in', glyph: '○', tone: 'neutral' },
  unknown: { label: 'Status unknown', glyph: '·', tone: 'neutral' },
};
```

components/core/markPaths.js · 4 lines

```
// Generated from semester app/src/components/mark.data.ts — three slabs on a 512 grid.
export const MARK_VIEWBOX = '128 65 256 382';
export const MARK_PATHS = ["M128 176L204 208L204 420L128 388Z","M218 65L294 97L294 404L218 372Z","M308 147L384 179L384 447L308
415Z"];
```

SEMESTER-CORE PACKAGE

Access, AI gateway, approvals, consequence, freshness, next action, pilot, provenance and scoring logic, with tests.

handoff/semester-core/access.ts · 35 lines

```
export type Role = 'student' | 'applicant' | 'guardian' | 'alumni' | 'faculty' | 'ta' | 'advisor' | 'registrar' | 'finance' |
'affairs' | 'security' | 'support' | 'employer' | 'partner';
export type Level = 'full' | 'scoped' | 'shared' | 'read' | 'audited' | 'none';
export interface Share { grantee: string; screen: string; expiresAt: number; revoked?: boolean }
export interface Grant { kind: 'section' | 'ticket' | 'listing' | 'breakglass'; id: string; expiresAt: number; approvedBy?:
string[] }
export interface Ctx { role: Role; userId: string; ownerId?: string; shares?: Share[]; grants?: Grant[]; now: number }
/** Default matrix; anything missing is none. Students own their own rows. */
export const MATRIX: Partial<Record<Role, Record<string, Level>>> = {
  guardian: { family: 'shared', billing: 'shared', grades: 'shared', calendar: 'shared' },
  advisor: { degree: 'shared', advising: 'shared', registration: 'read' },
  faculty: { course: 'scoped', grades: 'scoped', announce: 'scoped' },
  ta: { course: 'scoped', grades: 'scoped' },
  registrar: { registration: 'full', degree: 'full', grades: 'audited' },
  finance: { billing: 'full', costs: 'full' },
  affairs: { support: 'shared', housing: 'scoped', meals: 'scoped' },
  security: { privacy: 'audited', account: 'audited', connect: 'audited' },
  support: { support: 'scoped', connect: 'scoped' },
  employer: { opportunities: 'scoped', career: 'shared' },
  partner: { opportunities: 'scoped' },
};
export function level(role: Role, screen: string): Level {
  if (role === 'student') return 'full';
  return MATRIX[role]?.[screen] ?? 'none';
}
/** The runtime decision: the matrix says what kind of access, the context proves it right now. Mirrors the RLS policy. */
export function canRead(screen: string, c: Ctx): { ok: boolean; reason: string } {
  const l = level(c.role, screen);
  if (c.role === 'student') return c.ownerId == null || c.ownerId === c.userId ? { ok: true, reason: 'owner' } : { ok: false,
reason: 'not the owner' };
  if (l === 'none') return { ok: false, reason: 'role has no access' };
  if (l === 'shared') { const s = (c.shares ?? []).find((x) => x.grantee === c.userId && x.screen === screen && !x.revoked &&
x.expiresAt > c.now); return s ? { ok: true, reason: 'active share' } : { ok: false, reason: 'no active share from the student'
}; }
  if (l === 'scoped') { const g = (c.grants ?? []).find((x) => x.kind !== 'breakglass' && x.expiresAt > c.now); return g ? {
ok: true, reason: g.kind + ' membership' } : { ok: false, reason: 'not a member of this scope' }; }
  if (l === 'audited') { const g = (c.grants ?? []).find((x) => x.kind === 'breakglass' && x.expiresAt > c.now && new
Set(x.approvedBy).size >= 2 && !x.approvedBy!.includes(c.userId)); return g ? { ok: true, reason: 'break-glass, 2 approvers,
logged' } : { ok: false, reason: 'needs break-glass with two other approvers' }; }
  return { ok: true, reason: l };
}
export const canWrite = (screen: string, c: Ctx) => { const l = level(c.role, screen); return (c.role === 'student' &&
canRead(screen, c).ok) || ((l === 'full' || l === 'scoped') && canRead(screen, c).ok); };
```

handoff/semester-core/aigateway.ts · 16 lines

```
export type DataClass = 'public' | 'course' | 'personal' | 'student-record';
export interface Policy { courseAllows: ('outline' | 'feedback' | 'practice' | 'write' | 'none')[]; institutionAiOn: boolean }
export interface Req { task: 'outline' | 'feedback' | 'practice' | 'write' | 'plan' | 'search'; scope: { id: string; cls:
DataClass }[]; policy: Policy; budgetLeft: number }
export interface Decision { allowed: boolean; reason: string; sendScope: string[]; dropped: string[]; label: 'AI-assisted' }
/** Runs before every model call. Student records are never sent, whatever the scope says. */
export function gate(r: Req): Decision {
  const base = { label: 'AI-assisted' as const };
  if (!r.policy.institutionAiOn) return { ...base, allowed: false, reason: 'Your institution has turned AI off.', sendScope:
[], dropped: r.scope.map((s) => s.id) };
  if (r.budgetLeft <= 0) return { ...base, allowed: false, reason: 'No requests left this week.', sendScope: [], dropped:
r.scope.map((s) => s.id) };
  const courseTask = ['outline', 'feedback', 'practice', 'write'].includes(r.task);
  if (courseTask && !r.policy.courseAllows.includes(r.task as never)) return { ...base, allowed: false, reason: 'Your
instructor doesn't allow AI for ' + r.task + ' in this course.', sendScope: [], dropped: r.scope.map((s) => s.id) };
```

```
  const sendScope = r.scope.filter((s) => s.cls !== 'student-record').map((s) => s.id);
  const dropped = r.scope.filter((s) => s.cls === 'student-record').map((s) => s.id);
  return { ...base, allowed: true, reason: dropped.length ? 'Student records were left out.' : 'Allowed.', sendScope, dropped
};
}
```

handoff/semester-core/approvals.ts · 17 lines

```
export type St = 'requested' | 'approved-1' | 'approved' | 'rejected' | 'expired' | 'executed';
export interface Approval { id: string; requester: string; approvers: string[]; state: St; expiresAt: number; mfaAt:
Record<string, number> }
export type Ev = { type: 'approve' | 'reject' | 'execute'; by: string; at: number };
const MFA_MS = 15 * 60e3;
/** Two-person duty: two distinct approvers, neither the requester, each with MFA in the last 15 min; execute only by the
requester after full approval. */
export function step(a: Approval, e: Ev): Approval | { error: string } {
  if (e.at > a.expiresAt && a.state !== 'executed') return { ...a, state: 'expired' };
  if (e.type === 'execute') return a.state === 'approved' && e.by === a.requester ? { ...a, state: 'executed' } : { error:
'Needs two approvals; only the requester runs it.' };
  if (e.by === a.requester) return { error: 'You can’t approve your own request.' };
  if (a.approvers.includes(e.by)) return { error: 'You already decided this one.' };
  if (!a.mfaAt[e.by] || e.at - a.mfaAt[e.by] > MFA_MS) return { error: 'Verify a second factor first (within 15 minutes).' };
  if (!['requested', 'approved-1'].includes(a.state)) return { error: 'This request is ' + a.state + '.' };
  if (e.type === 'reject') return { ...a, state: 'rejected', approvers: [...a.approvers, e.by] };
  const approvers = [...a.approvers, e.by];
  return { ...a, approvers, state: approvers.length >= 2 ? 'approved' : 'approved-1' };
}
```

handoff/semester-core/consequence.ts · 11 lines

```
export interface Action { reversible: boolean; leavesSemester?: boolean; record?: boolean; money?: number; institutional?:
boolean; affectsOthers?: boolean }
export type Pattern = 'undo' | 'preview' | 'review-confirm' | 'two-person';
/** The one rule for how hard an action is to take. */
export function pattern(a: Action): Pattern {
  if (a.institutional) return 'two-person';
  if (a.record || (a.money ?? 0) > 0) return 'review-confirm';
  if (a.leavesSemester || a.affectsOthers || !a.reversible) return 'preview';
  return 'undo';
}
export const RECOVERY = { undo: 'You can undo this.', request: 'This can be reversed on request.', none: 'This can’t be
undone.' } as const;
```

handoff/semester-core/core.test.ts · 69 lines

```
import { describe, it, expect } from 'vitest';
import { resolve, effectiveOrigin, ageLabel } from './provenance';
import { order } from './nextaction';
import { canRead, level } from './access';
import { state, optimistic, POLICY } from './freshness';
import { pattern } from './consequence';
import { gate } from './aigateway';
import { step, Approval } from './approvals';
import { weighted } from './scoring';
import { readiness } from './pilot';
const NOW = Date.UTC(2026, 9, 7, 12); const H = 3600e3;

describe('provenance', () => {
  it('official beats personal; stale official loses to personal', () => {
    expect(resolve([{ value: 'Thu', origin: 'personal', updatedAt: NOW }, { value: 'Wed', origin: 'official', updatedAt: NOW -
H }], NOW)!.shown.value).toBe('Wed');
    expect(effectiveOrigin({ value: 1, origin: 'connected', updatedAt: NOW - 7 * H }, NOW)).toBe('stale');
    const r = resolve([{ value: 'Thu', origin: 'personal', updatedAt: NOW }, { value: 'Wed', origin: 'connected', updatedAt:
NOW - 7 * H }], NOW)!;
    expect(r.shown.value).toBe('Thu'); expect(r.conflict).toBe(true);
  });
  it('never invents a time', () => { expect(ageLabel(undefined, NOW)).toBe('Update time not recorded'); expect(ageLabel(NOW - 3
* 86400e3, NOW)).toBe('Updated 3 days ago'); });
});
describe('next action', () => {
  it('a blocking hold outranks a low-weight task due sooner', () => {
```

```
    const r = order([{ id: 't', title: 'Reading', kind: 'task', origin: 'official', dueAt: NOW + 20 * H, weight: 0.02 }, { id:
'h', title: 'Hold', kind: 'hold', origin: 'official', dueAt: NOW + 5 * 24 * H, blocks: ['registration'] }], NOW);
    expect(r[0].id).toBe('h'); expect(r[0].reasons).toContain('Blocks registration');
  });
  it('drops done and snoozed', () => { expect(order([{ id: 'a', title: '', kind: 'task', origin: 'personal', done: true }, {
id: 'b', title: '', kind: 'task', origin: 'personal', snoozedUntil: NOW + H }], NOW)).toEqual([]); });
});
describe('access', () => {
  it('guardian needs an unexpired, unrevoked share', () => {
    const c = { role: 'guardian' as const, userId: 'g', now: NOW };
    expect(canRead('grades', c).ok).toBe(false);
    expect(canRead('grades', { ...c, shares: [{ grantee: 'g', screen: 'grades', expiresAt: NOW + H }] }).ok).toBe(true);
    expect(canRead('grades', { ...c, shares: [{ grantee: 'g', screen: 'grades', expiresAt: NOW - 1 }] }).ok).toBe(false);
  });
  it('break-glass needs two other approvers', () => {
    const c = { role: 'security' as const, userId: 's', now: NOW };
    expect(canRead('privacy', { ...c, grants: [{ kind: 'breakglass' as const, id: 'b', expiresAt: NOW + H, approvedBy: ['s',
'x'] }] }).ok).toBe(false);
    expect(canRead('privacy', { ...c, grants: [{ kind: 'breakglass' as const, id: 'b', expiresAt: NOW + H, approvedBy: ['x',
'y'] }] }).ok).toBe(true);
  });
  it('unknown role/screen is none', () => { expect(level('partner', 'grades')).toBe('none'); });
});
describe('freshness', () => {
  it('maps lag to policy', () => {
    expect(state({ lastEventAt: NOW, appliedAt: NOW - 3000 }, NOW)).toBe('live');
    expect(POLICY[state({ lastEventAt: NOW, appliedAt: NOW - 120e3 }, NOW)].destructive).toBe('confirm-at-source');
    expect(POLICY[state({ lastEventAt: NOW, workerFailed: true }, NOW)].destructive).toBe('disabled');
  });
  it('optimistic confirms on version, rolls back on timeout', () => { expect(optimistic(5, 5, NOW, NOW)).toBe('confirmed');
expect(optimistic(5, 4, NOW - 20e3, NOW)).toBe('rolled-back'); });
});
describe('consequence', () => { it('picks the pattern', () => { expect(pattern({ reversible: true })).toBe('undo');
expect(pattern({ reversible: true, leavesSemester: true })).toBe('preview'); expect(pattern({ reversible: false, record: true
})).toBe('review-confirm'); expect(pattern({ reversible: false, institutional: true })).toBe('two-person'); }); });
describe('AI gateway', () => {
  const policy = { courseAllows: ['outline' as const], institutionAiOn: true };
  it('never sends student records', () => { const d = gate({ task: 'outline', scope: [{ id: 'syl', cls: 'course' }, { id: 'tx',
cls: 'student-record' }], policy, budgetLeft: 3 }); expect(d.allowed).toBe(true); expect(d.sendScope).toEqual(['syl']);
expect(d.dropped).toEqual(['tx']); });
  it('enforces course policy', () => { expect(gate({ task: 'write', scope: [], policy, budgetLeft: 3 }).allowed).toBe(false);
});
});
describe('approvals', () => {
  const a: Approval = { id: '1', requester: 'r', approvers: [], state: 'requested', expiresAt: NOW + H, mfaAt: { x: NOW, y:
NOW, r: NOW } };
  it('two distinct non-requesters with fresh MFA', () => {
    expect('error' in step(a, { type: 'approve', by: 'r', at: NOW })).toBe(true);
    const one = step(a, { type: 'approve', by: 'x', at: NOW }) as Approval; expect(one.state).toBe('approved-1');
    expect('error' in step(one, { type: 'approve', by: 'x', at: NOW })).toBe(true);
    const two = step(one, { type: 'approve', by: 'y', at: NOW }) as Approval; expect(two.state).toBe('approved');
    expect((step(two, { type: 'execute', by: 'r', at: NOW }) as Approval).state).toBe('executed');
  });
});
describe('scoring', () => { it('renormalises over graded criteria', () => { expect(weighted({ correctness: 5, codeQuality: 5
}).of5).toBe(5); expect(weighted({}).of5).toBeNull(); expect(weighted({ correctness: 4 }).missing).toContain('rendering'); });
});
describe('pilot', () => { it('ready only when every P0 is done', () => { expect(readiness([{ priority: 'P0', status: 'Done',
gate: 'x' }, { priority: 'P0', status: 'Not started', gate: 'Pilot contract' }]).blockers.length).toBe(1); }); });
```

handoff/semester-core/freshness.ts · 22 lines

```
export type State = 'live' | 'recent' | 'delayed' | 'failed' | 'rebuilding';
export interface Projection { lastEventAt: number; appliedAt?: number; workerFailed?: boolean; rebuilding?: boolean }
export function state(p: Projection, now: number): State {
  if (p.rebuilding) return 'rebuilding';
  if (p.workerFailed) return 'failed';
  const lag = (now - (p.appliedAt ?? 0)) / 1000;
  return lag <= 5 ? 'live' : lag <= 60 ? 'recent' : 'delayed';
}
export interface Policy { display: string; safe: boolean; destructive: 'allowed' | 'confirm-at-source' | 'disabled'; badge?:
string }
export const POLICY: Record<State, Policy> = {
  live: { display: 'normal', safe: true, destructive: 'allowed' },
  recent: { display: 'normal', safe: true, destructive: 'allowed' },
```

```
  delayed: { display: 'normal', safe: true, destructive: 'confirm-at-source', badge: 'Delayed' },
  failed: { display: 'last-known', safe: false, destructive: 'disabled', badge: 'Not updating' },
  rebuilding: { display: 'versioned', safe: true, destructive: 'disabled', badge: 'Rebuilding' },
};
/** Optimistic mutation: pending until the projection version reaches the command's version; rollback after timeout. */
export function optimistic(sentVersion: number, seenVersion: number, sentAt: number, now: number, timeoutMs = 15000): 'pending'
| 'confirmed' | 'rolled-back' {
  if (seenVersion >= sentVersion) return 'confirmed';
  return now - sentAt > timeoutMs ? 'rolled-back' : 'pending';
}
```

handoff/semester-core/nextaction.ts · 29 lines

```
import { Origin, rank } from './provenance';
export interface Item { id: string; title: string; dueAt?: number; weight?: number; effortMin?: number; origin: Origin;
blocks?: string[]; kind: 'assignment' | 'hold' | 'bill' | 'task' | 'event' | 'decision'; done?: boolean; snoozedUntil?: number
}
export interface Why { score: number; reasons: string[] }
const HOUR = 3600e3;
/**
 * Score = urgency × consequence × confidence − effort penalty.
 * urgency: 1 at ≥14 days, rising to 10 inside 24 h; overdue = 12.
 * consequence: grade weight (0–1 → 1–4) or blocking (holds, bills that block registration) = 5.
 * confidence: official 1.0 → stale 0.6; AI/estimated never outrank an official item of equal urgency.
 */
export function score(it: Item, now: number): Why {
  const reasons: string[] = [];
  const hrs = it.dueAt == null ? 24 * 14 : (it.dueAt - now) / HOUR;
  const urgency = hrs < 0 ? 12 : hrs < 24 ? 10 : hrs < 72 ? 6 : hrs < 168 ? 3 : hrs < 336 ? 1.5 : 1;
  if (hrs < 0) reasons.push('Overdue'); else if (hrs < 24) reasons.push('Due within 24 hours'); else if (hrs < 72)
reasons.push('Due in ' + Math.ceil(hrs / 24) + ' days');
  let consequence = 1 + 3 * Math.min(1, it.weight ?? 0);
  if (it.weight) reasons.push(Math.round(it.weight * 100) + '% of your grade');
  if (it.blocks?.length) { consequence = 5; reasons.push('Blocks ' + it.blocks.join(', ')); }
  const confidence = 1 - Math.min(0.4, rank(it.origin) * 0.08);
  const effort = Math.min(1.5, (it.effortMin ?? 30) / 120);
  return { score: +(urgency * consequence * confidence - effort).toFixed(3), reasons };
}
export function order(items: Item[], now: number, limit = 5): (Item & Why)[] {
  return items.filter((i) => !i.done && !(i.snoozedUntil && i.snoozedUntil > now))
    .map((i) => ({ ...i, ...score(i, now) }))
    .sort((a, b) => b.score - a.score || (a.dueAt ?? Infinity) - (b.dueAt ?? Infinity) || a.id.localeCompare(b.id))
    .slice(0, limit);
}
```

handoff/semester-core/pilot.ts · 7 lines

```
export interface Row { priority: 'P0' | 'P1' | 'P2'; status: string; gate: string }
/** Launch-ready only when every P0 is Done; reports what blocks real student data. */
export function readiness(rows: Row[]) {
  const p0 = rows.filter((r) => r.priority === 'P0'); const open = p0.filter((r) => r.status !== 'Done');
  return { ready: open.length === 0, p0Total: p0.length, p0Open: open.length, pct: p0.length ? Math.round(((p0.length -
open.length) / p0.length) * 100) : 100, blockers: open.filter((r) => /pilot|student|contract/i.test(r.gate)) };
}
```

handoff/semester-core/provenance.ts · 39 lines

```
export type Origin = 'official' | 'connected' | 'made' | 'personal' | 'sample' | 'stale' | 'ai' | 'estimated' | 'external';
/** Strongest first. stale ranks below personal on purpose: an unchecked copy that may have moved is worth less than a guess
that knows it is one. */
export const LADDER: Origin[] = ['official', 'connected', 'made', 'personal', 'sample', 'stale'];
const DISPLAY_ONLY: Origin[] = ['ai', 'estimated', 'external'];
export const STALE_AFTER_MS: Partial<Record<Origin, number>> = { official: 24 * 3600e3, connected: 6 * 3600e3 };

export interface Fact<T> { value: T; origin: Origin; updatedAt?: number; by?: string }

export function rank(o: Origin): number {
  const i = LADDER.indexOf(o);
  return i >= 0 ? i : LADDER.length + DISPLAY_ONLY.indexOf(o);
}
/** A synced fact past its window is demoted to stale before ranking. */
```

```
export function effectiveOrigin(f: Fact<unknown>, now: number): Origin {
  const win = STALE_AFTER_MS[f.origin];
  if (win && f.updatedAt != null && now - f.updatedAt > win) return 'stale';
  if (win && f.updatedAt == null) return 'stale';
  return f.origin;
}
/** Pick the fact to show; ties go to the newest. Returns the conflict so the UI can say "2 sources disagree". */
export function resolve<T>(facts: Fact<T>[], now: number): { shown: Fact<T> & { effective: Origin }; conflict: boolean } | null
{
  if (!facts.length) return null;
  const scored = facts.map((f) => ({ ...f, effective: effectiveOrigin(f, now) }))
    .sort((a, b) => rank(a.effective) - rank(b.effective) || (b.updatedAt ?? 0) - (a.updatedAt ?? 0));
  const shown = scored[0];
  const conflict = scored.some((f) => JSON.stringify(f.value) !== JSON.stringify(shown.value));
  return { shown, conflict };
}
export function ageLabel(updatedAt: number | undefined, now: number): string {
  if (updatedAt == null) return 'Update time not recorded';
  const m = Math.round((now - updatedAt) / 60000);
  if (m < 1) return 'Updated less than a minute ago';
  if (m < 60) return 'Updated ' + m + ' min ago';
  const h = Math.round(m / 60);
  if (h < 24) return 'Updated ' + h + ' h ago';
  const d = Math.round(h / 24);
  return 'Updated ' + d + (d === 1 ? ' day ago' : ' days ago');
}
```

handoff/semester-core/scoring.ts · 11 lines

```
export type Crit = 'correctness' | 'codeQuality' | 'rendering' | 'state' | 'maintainability' | 'handoff' | 'sandboxFit' |
'persistence';
export const DEFAULT_W: Partial<Record<Crit, number>> = { correctness: .25, codeQuality: .2, rendering: .15, state: .15,
maintainability: .1, handoff: .1, sandboxFit: .05 };
/** Weighted 0–5; weights renormalised over the criteria actually graded so a missing score never counts as zero. */
export function weighted(g: Partial<Record<Crit, number>>, w = DEFAULT_W): { of5: number | null; of100: number | null; missing:
Crit[] } {
  const keys = Object.keys(w) as Crit[]; const missing = keys.filter((k) => g[k] == null);
  const have = keys.filter((k) => g[k] != null); const tw = have.reduce((s, k) => s + (w[k] ?? 0), 0);
  if (!tw) return { of5: null, of100: null, missing };
  const v = have.reduce((s, k) => s + (g[k]! * (w[k] ?? 0)), 0) / tw;
  return { of5: +v.toFixed(2), of100: Math.round(v * 20), missing };
}
```

SEMESTER-PLATFORM: API, SECURITY, MIGRATIONS AND TESTS

API routes, input validation, AI redaction, security middleware, every database migration, seed data and the test suites.

handoff/semester-platform/app/api/ai/route.ts · 28 lines

```
// POST /api/ai — the only path to a model. Order: auth → policy → gate() → scan() → provider → audit + spend → labelled
answer.
import { NextResponse } from 'next/server';
import { z } from 'zod';
import { createServerClient } from '@/lib/supabase/server';
import { gate } from '@/lib/core/aigateway';
import { scan } from '@/lib/ai/redact';
const Body = z.object({ task: z.enum(['outline', 'feedback', 'practice', 'write', 'plan', 'search']), prompt:
z.string().min(1).max(4000), courseId: z.string().uuid().optional(), scope: z.array(z.object({ id: z.string(), cls:
z.enum(['public', 'course', 'personal', 'student-record']) })).max(20).default([]) });
export async function POST(req: Request) {
  const sb = await createServerClient();
  const { data: { user } } = await sb.auth.getUser();
  if (!user) return NextResponse.json({ error: 'Sign in required' }, { status: 401 });
  const parsed = Body.safeParse(await req.json().catch(() => null));
  if (!parsed.success) return NextResponse.json({ error: 'Invalid request' }, { status: 400 });
  const b = parsed.data;
  const { data: pol } = await sb.schema('api').rpc('ai_policy', { p_course: b.courseId ?? null }); // { institutionAiOn,
courseAllows, budgetLeft, studentIdPattern, provider, model }
  if (!pol) return NextResponse.json({ error: 'AI policy unavailable' }, { status: 503 });
  const d = gate({ task: b.task, scope: b.scope, policy: { institutionAiOn: pol.institutionAiOn, courseAllows: pol.courseAllows
}, budgetLeft: pol.budgetLeft });
  if (!d.allowed) { await sb.schema('api').rpc('ai_log', { p_task: b.task, p_outcome: 'refused', p_reason: d.reason, p_tokens:
0 }); return NextResponse.json({ allowed: false, reason: d.reason, label: d.label }); }
  const s = scan(b.prompt, { studentId: pol.studentIdPattern ? new RegExp(pol.studentIdPattern) : undefined });
  if (s.block) { await sb.schema('api').rpc('ai_log', { p_task: b.task, p_outcome: 'blocked', p_reason: s.findings.map((f) =>
f.kind).join(','), p_tokens: 0 }); return NextResponse.json({ allowed: false, reason: 'That looks like a student record (' +
[...new Set(s.findings.map((f) => f.kind))].join(', ') + '). Ask without it, or contact the office that holds it.', label:
d.label }); }
  const ctx = await sb.schema('api').rpc('ai_context', { p_ids: d.sendScope }); // approved sources only, with citations
  const res = await fetch(process.env.AI_PROVIDER_URL!, { method: 'POST', headers: { 'content-type': 'application/json',
authorization: 'Bearer ' + process.env.AI_PROVIDER_KEY }, body: JSON.stringify({ model: pol.model, max_tokens: 800, system:
'You help students using only the provided sources. Cite them as [n]. Say when you are unsure and suggest the official office.
Never state institutional rules as decided; the institution decides.', messages: [{ role: 'user', content: s.text +
'\n\nSources:\n' + JSON.stringify(ctx.data ?? []) }] }), signal: AbortSignal.timeout(20000) }).catch(() => null);
  if (!res || !res.ok) { await sb.schema('api').rpc('ai_log', { p_task: b.task, p_outcome: 'provider_error', p_reason:
String(res?.status ?? 'timeout'), p_tokens: 0 }); return NextResponse.json({ allowed: true, error: 'Ask Semester is unavailable
right now. The offices below can help.', label: d.label }, { status: 502 }); }
  const out = await res.json();
  await sb.schema('api').rpc('ai_log', { p_task: b.task, p_outcome: 'answered', p_reason: d.reason, p_tokens:
out.usage?.output_tokens ?? 0 });
  return NextResponse.json({ allowed: true, label: d.label, answer: out.content?.[0]?.text ?? '', sources: ctx.data ?? [],
dropped: d.dropped.length, redacted: s.findings.length, model: pol.model });
}
```

handoff/semester-platform/app/api/community/reports/route.ts · 12 lines

```
import { NextResponse } from 'next/server';
import { createClient } from '@/lib/supabase/server';
import { Report } from '@/lib/validate';
export async function POST(req: Request) {
  const parsed = Report.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error: 'Pick a reason' }, { status: 400 });
  const sb = await createClient();
  const { error } = await sb.schema('community').from('reports').insert({ target_type: parsed.data.targetType, target_id:
parsed.data.targetId, reason: parsed.data.reason, note: parsed.data.note });
  if (error) return NextResponse.json({ error: 'Could not send the report. Try again.' }, { status: 500 });
```

```
  return NextResponse.json({ message: 'Report sent to campus moderators.' }, { status: 201 });
}
```

handoff/semester-platform/app/api/market/listings/route.ts · 16 lines

```
import { NextResponse } from 'next/server';
import { createClient } from '@/lib/supabase/server';
import { Listing, integrityPrecheck } from '@/lib/validate';
export async function POST(req: Request) {
  const parsed = Listing.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error: 'Check the listing details', issues: parsed.error.flatten() }, {
status: 400 });
  const sb = await createClient();
  const { data: { user } } = await sb.auth.getUser();
  if (!user) return NextResponse.json({ error: 'Sign in to sell' }, { status: 401 });
  const pre = integrityPrecheck(parsed.data.title);
  const { data, error } = await sb.schema('market').from('listings').insert({ title: parsed.data.title, category:
parsed.data.category, price_cents: parsed.data.priceCents }).select().single();
  if (error) return NextResponse.json({ error: 'Your campus may not have the marketplace turned on.' }, { status: 403 });
  await sb.rpc('log', { a: 'listing.create', t: data.id, b: null, af: { ...data, precheck: pre } });
  return NextResponse.json({ listing: data, message: pre.hold ? 'Submitted. It will be reviewed before going live.' :
'Submitted. Live after the automatic check.' }, { status: 201 });
}
```

handoff/semester-platform/lib/ai/redact.ts · 22 lines

```
// Content scan that runs AFTER gate() and BEFORE any provider call. gate() trusts scope labels; this catches records typed or
pasted into the prompt.
// Patterns are deliberately conservative: a false positive costs a rephrase, a false negative costs a FERPA incident.
export interface Finding { kind: string; match: string }
export interface Scan { text: string; findings: Finding[]; block: boolean }
export interface TenantPatterns { studentId?: RegExp; extra?: { kind: string; re: RegExp }[] }
const BASE: { kind: string; re: RegExp; block?: boolean }[] = [
  { kind: 'ssn', re: /\b\d{3}-\d{2}-\d{4}\b/g, block: true },
  { kind: 'email', re: /\b[\w.+-]+@[\w-]+\.[\w.]+\b/g },
  { kind: 'phone', re: /\b(?:\+?1[ .-]?)?\(?\d{3}\)?[ .-]?\d{3}[ .-]?\d{4}\b/g },
  { kind: 'dob', re: /\b(?:born|dob|date of birth)[:\s]+\d{1,2}[\/.-]\d{1,2}[\/.-]\d{2,4}\b/gi, block: true },
  { kind: 'gpa', re: /\bgpa\s*(?:of|is|:)?\s*[0-4]\.\d{1,2}\b/gi, block: true },
  { kind: 'grade-record', re: /\b(?:got|received|earned|my grade (?:is|was))\s+(?:an?\s+)?[A-F][+-]?\s+(?:in|on)\b/gi, block:
true },
  { kind: 'transcript', re: /\b(?:transcript|academic record|disciplinary|accommodation (?:letter|plan)|financial aid
award)\b/gi, block: true },
  { kind: 'card', re: /\b(?:\d[ -]?){13,16}\b/g, block: true },
];
export function scan(input: string, t: TenantPatterns = {}): Scan {
  const rules = [...BASE, ...(t.studentId ? [{ kind: 'student-id', re: new RegExp(t.studentId.source, 'g'), block: true }] :
[]), ...(t.extra ?? []).map((e) => ({ ...e, block: true }))];
  const findings: Finding[] = []; let text = input; let block = false;
  for (const r of rules) { text = text.replace(r.re, (m) => { findings.push({ kind: r.kind, match: m }); if (r.block) block =
true; return '[' + r.kind + ' removed]'; }); }
  return { text, findings, block };
}
```

handoff/semester-platform/lib/validate.ts · 11 lines

```
import { z } from 'zod';
export const Listing = z.object({ title: z.string().min(3).max(120), category:
z.enum(['textbooks','gear','housing','services','tickets']), priceCents: z.number().int().min(0).max(1_000_000) });
export const Offer = z.object({ listingId: z.string().uuid(), amountCents: z.number().int().positive() });
export const Post = z.object({ groupId: z.string().uuid().nullable(), body: z.string().min(1).max(5000) });
export const Report = z.object({ targetType: z.enum(['post','message','listing','profile']), targetId: z.string().uuid(),
reason: z.enum(['harassment','spam','integrity','unsafe','other']), note: z.string().max(2000).optional() });
const BANNED = [/answer\s*key/i, /exam\s*\d*\s*(answers|solutions)/i, /graded\s*(essay|paper|work)/i, /write\s*your\s*
(essay|paper)/i];
export function integrityPrecheck(title: string): { hold: boolean; reason?: string } {
  const hit = BANNED.find(r => r.test(title));
  return hit ? { hold: true, reason: 'Possible academic integrity violation' } : { hold: false };
}
```

handoff/semester-platform/middleware.security.ts · 27 lines

```
// Security headers + session refresh for every request. Copy to the app root as middleware.ts (merge with existing auth
middleware).
import { NextResponse, type NextRequest } from 'next/server';
const CSP = [
  "default-src 'self'",
  "script-src 'self'",
  "style-src 'self' 'unsafe-inline'",
  "img-src 'self' data: blob: https://*.supabase.co",
  "connect-src 'self' https://*.supabase.co wss://*.supabase.co",
  "font-src 'self'",
  "frame-ancestors 'none'",
  "base-uri 'self'",
  "form-action 'self'",
  "upgrade-insecure-requests",
].join('; ');
export function middleware(req: NextRequest) {
  const res = NextResponse.next();
  res.headers.set('Content-Security-Policy', CSP);
  res.headers.set('Strict-Transport-Security', 'max-age=63072000; includeSubDomains; preload');
  res.headers.set('X-Content-Type-Options', 'nosniff');
  res.headers.set('Referrer-Policy', 'strict-origin-when-cross-origin');
  res.headers.set('Permissions-Policy', 'camera=(self), microphone=(self), geolocation=(self), payment=(), usb=()');
  res.headers.set('Cross-Origin-Opener-Policy', 'same-origin');
  if (req.nextUrl.pathname.startsWith('/api/')) res.headers.set('Cache-Control', 'no-store');
  return res;
}
export const config = { matcher: ['/((?!_next/static|_next/image|favicon.ico).*)'] };
```

handoff/semester-platform/supabase/migrations/010_platform_core.sql · 16 lines

```
-- Tenancy, profiles, roles, audit
create schema if not exists core; create schema if not exists audit;
create table core.institutions(id uuid primary key default gen_random_uuid(), name text not null, features jsonb not null
default '{}'::jsonb, created_at timestamptz default now());
create table core.profiles(id uuid primary key references auth.users on delete cascade, institution_id uuid not null references
core.institutions, display_name text not null, role text not null check (role in
('student','faculty','ta','advisor','registrar','student_affairs','it_admin','moderator','support')), visibility jsonb not null
default '{"major":true,"class_year":false}'::jsonb, created_at timestamptz default now());
create or replace function core.my_institution() returns uuid language sql stable as $$ select institution_id from
core.profiles where id = auth.uid() $$;
create or replace function core.my_role() returns text language sql stable as $$ select role from core.profiles where id =
auth.uid() $$;
create or replace function core.feature_on(f text) returns boolean language sql stable as $$ select coalesce((features-
>>f)::boolean,false) from core.institutions where id = core.my_institution() $$;
alter table core.profiles enable row level security;
create policy profiles_same_campus on core.profiles for select using (institution_id = core.my_institution());
create policy profiles_self_update on core.profiles for update using (id = auth.uid()) with check (id = auth.uid() and role =
core.my_role());
create table audit.events(id bigserial primary key, at timestamptz default now(), actor uuid, actor_role text, institution_id
uuid, action text not null, target text, before jsonb, after jsonb);
alter table audit.events enable row level security;
create policy audit_admin_read on audit.events for select using (institution_id = core.my_institution() and core.my_role() in
('it_admin','registrar'));
-- inserts only through security definer function
create or replace function audit.log(a text, t text, b jsonb, af jsonb) returns void language sql security definer set
search_path = '' as $$ insert into audit.events(actor,actor_role,institution_id,action,target,before,after) values (auth.uid(),
core.my_role(), core.my_institution(), a, t, b, af) $$;
```

handoff/semester-platform/supabase/migrations/020_community.sql · 22 lines

```
create schema if not exists community;
create table community.groups(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), name text not null, kind text not null check (kind in ('course','club','community','rec')), created_by
uuid not null default auth.uid(), created_at timestamptz default now());
create table community.members(group_id uuid references community.groups on delete cascade, profile_id uuid references
core.profiles on delete cascade, role text not null default 'member' check (role in ('member','leader')), primary
key(group_id,profile_id));
create table community.posts(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), group_id uuid references community.groups on delete cascade, author uuid not null default auth.uid(),
body text not null check (length(body) between 1 and 5000), status text not null default 'visible' check (status in
('visible','held','removed')), created_at timestamptz default now());
create table community.reports(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
```

```
core.my_institution(), target_type text not null check (target_type in ('post','message','listing','profile')), target_id uuid
not null, reason text not null, note text, reporter uuid not null default auth.uid(), status text not null default 'open',
created_at timestamptz default now());
create table community.events(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), group_id uuid references community.groups, title text not null, starts_at timestamptz not null, location
text);
create table community.rsvps(event_id uuid references community.events on delete cascade, profile_id uuid default auth.uid()
references core.profiles on delete cascade, primary key(event_id,profile_id));
alter table community.groups enable row level security; alter table community.members enable row level security; alter table
community.posts enable row level security; alter table community.reports enable row level security; alter table
community.events enable row level security; alter table community.rsvps enable row level security;
create policy g_read on community.groups for select using (institution_id = core.my_institution() and
core.feature_on('community'));
create policy g_create on community.groups for insert with check (institution_id = core.my_institution() and
core.feature_on('community'));
create policy m_read on community.members for select using (exists(select 1 from community.groups g where g.id = group_id and
g.institution_id = core.my_institution()));
create policy m_join on community.members for insert with check (profile_id = auth.uid() and role = 'member');
create policy p_read on community.posts for select using (institution_id = core.my_institution() and (status = 'visible' or
author = auth.uid() or core.my_role() in ('moderator','student_affairs')));
create policy p_write on community.posts for insert with check (author = auth.uid() and core.feature_on('community') and
(group_id is null or exists(select 1 from community.members m where m.group_id = posts.group_id and m.profile_id =
auth.uid())));
create policy p_moderate on community.posts for update using (core.my_role() in ('moderator','student_affairs') and
institution_id = core.my_institution());
create policy r_create on community.reports for insert with check (reporter = auth.uid());
create policy r_mod on community.reports for select using (institution_id = core.my_institution() and core.my_role() in
('moderator','student_affairs'));
create policy e_read on community.events for select using (institution_id = core.my_institution());
-- RSVPs visible to self and to group leaders only
create policy rs_self on community.rsvps for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());
create policy rs_leader on community.rsvps for select using (exists(select 1 from community.events e join community.members m
on m.group_id = e.group_id where e.id = event_id and m.profile_id = auth.uid() and m.role = 'leader'));
```

handoff/semester-platform/supabase/migrations/030_marketplace.sql · 20 lines

```
create schema if not exists market;
create table market.listings(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), seller uuid not null default auth.uid(), title text not null, category text not null check (category in
('textbooks','gear','housing','services','tickets')), price_cents int not null check (price_cents >= 0), status text not null
default 'pending_check' check (status in ('pending_check','live','held','sold','removed')), integrity_score real, created_at
timestamptz default now());
create table market.offers(id uuid primary key default gen_random_uuid(), listing_id uuid not null references market.listings
on delete cascade, buyer uuid not null default auth.uid(), amount_cents int not null check (amount_cents > 0), status text not
null default 'open' check (status in ('open','accepted','declined','paid','completed','disputed','refunded')), processor_ref
text, created_at timestamptz default now());
create table market.drops(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), listing_id uuid references market.listings, seller_is_official boolean not null default false, ends_at
timestamptz not null, approved boolean not null default false);
create table market.bids(id bigserial primary key, drop_id uuid references market.drops on delete cascade, bidder uuid not null
default auth.uid(), amount_cents int not null, at timestamptz default now());
alter table market.listings enable row level security; alter table market.offers enable row level security; alter table
market.drops enable row level security; alter table market.bids enable row level security;
create policy l_read on market.listings for select using (institution_id = core.my_institution() and
core.feature_on('marketplace') and (status = 'live' or seller = auth.uid() or core.my_role() in
('moderator','student_affairs')));
-- new listings always start pending_check; a worker runs the integrity classifier and sets live/held
create policy l_create on market.listings for insert with check (seller = auth.uid() and status = 'pending_check' and
core.feature_on('marketplace'));
create policy l_seller_update on market.listings for update using (seller = auth.uid()) with check (seller = auth.uid() and
status in ('pending_check','sold','removed'));
create policy l_mod on market.listings for update using (core.my_role() in ('moderator','student_affairs') and institution_id =
core.my_institution());
create policy o_parties on market.offers for select using (buyer = auth.uid() or exists(select 1 from market.listings l where
l.id = listing_id and l.seller = auth.uid()));
create policy o_create on market.offers for insert with check (buyer = auth.uid() and status = 'open' and exists(select 1 from
market.listings l where l.id = listing_id and l.status = 'live' and l.seller <> auth.uid()));
create policy d_read on market.drops for select using (institution_id = core.my_institution() and approved);
create policy b_create on market.bids for insert with check (bidder = auth.uid() and exists(select 1 from market.drops d where
d.id = drop_id and d.approved and d.ends_at > now()));
create policy b_read on market.bids for select using (exists(select 1 from market.drops d where d.id = drop_id and
d.institution_id = core.my_institution()));
-- Bids must exceed current max
create or replace function market.check_bid() returns trigger language plpgsql as $$ begin if new.amount_cents <=
coalesce((select max(amount_cents) from market.bids where drop_id = new.drop_id),0) then raise exception 'Bid must be higher
```

```
than the current bid'; end if; return new; end $$;
create trigger bid_higher before insert on market.bids for each row execute function market.check_bid();
```

handoff/semester-platform/supabase/migrations/040_ops.sql · 18 lines

```
-- Company operations. No student identifiers in this schema, ever.
create schema if not exists ops;
create table ops.staff(id uuid primary key references auth.users, role text not null check (role in
('ceo','cfo','finance','comms','social','sales','revops','cs','support','tns','legal','people','security','data','partnerships'
)));
create or replace function ops.my_role() returns text language sql stable as $$ select role from ops.staff where id =
auth.uid() $$;
create table ops.accounts(id uuid primary key default gen_random_uuid(), institution_name text not null, acv_cents bigint,
term_ends date, health int check (health between 0 and 100), csm uuid references ops.staff);
create table ops.invoices(id text primary key, account_id uuid references ops.accounts, amount_cents bigint not null, due date
not null, status text not null default 'sent');
create table ops.social_posts(id uuid primary key default gen_random_uuid(), body text not null, channels text[] not null,
scheduled_at timestamptz, author uuid default auth.uid(), status text not null default 'draft' check (status in
('draft','needs_approval','approved','scheduled','published','paused','rejected')), legal_review boolean default false);
create table ops.settings(key text primary key, value jsonb not null);
alter table ops.accounts enable row level security; alter table ops.invoices enable row level security; alter table
ops.social_posts enable row level security; alter table ops.settings enable row level security;
create policy acc_read on ops.accounts for select using (ops.my_role() in
('ceo','cfo','finance','sales','revops','cs','partnerships'));
create policy inv_read on ops.invoices for select using (ops.my_role() in ('ceo','cfo','finance'));
create policy sp_read on ops.social_posts for select using (ops.my_role() in ('ceo','comms','social'));
create policy sp_draft on ops.social_posts for insert with check (ops.my_role() in ('comms','social') and status in
('draft','needs_approval'));
create policy sp_approve on ops.social_posts for update using (ops.my_role() = 'comms');
create policy st_read on ops.settings for select using (ops.my_role() is not null);
create policy st_crisis on ops.settings for update using (ops.my_role() in ('comms','ceo') and key = 'crisis_mode');
-- Guard: ops staff without a campus profile can never read edu/community/market (they have no core.profiles row, so
my_institution() is null).
```

handoff/semester-platform/supabase/migrations/050_security_hardening.sql · 80 lines

```
-- 050 · Security hardening from the Oct 2026 review of migrations 010–040.
-- Each block names the finding it closes (see docs/secure/Platform Security Review.html). Not yet run: apply in staging and
prove with tests/security.test.ts.

-- F1 core.institutions had no RLS: any signed-in user could read every tenant's feature flags.
alter table core.institutions enable row level security;
create policy inst_own_read on core.institutions for select using (id = core.my_institution());

-- F2 CRITICAL ops.staff had no RLS: a tenant user could read company staff or insert themselves as 'ceo' and gain every ops
policy.
alter table ops.staff enable row level security;
create policy staff_self_read on ops.staff for select using (id = auth.uid());
revoke insert, update, delete on ops.staff from anon, authenticated;
-- staff rows are created only by the service role from the company IdP.

-- F3 Helper functions read RLS-protected tables from inside RLS policies (recursion risk, planner surprises). Make them
definer functions with a fixed search_path.
create or replace function core.my_institution() returns uuid language sql stable security definer set search_path = '' as $$
select institution_id from core.profiles where id = auth.uid() $$;
create or replace function core.my_role() returns text language sql stable security definer set search_path = '' as $$ select
role from core.profiles where id = auth.uid() $$;
create or replace function core.feature_on(f text) returns boolean language sql stable security definer set search_path = '' as
$$ select coalesce((i.features->>f)::boolean, false) from core.institutions i where i.id = core.my_institution() $$;
create or replace function ops.my_role() returns text language sql stable security definer set search_path = '' as $$ select
role from ops.staff where id = auth.uid() $$;
revoke all on function core.my_institution(), core.my_role(), core.feature_on(text), ops.my_role() from anon;

-- F4 profiles_same_campus exposed every column (role, visibility settings) of every profile on campus, ignoring per-field
visibility.
drop policy if exists profiles_same_campus on core.profiles;
create policy profiles_self_read on core.profiles for select using (id = auth.uid());
create policy profiles_staff_read on core.profiles for select using (institution_id = core.my_institution() and core.my_role()
in ('advisor','registrar','student_affairs','it_admin','support','moderator'));
-- Directory for everyone else: only display name and fields the person made visible.
create or replace view core.directory with (security_barrier = true) as
  select p.id, p.display_name,
         case when (p.visibility->>'major')::boolean then p.visibility->>'major_value' end as major,
```

```
         case when (p.visibility->>'class_year')::boolean then p.visibility->>'class_year_value' end as class_year
  from core.profiles p
  where p.institution_id = core.my_institution() and coalesce((p.visibility->>'findable')::boolean, false);
grant select on core.directory to authenticated;

-- F5 HIGH profiles_self_update let a user move themselves to another institution_id (cross-tenant escalation). Freeze identity
columns.
create or replace function core.freeze_profile_identity() returns trigger language plpgsql security definer set search_path =
'' as $$
begin
  if new.id <> old.id or new.institution_id <> old.institution_id or new.role <> old.role then
    raise exception 'identity columns are managed by SSO provisioning';
  end if;
  return new;
end $$;
drop trigger if exists profiles_freeze on core.profiles;
create trigger profiles_freeze before update on core.profiles for each row execute function core.freeze_profile_identity();

-- F6 audit.log was executable by any user, so clients could write arbitrary audit rows.
revoke execute on function audit.log(text, text, jsonb, jsonb) from anon, authenticated;
-- server command functions (security definer) call audit.log; grant to the role that owns them:
-- grant execute on function audit.log(text, text, jsonb, jsonb) to service_role;

-- F7 Audit must be append-only.
revoke update, delete, truncate on audit.events from anon, authenticated;

-- F8 market.bids exposed every bidder's identity to the whole campus.
drop policy if exists b_read on market.bids;
create policy b_own on market.bids for select using (bidder = auth.uid());
create or replace view market.drop_bid_summary with (security_barrier = true) as
  select b.drop_id, count(*) as bids, max(b.amount_cents) as top_bid_cents
  from market.bids b join market.drops d on d.id = b.drop_id
  where d.institution_id = core.my_institution()
  group by b.drop_id;
grant select on market.drop_bid_summary to authenticated;

-- F9 community.members exposed membership of every group (including sensitive ones) campus-wide.
-- A policy on members cannot query members itself (infinite recursion), so membership is checked in a definer function.
create or replace function community.is_member(g uuid) returns boolean language sql stable security definer set search_path =
'' as $ select exists(select 1 from community.members m where m.group_id = g and m.profile_id = auth.uid()) $;
revoke all on function community.is_member(uuid) from anon;
drop policy if exists m_read on community.members;
create policy m_read_same_group on community.members for select using (
  profile_id = auth.uid() or community.is_member(group_id) or (core.my_role() in ('moderator','student_affairs') and exists
(select 1 from community.groups g where g.id = group_id and g.institution_id = core.my_institution())));

-- F10 rsvps 'all' policy checked only profile_id, so a user could RSVP to another institution's event by id.
drop policy if exists rs_self on community.rsvps;
create policy rs_self_read on community.rsvps for select using (profile_id = auth.uid());
create policy rs_self_write on community.rsvps for insert with check (profile_id = auth.uid() and exists (select 1 from
community.events e where e.id = event_id and e.institution_id = core.my_institution()));
create policy rs_self_delete on community.rsvps for delete using (profile_id = auth.uid());

-- F11 Default-deny for anonymous users on every schema.
revoke all on all tables in schema core, audit, community, market, ops from anon;
alter default privileges in schema core, audit, community, market, ops revoke all on tables from anon;
```

handoff/semester-platform/supabase/migrations/060_command_functions.sql · 101 lines

```
-- 060 · Write paths as audited command functions (closes "no insert policies" from the security review).
-- Clients never insert into these tables directly; every write goes through a function that validates, rate-limits, writes the
audit event first, then applies.
-- Not yet run. Prove with tests/commands.test.ts.

create schema if not exists api;

-- Simple per-user rate limit (window, max) backed by a counter table.
create table if not exists core.rate_counters(actor uuid not null, action text not null, window_start timestamptz not null, n
int not null default 0, primary key(actor, action, window_start));
alter table core.rate_counters enable row level security; -- no policies: definer functions only
create or replace function core.rate_ok(a text, per interval, max_n int) returns boolean language plpgsql security definer set
search_path = '' as $$
declare w timestamptz := date_trunc('minute', now()); c int;
begin
  insert into core.rate_counters(actor, action, window_start, n) values (auth.uid(), a, w, 1)
```

```
  on conflict (actor, action, window_start) do update set n = core.rate_counters.n + 1 returning n into c;
  select coalesce(sum(n),0) into c from core.rate_counters where actor = auth.uid() and action = a and window_start > now() -
per;
  return c <= max_n;
end $$;

-- Re-grant audit.log to the owner of the command functions (definer functions run as owner).
grant execute on function audit.log(text, text, jsonb, jsonb) to postgres;

-- Create a post. Public (non-group) posts start held for the AI pre-screen; group posts are visible.
create or replace function api.create_post(p_group uuid, p_body text) returns uuid language plpgsql security definer set
search_path = '' as $$
declare pid uuid; st text;
begin
  if auth.uid() is null then raise exception 'sign in required'; end if;
  if not core.feature_on('community') then raise exception 'community is off for this institution'; end if;
  if length(coalesce(p_body,'')) not between 1 and 5000 then raise exception 'post must be 1–5000 characters'; end if;
  if not core.rate_ok('create_post', interval '10 minutes', 20) then raise exception 'too many posts, try again shortly'; end
if;
  if p_group is not null and not community.is_member(p_group) then raise exception 'not a member of this group'; end if;
  st := case when p_group is null then 'held' else 'visible' end;
  pid := gen_random_uuid();
  perform audit.log('community.post.create', pid::text, null, jsonb_build_object('group', p_group, 'status', st, 'length',
length(p_body)));
  insert into community.posts(id, institution_id, group_id, author, body, status) values (pid, core.my_institution(), p_group,
auth.uid(), p_body, st);
  return pid;
end $$;

-- Create a listing. Always starts pending the academic-integrity check; never live on insert.
create or replace function api.create_listing(p_title text, p_category text, p_price_cents int) returns uuid language plpgsql
security definer set search_path = '' as $$
declare lid uuid;
begin
  if auth.uid() is null then raise exception 'sign in required'; end if;
  if not core.feature_on('marketplace') then raise exception 'marketplace is off for this institution'; end if;
  if core.my_role() <> 'student' then raise exception 'only students can list items'; end if;
  if length(coalesce(p_title,'')) not between 3 and 120 then raise exception 'title must be 3–120 characters'; end if;
  if p_category not in ('textbooks','gear','housing','services','tickets') then raise exception 'unknown category'; end if;
  if p_price_cents < 0 or p_price_cents > 1000000 then raise exception 'price out of range'; end if;
  if p_title ~* '(answer key|exam|test bank|solutions manual|graded essay|homework answers)' then raise exception 'listings
that break academic integrity rules are not allowed'; end if;
  if not core.rate_ok('create_listing', interval '1 day', 10) then raise exception 'daily listing limit reached'; end if;
  lid := gen_random_uuid();
  perform audit.log('market.listing.create', lid::text, null, jsonb_build_object('category', p_category, 'price_cents',
p_price_cents));
  insert into market.listings(id, institution_id, seller, title, category, price_cents, status) values (lid,
core.my_institution(), auth.uid(), p_title, p_category, p_price_cents, 'pending_check');
  return lid;
end $$;

-- Make an offer. Not on your own listing; listing must be live and on your campus.
create or replace function api.make_offer(p_listing uuid, p_amount_cents int) returns uuid language plpgsql security definer
set search_path = '' as $$
declare oid uuid; l record;
begin
  select * into l from market.listings where id = p_listing;
  if l is null or l.institution_id <> core.my_institution() or l.status <> 'live' then raise exception 'listing not available';
end if;
  if l.seller = auth.uid() then raise exception 'you cannot offer on your own listing'; end if;
  if p_amount_cents <= 0 or p_amount_cents > 1000000 then raise exception 'offer out of range'; end if;
  if not core.rate_ok('make_offer', interval '1 hour', 30) then raise exception 'too many offers, try again later'; end if;
  oid := gen_random_uuid();
  perform audit.log('market.offer.create', oid::text, null, jsonb_build_object('listing', p_listing, 'amount_cents',
p_amount_cents));
  insert into market.offers(id, listing_id, buyer, amount_cents) values (oid, p_listing, auth.uid(), p_amount_cents);
  return oid;
end $$;

-- Report content. The target must exist on the reporter's campus; reporter identity is visible to moderators only.
create or replace function api.report(p_type text, p_target uuid, p_reason text, p_note text) returns uuid language plpgsql
security definer set search_path = '' as $$
declare rid uuid; ok boolean;
begin
  if p_type not in ('post','message','listing','profile') then raise exception 'unknown report type'; end if;
  if p_reason not in ('harassment','hate','threat','privacy','integrity','scam','illegal','other') then raise exception
'unknown reason'; end if;
```

```
  ok := case p_type
    when 'post' then exists(select 1 from community.posts where id = p_target and institution_id = core.my_institution())
    when 'listing' then exists(select 1 from market.listings where id = p_target and institution_id = core.my_institution())
    when 'profile' then exists(select 1 from core.profiles where id = p_target and institution_id = core.my_institution())
    else true end;
  if not ok then raise exception 'target not found'; end if;
  if not core.rate_ok('report', interval '1 hour', 20) then raise exception 'too many reports, try again later'; end if;
  rid := gen_random_uuid();
  perform audit.log('community.report.create', rid::text, null, jsonb_build_object('type', p_type, 'reason', p_reason));
  insert into community.reports(id, institution_id, target_type, target_id, reason, note, reporter) values (rid,
core.my_institution(), p_type, p_target, p_reason, left(p_note, 2000), auth.uid());
  return rid;
end $$;

-- Only signed-in users may call commands; direct table writes stay closed.
revoke all on schema api from anon;
grant usage on schema api to authenticated;
grant execute on function api.create_post(uuid, text), api.create_listing(text, text, int), api.make_offer(uuid, int),
api.report(text, uuid, text, text) to authenticated;
revoke insert on community.posts, community.reports, market.listings, market.offers from authenticated;

-- Storage: listing photos and post attachments. Path convention: <institution_id>/<owner uid>/<file>.
-- insert into storage.buckets (id, name, public) values ('listing-photos', 'listing-photos', false) on conflict do nothing;
create policy photos_campus_read on storage.objects for select using (bucket_id = 'listing-photos' and
(storage.foldername(name))[1] = core.my_institution()::text);
create policy photos_owner_write on storage.objects for insert with check (bucket_id = 'listing-photos' and
(storage.foldername(name))[1] = core.my_institution()::text and (storage.foldername(name))[2] = auth.uid()::text);
create policy photos_owner_delete on storage.objects for delete using (bucket_id = 'listing-photos' and
(storage.foldername(name))[2] = auth.uid()::text);
```

handoff/semester-platform/supabase/migrations/070_ai_gateway.sql · 22 lines

```
-- 070 · AI gateway support: per-tenant policy, context of approved sources only, usage log. Not yet run.
create table if not exists core.ai_policy(institution_id uuid primary key references core.institutions, ai_on boolean not null
default false, weekly_budget int not null default 50, student_id_pattern text, provider text not null default 'approved-
default', model text not null);
alter table core.ai_policy enable row level security;
create policy aip_admin on core.ai_policy for select using (institution_id = core.my_institution() and core.my_role() =
'it_admin');
create table if not exists core.course_ai_rules(course_id uuid primary key, institution_id uuid not null references
core.institutions, allows text[] not null default '{}');
alter table core.course_ai_rules enable row level security;
create policy car_read on core.course_ai_rules for select using (institution_id = core.my_institution());
create table if not exists audit.ai_usage(id bigserial primary key, at timestamptz default now(), actor uuid, institution_id
uuid, task text, outcome text check (outcome in ('answered','refused','blocked','provider_error')), reason text, tokens int);
alter table audit.ai_usage enable row level security;
create policy aiu_admin on audit.ai_usage for select using (institution_id = core.my_institution() and core.my_role() =
'it_admin');
revoke update, delete, truncate on audit.ai_usage from anon, authenticated;
-- Note: ai_usage stores task, outcome and reason codes only. Never the prompt or the answer.
create or replace function api.ai_policy(p_course uuid) returns jsonb language sql stable security definer set search_path = ''
as $$
  select jsonb_build_object('institutionAiOn', p.ai_on, 'courseAllows', coalesce(r.allows, '{}'), 'model', p.model,
'studentIdPattern', p.student_id_pattern,
    'budgetLeft', p.weekly_budget - (select count(*) from audit.ai_usage u where u.actor = auth.uid() and u.outcome =
'answered' and u.at > now() - interval '7 days'))
  from core.ai_policy p left join core.course_ai_rules r on r.course_id = p_course and r.institution_id = p.institution_id
  where p.institution_id = core.my_institution() $$;
create or replace function api.ai_log(p_task text, p_outcome text, p_reason text, p_tokens int) returns void language sql
security definer set search_path = '' as $$
  insert into audit.ai_usage(actor, institution_id, task, outcome, reason, tokens) values (auth.uid(), core.my_institution(),
p_task, p_outcome, left(p_reason, 200), p_tokens) $$;
grant execute on function api.ai_policy(uuid), api.ai_log(text, text, text, int) to authenticated;
-- api.ai_context(p_ids) returns approved course/public sources by id with titles for citation; implement against the course
resource tables in stream 06.
```

handoff/semester-platform/supabase/migrations/080_sources_storage_delegations.sql · 43 lines

```
-- 080 · Remaining review items: approved-source context for AI, storage buckets, sessions and delegations. Not yet run.

-- Approved sources for AI answers: course resources and public institution pages only, never student records.
create table if not exists core.sources(id uuid primary key default gen_random_uuid(), institution_id uuid not null references
core.institutions, course_id uuid, kind text not null check (kind in ('policy','course','catalog','support','public')), title
```

```
text not null, url text, body text not null, version text, updated_at timestamptz default now(), approved boolean not null
default false);
alter table core.sources enable row level security;
create policy src_read on core.sources for select using (institution_id = core.my_institution() and approved);
create or replace function api.ai_context(p_ids text[]) returns jsonb language sql stable security definer set search_path = ''
as $$
  select coalesce(jsonb_agg(jsonb_build_object('n', rn, 'id', id, 'title', title, 'kind', kind, 'version', version, 'url', url,
'excerpt', left(body, 1500)) order by rn), '[]'::jsonb)
  from (select s.*, row_number() over (order by s.kind = 'policy' desc, s.updated_at desc) rn from core.sources s
        where s.institution_id = core.my_institution() and s.approved and s.id::text = any(p_ids) limit 8) t $$;
grant execute on function api.ai_context(text[]) to authenticated;

-- Storage: every bucket private; path = <institution_id>/<owner uid>/<file>.
-- buckets: 'student-files' (owner only), 'course-materials' (course members read, faculty write), 'evidence' (Semester trust
team only, service role).
create policy sf_owner on storage.objects for all using (bucket_id = 'student-files' and (storage.foldername(name))[1] =
core.my_institution()::text and (storage.foldername(name))[2] = auth.uid()::text) with check (bucket_id = 'student-files' and
(storage.foldername(name))[1] = core.my_institution()::text and (storage.foldername(name))[2] = auth.uid()::text);
create policy cm_read on storage.objects for select using (bucket_id = 'course-materials' and (storage.foldername(name))[1] =
core.my_institution()::text);
create policy cm_write on storage.objects for insert with check (bucket_id = 'course-materials' and (storage.foldername(name))
[1] = core.my_institution()::text and core.my_role() in ('faculty','ta'));
-- 'evidence' bucket: no policies for authenticated; service role only.

-- Sessions and delegations (account switcher, remote sign-out, delegated access with expiry).
create table if not exists core.delegations(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), grantor uuid not null default auth.uid(), grantee uuid not null, scope text[] not null, purpose text not
null, expires_at timestamptz not null check (expires_at <= now() + interval '365 days'), revoked_at timestamptz, created_at
timestamptz default now());
alter table core.delegations enable row level security;
create policy dl_parties on core.delegations for select using (grantor = auth.uid() or grantee = auth.uid());
create or replace function api.grant_delegation(p_grantee uuid, p_scope text[], p_purpose text, p_expires timestamptz) returns
uuid language plpgsql security definer set search_path = '' as $$
declare did uuid;
begin
  if not exists(select 1 from core.profiles where id = p_grantee and institution_id = core.my_institution()) then raise
exception 'recipient not found'; end if;
  if p_expires <= now() or p_expires > now() + interval '365 days' then raise exception 'expiry must be within a year'; end if;
  did := gen_random_uuid();
  perform audit.log('core.delegation.grant', did::text, null, jsonb_build_object('grantee', p_grantee, 'scope', p_scope,
'expires', p_expires));
  insert into core.delegations(id, grantee, scope, purpose, expires_at) values (did, p_grantee, p_scope, p_purpose, p_expires);
  return did;
end $$;
create or replace function api.revoke_delegation(p_id uuid) returns void language plpgsql security definer set search_path = ''
as $$
begin
  perform audit.log('core.delegation.revoke', p_id::text, null, null);
  update core.delegations set revoked_at = now() where id = p_id and grantor = auth.uid() and revoked_at is null;
  if not found then raise exception 'not found'; end if;
end $$;
create or replace function core.has_delegation(p_grantor uuid, p_scope text) returns boolean language sql stable security
definer set search_path = '' as $$
  select exists(select 1 from core.delegations d where d.grantor = p_grantor and d.grantee = auth.uid() and p_scope =
any(d.scope) and d.revoked_at is null and d.expires_at > now()) $$;
grant execute on function api.grant_delegation(uuid, text[], text, timestamptz), api.revoke_delegation(uuid) to authenticated;
```

handoff/semester-platform/supabase/seed.sql · 41 lines

```
set search_path = public, extensions;
-- Seed runs as the postgres role, so RLS and defaults that call auth.uid() are bypassed or null: every row sets institution_id
and owner explicitly.
-- Test seed for local Supabase (supabase/seed.sql). Synthetic people only. IDs are fixed so tests can reference them via
.env.test.
-- Local test users. auth.identities rows are required for password sign-in on current Supabase versions.
insert into auth.users (instance_id, id, aud, role, email, encrypted_password, email_confirmed_at, raw_app_meta_data,
raw_user_meta_data, created_at, updated_at, confirmation_token, email_change, email_change_token_new, recovery_token) values
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000a001','authenticated','authenticated','student@campus-a.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000a002','authenticated','authenticated','student2@campus-a.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-00000000a003','authenticated','authenticated','seller@campus-
a.test',crypt('Test-pass-1!', gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
```

```
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000a004','authenticated','authenticated','advisor@campus-a.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-00000000a005','authenticated','authenticated','it@campus-
a.test',crypt('Test-pass-1!', gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000b001','authenticated','authenticated','student@campus-b.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000c001','authenticated','authenticated','cfo@semester.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000c002','authenticated','authenticated','social@semester.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','','');
insert into auth.identities (provider_id, user_id, identity_data, provider, last_sign_in_at, created_at, updated_at) values
 ('00000000-0000-0000-0000-00000000a001','00000000-0000-0000-0000-00000000a001','{"sub":"00000000-0000-0000-0000-
00000000a001","email":"student@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000a002','00000000-0000-0000-0000-00000000a002','{"sub":"00000000-0000-0000-0000-
00000000a002","email":"student2@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000a003','00000000-0000-0000-0000-00000000a003','{"sub":"00000000-0000-0000-0000-
00000000a003","email":"seller@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000a004','00000000-0000-0000-0000-00000000a004','{"sub":"00000000-0000-0000-0000-
00000000a004","email":"advisor@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000a005','00000000-0000-0000-0000-00000000a005','{"sub":"00000000-0000-0000-0000-
00000000a005","email":"it@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000b001','00000000-0000-0000-0000-00000000b001','{"sub":"00000000-0000-0000-0000-
00000000b001","email":"student@campus-b.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000c001','00000000-0000-0000-0000-00000000c001','{"sub":"00000000-0000-0000-0000-
00000000c001","email":"cfo@semester.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000c002','00000000-0000-0000-0000-00000000c002','{"sub":"00000000-0000-0000-0000-
00000000c002","email":"social@semester.test"}','email',now(),now(),now());
insert into core.institutions (id, name, features) values
 ('00000000-0000-0000-0000-0000000000aa','Campus A','{"community":true,"marketplace":true}'),
 ('00000000-0000-0000-0000-0000000000bb','Campus B','{"community":true,"marketplace":true}');
insert into core.profiles (id, institution_id, display_name, role, visibility) values
 ('00000000-0000-0000-0000-00000000a001','00000000-0000-0000-0000-0000000000aa','Student
A','student','{"findable":true,"major":true,"major_value":"Biology"}'),
 ('00000000-0000-0000-0000-00000000a002','00000000-0000-0000-0000-0000000000aa','Student Two','student','{"findable":false}'),
 ('00000000-0000-0000-0000-00000000a003','00000000-0000-0000-0000-0000000000aa','Seller','student','{"findable":true}'),
 ('00000000-0000-0000-0000-00000000a004','00000000-0000-0000-0000-0000000000aa','Advisor Lee','advisor','{}'),
 ('00000000-0000-0000-0000-00000000a005','00000000-0000-0000-0000-0000000000aa','IT Admin','it_admin','{}'),
 ('00000000-0000-0000-0000-00000000b001','00000000-0000-0000-0000-0000000000bb','Student B','student','{}');
insert into ops.staff (id, role) values ('00000000-0000-0000-0000-00000000c001','cfo'),('00000000-0000-0000-0000-
00000000c002','social');
insert into community.groups (id, institution_id, name, kind, created_by) values ('00000000-0000-0000-0000-
00000000d001','00000000-0000-0000-0000-0000000000aa','Private study group','community','00000000-0000-0000-0000-00000000a001');
insert into community.members (group_id, profile_id, role) values ('00000000-0000-0000-0000-00000000d001','00000000-0000-0000-
0000-00000000a001','leader');
insert into community.posts (id, institution_id, author, body) values ('00000000-0000-0000-0000-00000000e001','00000000-0000-
0000-0000-0000000000aa','00000000-0000-0000-0000-00000000a001','Seed post');
insert into community.events (id, institution_id, title, starts_at) values ('00000000-0000-0000-0000-00000000f0b1','00000000-
0000-0000-0000-0000000000bb','Campus B event', now() + interval '7 days');
insert into market.listings (id, institution_id, seller, title, category, price_cents, status) values ('00000000-0000-0000-
0000-00000000aa11','00000000-0000-0000-0000-0000000000aa','00000000-0000-0000-0000-00000000a003','Seed
lamp','gear',500,'live');
insert into core.ai_policy (institution_id, ai_on, weekly_budget, student_id_pattern, model) values ('00000000-0000-0000-0000-
0000000000aa', true, 50, '\\b\\d{9}\\b', 'approved-default');
-- If a column above does not exist in the repo's schema, fix the seed to match the migration (the migration is the source of
truth), never the reverse.
```

handoff/semester-platform/tests/ai-redteam.test.ts · 22 lines

```
// AI red-team suite (launch gate). Unit-level: no provider call needed. npx vitest run tests/ai-redteam.test.ts
import { describe, it, expect } from 'vitest';
import { gate } from '../lib/core/aigateway';
import { scan } from '../lib/ai/redact';
const SID = /\b\d{9}\b/; // tenant student-ID format, e.g. Vanderbilt-style 9 digits (set per tenant)
const policy = { institutionAiOn: true, courseAllows: ['outline', 'practice'] as any };
const blocked = [
  'My student ID is 004512398, what classes am I in?', 'Summarize my transcript', 'I got a C- in BIO 2100, explain my GPA
2.71',
  'My SSN is 123-45-6789', 'DOB: 04/12/2005 can you check my aid?', 'Here is my accommodation letter text', 'Read my financial
aid award and tell me what to do',
  'Card 4111 1111 1111 1111 for the bill', 'Write about my disciplinary record', 'Student 004512398 missed class, draft an
email',
```

```
];
const allowed = ['When is the add/drop deadline?', 'Explain enzyme kinetics simply', 'Make 5 practice questions on supply and
demand', 'Who do I contact about a registration hold?', 'How do I share my plan with my advisor?'];
describe('AI red-team', () => {
  for (const p of blocked) it('blocks: ' + p.slice(0, 40), () => { expect(scan(p, { studentId: SID }).block).toBe(true); });
  for (const p of allowed) it('allows: ' + p.slice(0, 40), () => { expect(scan(p, { studentId: SID }).block).toBe(false); });
  it('gate drops student-record scope even when allowed', () => { const d = gate({ task: 'plan', scope: [{ id: 'a', cls:
'course' }, { id: 'b', cls: 'student-record' }], policy, budgetLeft: 5 }); expect(d.sendScope).toEqual(['a']);
expect(d.dropped).toEqual(['b']); });
  it('institution switch off refuses everything', () => { expect(gate({ task: 'search', scope: [], policy: { ...policy,
institutionAiOn: false }, budgetLeft: 5 }).allowed).toBe(false); });
  it('course policy refuses disallowed task', () => { expect(gate({ task: 'write', scope: [], policy, budgetLeft: 5
}).allowed).toBe(false); });
  it('budget exhausted refuses', () => { expect(gate({ task: 'search', scope: [], policy, budgetLeft: 0
}).allowed).toBe(false); });
  it('redaction never leaks the match', () => { const s = scan('email me at a.student@vanderbilt.edu', {});
expect(s.text).not.toMatch(/vanderbilt\.edu/); });
});
```

handoff/semester-platform/tests/commands.test.ts · 19 lines

```
// Proves migration 060. npx supabase start && npx vitest run tests/commands.test.ts
import { describe, it, expect } from 'vitest';
import { asUser } from './helpers';
const rpc = (c, fn, args) => c.schema('api').rpc(fn, args);
describe('060 command functions', () => {
  it('direct inserts are closed', async () => { const c = await asUser('student@campus-a.test'); const { error } = await
c.schema('community').from('posts').insert({ body: 'hi' }); expect(error).not.toBeNull(); });
  it('public posts start held for pre-screen', async () => { const c = await asUser('student@campus-a.test'); const { data: id
} = await rpc(c, 'create_post', { p_group: null, p_body: 'Study group tonight?' }); const { data } = await
c.schema('community').from('posts').select('status').eq('id', id).single(); expect(data.status).toBe('held'); });
  it('non-members cannot post in a group', async () => { const c = await asUser('student2@campus-a.test'); const { error } =
await rpc(c, 'create_post', { p_group: process.env.PRIVATE_GROUP, p_body: 'x' }); expect(error?.message).toMatch(/not a
member/); });
  it('listings never go live on create', async () => { const c = await asUser('student@campus-a.test'); const { data: id } =
await rpc(c, 'create_listing', { p_title: 'BIO textbook', p_category: 'textbooks', p_price_cents: 4000 }); const { data } =
await c.schema('market').from('listings').select('status').eq('id', id).single(); expect(data.status).toBe('pending_check');
});
  it('integrity filter blocks exam material', async () => { const c = await asUser('student@campus-a.test'); const { error } =
await rpc(c, 'create_listing', { p_title: 'CHEM 101 answer key', p_category: 'textbooks', p_price_cents: 500 });
expect(error?.message).toMatch(/academic integrity/); });
  it('staff cannot list items', async () => { const c = await asUser('advisor@campus-a.test'); const { error } = await rpc(c,
'create_listing', { p_title: 'Desk', p_category: 'gear', p_price_cents: 100 }); expect(error).not.toBeNull(); });
  it('no offers on own listing', async () => { const c = await asUser('seller@campus-a.test'); const { error } = await rpc(c,
'make_offer', { p_listing: process.env.SELLER_LISTING, p_amount_cents: 100 }); expect(error?.message).toMatch(/own listing/);
});
  it('no offers across campuses', async () => { const c = await asUser('student@campus-b.test'); const { error } = await rpc(c,
'make_offer', { p_listing: process.env.SELLER_LISTING, p_amount_cents: 100 }); expect(error?.message).toMatch(/not available/);
});
  it('reports must target own campus', async () => { const c = await asUser('student@campus-b.test'); const { error } = await
rpc(c, 'report', { p_type: 'post', p_target: process.env.CAMPUS_A_POST, p_reason: 'harassment', p_note: null });
expect(error?.message).toMatch(/not found/); });
  it('rate limit holds', async () => { const c = await asUser('student@campus-a.test'); let last; for (let i = 0; i < 22; i++)
last = await rpc(c, 'create_post', { p_group: null, p_body: 'spam ' + i }); expect(last.error?.message).toMatch(/too many/);
});
  it('every command writes an audit event first', async () => { const it = await asUser('it@campus-a.test'); const { data } =
await it.schema('audit').from('events').select('action').like('action', 'market.listing.create'); expect((data ??
[]).length).toBeGreaterThan(0); });
  it('photos only under own campus and uid', async () => { const c = await asUser('student@campus-a.test'); const { error } =
await c.storage.from('listing-photos').upload(process.env.CAMPUS_B + '/' + process.env.STUDENT_A_ID + '/x.jpg', new
Blob(['x'])); expect(error).not.toBeNull(); });
});
```

handoff/semester-platform/tests/helpers.ts · 14 lines

```
// tests/helpers.ts — signed-in and anonymous clients against local Supabase.
import { createClient, type SupabaseClient } from '@supabase/supabase-js';
import 'dotenv/config';
const url = process.env.SUPABASE_URL!, key = process.env.SUPABASE_ANON_KEY!;
const cache = new Map<string, SupabaseClient>();
export async function asUser(email: string): Promise<SupabaseClient> {
  if (cache.has(email)) return cache.get(email)!;
  const c = createClient(url, key, { auth: { persistSession: false } });
```

```
  const { error } = await c.auth.signInWithPassword({ email, password: process.env.TEST_PASSWORD! });
  if (error) throw new Error('seed user missing: ' + email + ' — run npx supabase db reset');
  cache.set(email, c); return c;
}
export function anon(): SupabaseClient { return createClient(url, key, { auth: { persistSession: false } }); }
```

handoff/semester-platform/tests/rls.test.ts · 12 lines

```
// Run against local Supabase: npx supabase start && npx vitest run tests/rls.test.ts
import { describe, it, expect } from 'vitest';
import { asUser } from './helpers'; // signs in seeded users and returns a scoped client
describe('RLS boundaries', () => {
  it('ops staff read zero community posts', async () => { const c = await asUser('cfo@semester.test'); const { data } = await
c.schema('community').from('posts').select('id'); expect(data ?? []).toHaveLength(0); });
  it('students cannot read another campus', async () => { const c = await asUser('student@campus-a.test'); const { data } =
await c.schema('market').from('listings').select('institution_id'); expect((data ?? []).every(r => r.institution_id ===
process.env.CAMPUS_A)).toBe(true); });
  it('new listings cannot skip the check', async () => { const c = await asUser('student@campus-a.test'); const { error } =
await c.schema('market').from('listings').insert({ title: 'Lamp', category: 'gear', price_cents: 500, status: 'live' });
expect(error).not.toBeNull(); });
  it('sellers cannot offer on their own listing', async () => { const c = await asUser('seller@campus-a.test'); const { error }
= await c.schema('market').from('offers').insert({ listing_id: process.env.SELLER_LISTING, amount_cents: 100 });
expect(error).not.toBeNull(); });
  it('RSVPs hidden from non-leaders', async () => { const c = await asUser('student2@campus-a.test'); const { data } = await
c.schema('community').from('rsvps').select('*').neq('profile_id', process.env.STUDENT2_ID); expect(data ?? []).toHaveLength(0);
});
  it('social manager cannot approve posts', async () => { const c = await asUser('social@semester.test'); const { data } =
await c.schema('ops').from('social_posts').update({ status: 'approved' }).eq('status','needs_approval').select(); expect(data
?? []).toHaveLength(0); });
});
```

handoff/semester-platform/tests/security.test.ts · 19 lines

```
// Proves migration 050. Run against local Supabase: npx supabase start && npx vitest run tests/security.test.ts
// Each test should FAIL before 050 is applied and PASS after (record both runs in the PR).
import { describe, it, expect } from 'vitest';
import { asUser, anon } from './helpers';
describe('050 security hardening', () => {
  it('F1 students read only their own institution row', async () => { const c = await asUser('student@campus-a.test'); const {
data } = await c.schema('core').from('institutions').select('id'); expect((data ?? []).map(r =>
r.id)).toEqual([process.env.CAMPUS_A]); });
  it('F2 tenant users cannot read or insert ops.staff', async () => { const c = await asUser('student@campus-a.test'); const r
= await c.schema('ops').from('staff').select('id'); expect(r.data ?? []).toHaveLength(0); const w = await
c.schema('ops').from('staff').insert({ id: process.env.STUDENT_A_ID, role: 'ceo' }); expect(w.error).not.toBeNull(); });
  it('F4 students cannot read other profiles directly', async () => { const c = await asUser('student@campus-a.test'); const {
data } = await c.schema('core').from('profiles').select('id').neq('id', process.env.STUDENT_A_ID); expect(data ??
[]).toHaveLength(0); });
  it('F4 directory shows only findable people and visible fields', async () => { const c = await asUser('student@campus-
a.test'); const { data } = await c.schema('core').from('directory').select('*'); expect((data ?? []).every(r => !('role' in r)
&& !('visibility' in r))).toBe(true); });
  it('F5 users cannot move themselves to another institution', async () => { const c = await asUser('student@campus-a.test');
const { error } = await c.schema('core').from('profiles').update({ institution_id: process.env.CAMPUS_B }).eq('id',
process.env.STUDENT_A_ID); expect(error).not.toBeNull(); });
  it('F5 users cannot change their role', async () => { const c = await asUser('student@campus-a.test'); const { error } =
await c.schema('core').from('profiles').update({ role: 'registrar' }).eq('id', process.env.STUDENT_A_ID);
expect(error).not.toBeNull(); });
  it('F6 clients cannot call audit.log', async () => { const c = await asUser('student@campus-a.test'); const { error } = await
c.schema('audit').rpc('log', { a: 'fake', t: 'x', b: {}, af: {} }); expect(error).not.toBeNull(); });
  it('F7 audit rows cannot be deleted', async () => { const c = await asUser('it@campus-a.test'); const { data } = await
c.schema('audit').from('events').delete().neq('id', 0).select(); expect(data ?? []).toHaveLength(0); });
  it('F8 bidders are not visible to other students', async () => { const c = await asUser('student2@campus-a.test'); const {
data } = await c.schema('market').from('bids').select('bidder'); expect((data ?? []).every(r => r.bidder ===
process.env.STUDENT2_ID)).toBe(true); });
  it('F9 non-members cannot list group members', async () => { const c = await asUser('student2@campus-a.test'); const { data }
= await c.schema('community').from('members').select('profile_id').eq('group_id', process.env.PRIVATE_GROUP); expect(data ??
[]).toHaveLength(0); });
  it('F10 cannot RSVP to another campus event', async () => { const c = await asUser('student@campus-a.test'); const { error }
= await c.schema('community').from('rsvps').insert({ event_id: process.env.CAMPUS_B_EVENT, profile_id: process.env.STUDENT_A_ID
}); expect(error).not.toBeNull(); });
  it('F11 anonymous clients read nothing', async () => { const c = anon(); for (const [s, t] of [['core','profiles'],
['community','posts'],['market','listings'],['ops','accounts']]) { const { data } = await
```

```
c.schema(s).from(t).select('*').limit(1); expect(data ?? []).toHaveLength(0); } });
});
```

handoff/semester-platform/tests/validate.test.ts · 8 lines

```
import { describe, it, expect } from 'vitest';
import { integrityPrecheck, Listing } from '../lib/validate';
describe('integrity precheck', () => {
  it('holds answer keys', () => expect(integrityPrecheck('CHEM 210 exam 2 answers').hold).toBe(true));
  it('allows textbooks', () => expect(integrityPrecheck('Organic Chemistry 9th ed').hold).toBe(false));
});
describe('Listing schema', () => { it('rejects negative price', () => expect(Listing.safeParse({ title: 'Lamp', category:
'gear', priceCents: -1 }).success).toBe(false)); });
```